# פרויקט סופי רשתות תקשורת

סמסטר א', מועד הגשה 16.03.2026

מגישים:
דניאל סטציוק 325174589, אור מצליח 212105209, איתיאל הלוי 329522411.

---

**הוראות הפעלה.....2**
הרצת שרת DHCP:.....2
הרצת שרת DNS:.....3
הרצה מאוחדת.....6
**מידע כללי.....11**
האפליקציה שבחרנו.....11
הסבר ארכיטקטורה.....12
**DHCP.....13**
לוגיקת ניהול הכתובות בשרת (IP Allocation).....18
מנגנוני הגנה וגיבוי (Reliability).....20
דגשים במימוש פרוטוקול.....20
**DNS.....32**
מבנה Question Section.....34
מימוש השרתים עצמם.....43
**Application.....57**
פרוטוקול DASH:.....63
**RUDP.....65**
מפרט פרוטוקול.....66
**שאלות עיוניות.....88**
**בונוס.....91**
DNSSEC.....91
API Server.....101
Electron - GUI (DNS).....102
**WIRESHARK:.....105**
SERVER + PROXY:.....105
DROPPED PACKETS.....111
ARP:.....112

# הוראות הפעלה

מסמך זה מרכז את הנחיות ההפעלה עבור רכיבי ה-**DHCP**, ה-**DNS** והאפליקציה (**Application**). הוראות מפרטות את תהליך ההרצה עבור כל רכיב באופן עצמאי, וכן עבור הפרויקט כמערכת אחודה מסונכרנת.

### דרישות מערכת וסביבת עבודה

- **מערכת הפעלה:** הפרויקט פותח, עבר אופטימיזציה ונבדק במערכת **Windows 11**, וזוהי סביבת העבודה המומלצת לבדיקת הקוד. אף על פי הגרסאות הקודמות הורצו בסביבות:
  Linux Ubuntu 22.04 ו-macOS, הרצה במערכות אלו עשויה לדרוש הרשאות מערכת נוספות והתאמות קונפיגורציה ייעודיות.
- **שפות תכנות וגרסאות:**
  - **Python:** גרסה 3.10 ומעלה.
  - **Node.js:** גרסה 11.0 ומעלה.

---

### התקנת תלויות (Dependencies)

#### סביבת Python

לצורך הגדרת סביבת העבודה ב-Python, יש להתקין את כלל חבילות התוכנה הנדרשות המופיעות בקובץ **requirements.txt**.

#### סביבת Node.js וממשק ה-DNS

רכיב ה-DNS בפרויקט זה כולל ממשק משתמש גרפי (**GUI**) המבוסס על פלטפורמת **Electron**. לצורך התקנת רכיבי הליבה והתלויות של הממשק, יש לנווט לספרייה: **DNS/GUI_Client** ולבצע התקנה של חבילות המקור באמצעות הפקודה: **npm install**

---

## הרצת שרת DHCP:

לשם הפעלה של שרת DHCP יש לממש את הצעדים הבאים:

```
cd .\dhcp
python .\dhcp_server.py
```

לאחר מכן יהיה מודפס:

```
[*] DHCP Server active on Port 6700
```

## הרצת שרת DNS:

לשם הפעלת שרת הDNS יש לממש את הצעדים הבאים:

1. מעבר לתיקיית DNS מהתיקייה הראשית:

```
cd DNS
```

2. הפעלת התוכנית:

```
python .\main.py
```

לאחר מכן ניתן יהיה לראות הודעה הבאה מודפסת בתוך הקונסולה:

```
==================================================
[*] Initiating Custom DNS Infrastructure Boot Sequence...
[*] Mode: STANDARD INSECURE MODE
==================================================

[*] Launching Root Server...
[*] Booting Local Root Server...
[*] Loaded 5 records from root.zone
[*] Successfully loaded 5 total records across 1 files.
[*] Server Active on 127.0.0.3:53 (Max Workers: 100)

[*] Launching TLD Server...
[*] Booting Local TLD Server (.homelab)...
[*] Loaded 5 records from homelab.zone
[*] Successfully loaded 5 total records across 1 files.
[*] Server Active on 127.0.0.4:53 (Max Workers: 100)

[*] Launching Auth Server...
[*] Booting Local Auth Server...
[*] Loaded 4 records from originserver.homelab.zone
[*] Loaded 7 records from test.homelab.zone
[*] Successfully loaded 11 total records across 2 files.
[*] Server Active on 127.0.0.5:53 (Max Workers: 100)

[*] Launching Resolver...
[*] Booting Resolver Manager...
[*] Loaded 0 records from
D:\YourPath\network_final\Networks_Project\DNS\resolver\data\dns_cache.db

========================================
[*] DNS Resolver Online
[*] Cache Path:
D:\YourPath\network_final\Networks_Project\DNS\resolver\data\dns_cache.db
========================================
```

```
[*] Listener thread started on 127.0.0.2:53
[*] API Server booting on http://127.0.0.1:8000

==================================================
[*] ALL SYSTEMS ONLINE AND LISTENING
[*] Press Ctrl+C at any time to gracefully stop all servers.
==================================================
```

במידה ונרצה לעצור את הריצה נלחץ על ctrl+c, תודפס ההודעה הבאה:

```
[*] ResolverManager initiating shutdown sequence...
[*] API Server shutting down...
[*] All subsystems offline. Goodbye.

==================================================
[*] Master kill signal received. Shutting down infrastructure...
==================================================
[*] Terminating Root Server...
[*] Terminating TLD Server...
[*] Terminating Auth Server...
[*] Terminating Resolver...

[*] All DNS services are offline. Goodbye!
```

3. הפעלת ה gui, דורש node.js מותקן (שימו לב, זה בונוס ואינו הכרחי לתפקוד שרת הDNS):
יש לעבור לתיקיה של ה GUI קודם:

```
cd .\DNS\GUI-Client\
```

יש לוודא שהתקנתם את כל הספריות קודם, על ידי הרצת הפקודה:

```
DNS\GUI-Client> npm install
```

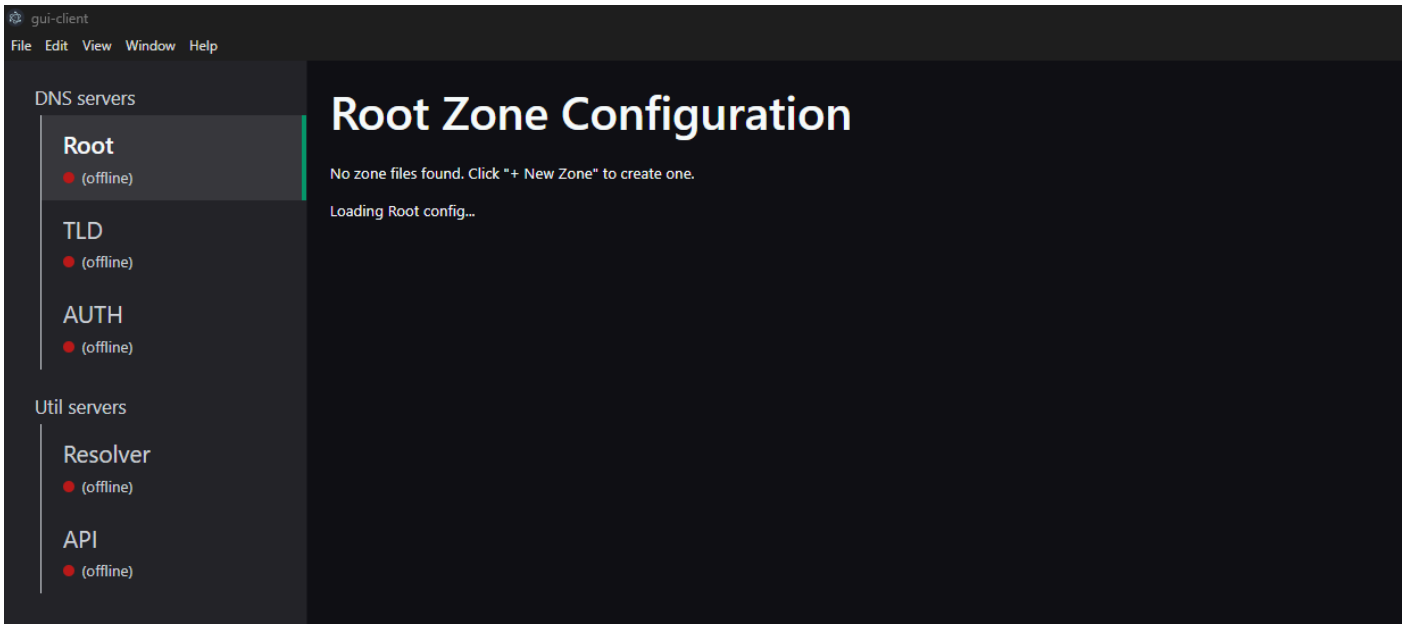
לאחר ההתקנה תופיע ההודעה:

```
up to date, audited 616 packages in 4s

166 packages are looking for funding
  run `npm fund` for details
```

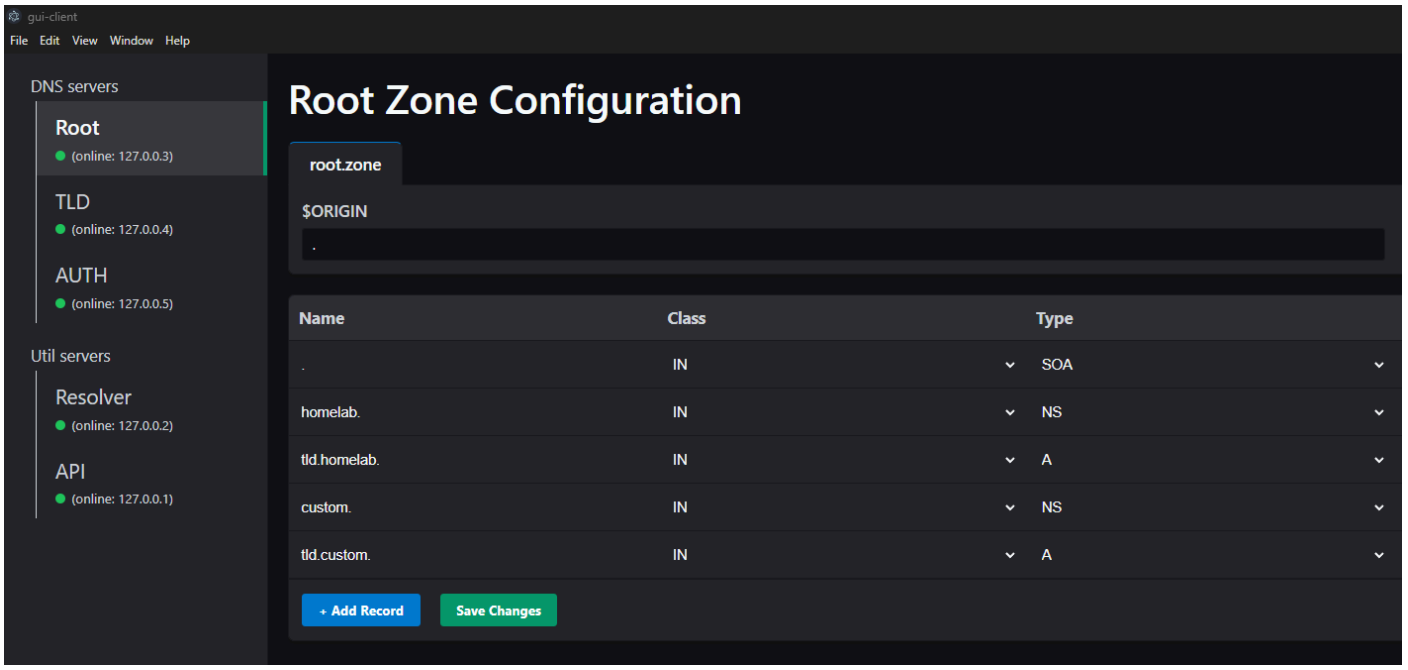
כעת ניתן להפעיל את הGUI:

```
DNS\GUI-Client> npm run dev
```

לאחר מכן אמורה להיפתח התוכנה של Electron:




אם השרת ה DNS כבוי זה יראה כמו התמונה העליונה, במידה והשרת פועל ורץ, הוא יראה באופן הבא:




**אזהרה - שימו לב!**

על מנת לסגור את הריצה של התוכנית של ה GUI אינו מספיק לבצע סגירה של התוכנית על ידי לחיצה על ה-X אלא נדרש גם ללחוץ **ctrl+c** בקונסולה עצמה.

## הרצה מאוחדת:

1. נריץ את הקובץ run_system.ps1 על ידי הרצת הפקודה בתוך PowerShell:

```
./run_system.ps1
```

2. הקובץ יריץ את כל הקבצים הרלוונטים שלב אחרי שלב:
   - **שלב 1:** הרצת שרתי הDNS והרצת שרת הDHCP
     - תוצג הודעה על תחילת שלב 1 - ויוצג האם יש שימוש במצב dnssec

```
[1/3] Starting DHCP and Main...
==================================================
[*] Initiating Custom DNS Infrastructure Boot Sequence...
[*] Mode: STANDARD INSECURE MODE
==================================================
```

     - יופעל שרת ה-Root בשילוב טעינת הconfig

```
[*] Launching Root Server...
[*] Booting Local Root Server...
[*] Loaded 5 records from root.zone
[*] Successfully loaded 5 total records across 1 files.
[*] Server Active on 127.0.0.3:53 (Max Workers: 100)
```

     - יופעל שרת ה-TLD בשילוב טעינת הconfig

```
[*] Launching TLD Server...
[*] DHCP Server active on Port 6700
[*] Booting Local TLD Server (.homelab)...
[*] Loaded 5 records from homelab.zone
[*] Successfully loaded 5 total records across 1 files.
[*] Server Active on 127.0.0.4:53 (Max Workers: 100)
```

     - יופעל שרת ה-Auth בשילוב טעינת הconfig

```
[*] Launching Auth Server...
[*] Booting Local Auth Server...
[*] Loaded 4 records from originserver.homelab.zone
[*] Loaded 7 records from test.homelab.zone
[*] Successfully loaded 11 total records across 2 files.
[*] Server Active on 127.0.0.5:53 (Max Workers: 100)
```

     - יופעל הresolver

```
[*] Launching Resolver...
```

   - **שלב 2:** הרצת השרת והproxy לצפייה בסרטונים
     - תוצג הודעה על תחילת שלב 2

```
[2/3] Starting Unified Services (Multi-Protocol Mode)...
-----------------------------------------
[*] SYSTEM RUNNING.
[*] Access the Library at http://localhost:5000
[*] Press CTRL+C to stop all services.
-----------------------------------------
```

- ייטען הcache של הdns בצירוף הדפסת הpath שלו:

```
[*] Booting Resolver Manager...
[*] Loaded 0 records from
C:\YourPath\Networks_Project\DNS\resolver\data\dns_cache.db

========================================
[*] DNS Resolver Online
[*] Cache Path: C:\YourPath\Networks_Project\DNS\resolver\data\dns_cache.db
========================================
```

- תודפס כתובת הresolver וכתובת לשרת הAPI (ניתן לגשת לשרת ה-API דרך הדפדפן)

```
[*] Listener thread started on 127.0.0.2:53
[*] API Server booting on http://127.0.0.1:8000
```

- שרת הווידאו יפנה לשרת הDHCP לקבלת הכתובת שלו 127.0.0.13 (שמורה לפי זיהוי)

```
[DISCOVER] Received from PID: OriginServer at ('127.0.0.1', 56978)
 |---> [OFFER] Sending 127.0.0.13 to PID: OriginServer

[REQUEST] Received from PID: OriginServer
 |---> [ACK] Lease Confirmed for 127.0.0.13
[V] DHCP Success: OriginServer assigned 127.0.0.13
╫ UNIFIED ORIGIN SERVER LISTENING ON 127.0.0.13:9000 ╫
```

- נקבל הודעה על סיום הרצת התשתית

```
==================================================
[*] ALL SYSTEMS ONLINE AND LISTENING
[*] Press Ctrl+C at any time to gracefully stop all servers.
==================================================
```

- קביעת כתובת IP דרך שרת הDHCP וקבלת כתובת url דרך הresolver לקביעת הפרטים על שרת הproxy שיישמש לצפייה בסרטונים דרך הדפדפן

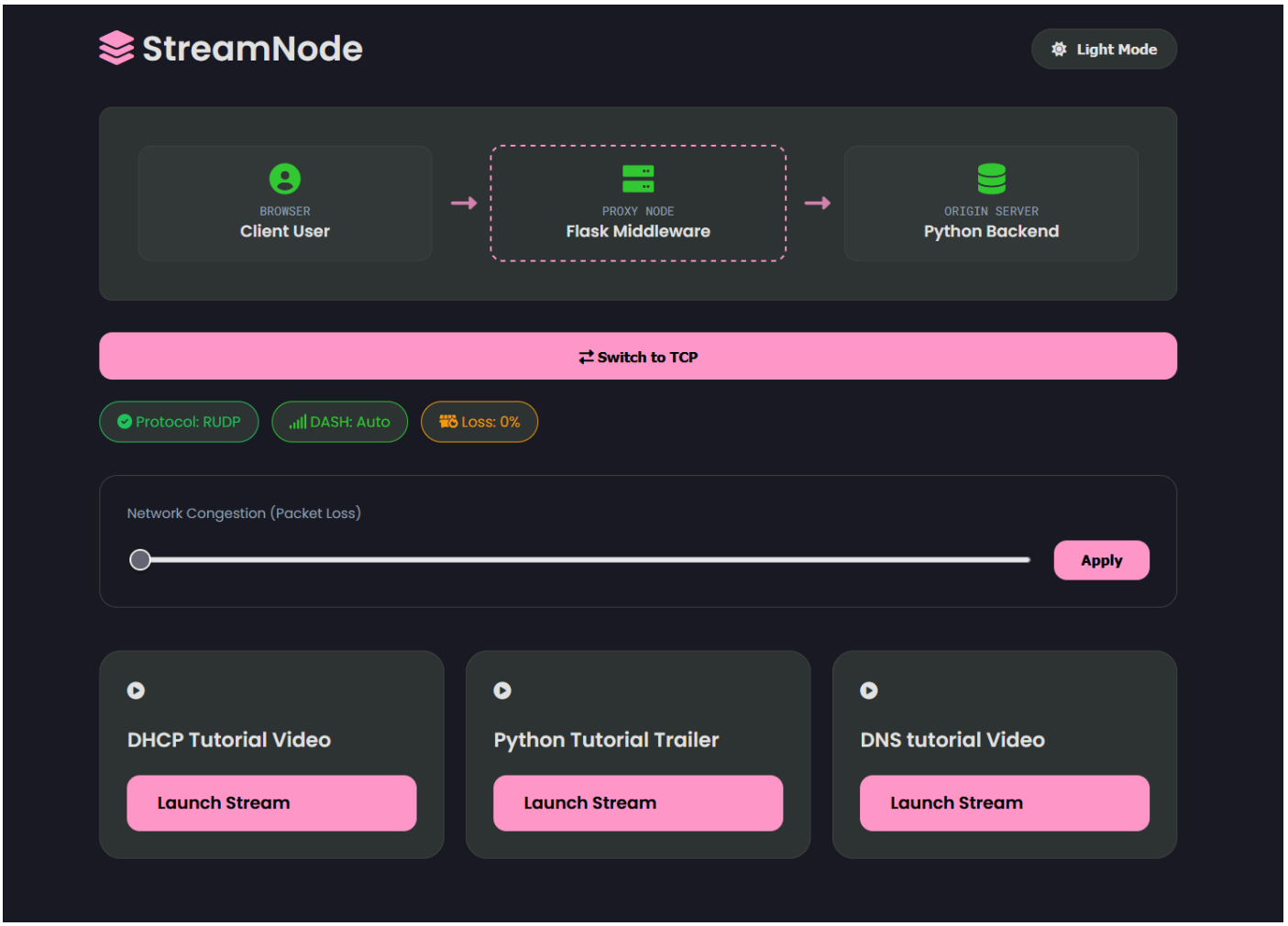
```
[DNS] 🔍 Resolving originserver.homelab via DNS (127.0.0.2:53)...
[DEBUG] Thread 29568 started processing ('127.0.0.1', 56980)
[CACHE MISS] Resolving A originserver.homelab....
    [Iterative Jump 1] Asking 127.0.0.3 for originserver.homelab.
[*] Delegating originserver.homelab. to .homelab. nameservers.
    [*] Delegation received. Moving to: 127.0.0.4
    [Iterative Jump 2] Asking 127.0.0.4 for originserver.homelab.
[*] Delegating originserver.homelab. to originserver.homelab. nameservers.
    [*] Delegation received. Moving to: 127.0.0.5
    [Iterative Jump 3] Asking 127.0.0.5 for originserver.homelab.
[*] ANSWER: Appended A record for originserver.homelab.
    [*] Final exact answer found at 127.0.0.5!
[*] Client did not request DNSSEC. Bypassing validation math for
originserver.homelab..
[DNS] ✅ Successfully resolved originserver.homelab to 127.0.0.13:9000

[DISCOVER] Received from PID: ProxyNode-31244-4493 at ('127.0.0.1', 56984)
 |---> [POOL] Assigned 127.0.0.31 to PID: ProxyNode-31244-4493
 |---> [OFFER] Sending 127.0.0.31 to PID: ProxyNode-31244-4493

[REQUEST] Received from PID: ProxyNode-31244-4493
 |---> [ACK] Lease Confirmed for 127.0.0.31
[V] DHCP Success: ProxyNode-31244-4493 assigned 127.0.0.31
```

- בסיום ההליך נקבל הודעה מסודרת על פרטי הproxy ביחד עם הפרמטרים שנקבעו:

```
╔═══════════════════════════════════════════════════════════════════════════╗
║═══════════════════════════════════════════════════════════════════════════╕
╒═══════════════════════════════════════════════════════════════════════════╕
═══════════════════════════════════════════════════════════════════════════
```

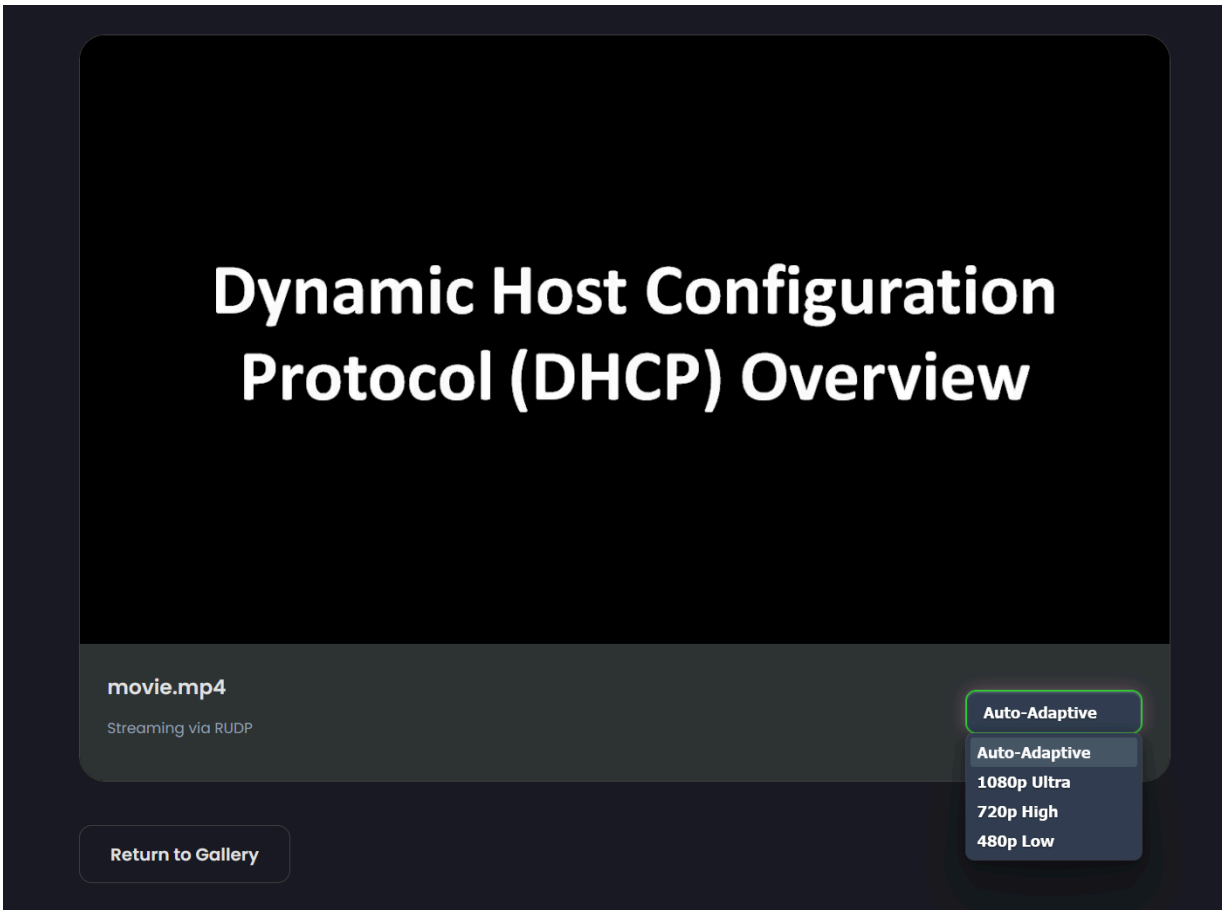
```
Component Summary:
  1. DHCP Server    -> Assigns IPs to clients
  2. DNS Server     -> Resolves domain names
  3. Origin Server  -> Streams video files
  4. Proxy          -> Edge caching '&' load balancing

Access the system at: http://127.0.0.30:5000

Shutdown Instructions:
  Press CTRL+C to gracefully stop all services
```

3. גישה לאתר הproxy לצפייה בסרטונים:
   - נפתח דפדפן לבחירתנו ונכניס אליו את הכתובת מהסיכום תחת Listening.




חשוב **לשים לב** - הכתובת המודפסת עלולה להשתנות - בהתאם לכתובת שנקבל משרת ה-DHCP.

- באתר מוצג גרף מופשט של התקשורת של ה-client מול הproxy והתקשורת של הproxy אל מול השרת שמחזיק את הסרטונים. נוכל לבחור להחליף פורטוקול (לתקשורת של הproxy מול השרת) להגדיר את אחוז איבוד חבילות ולבחור 1 מתוך 3 הסרטונים לצפייה. לאחר בחירת אחד הסרטונים על ידי לחיצת Launch Stream יוצג לנו נגן וידאו.



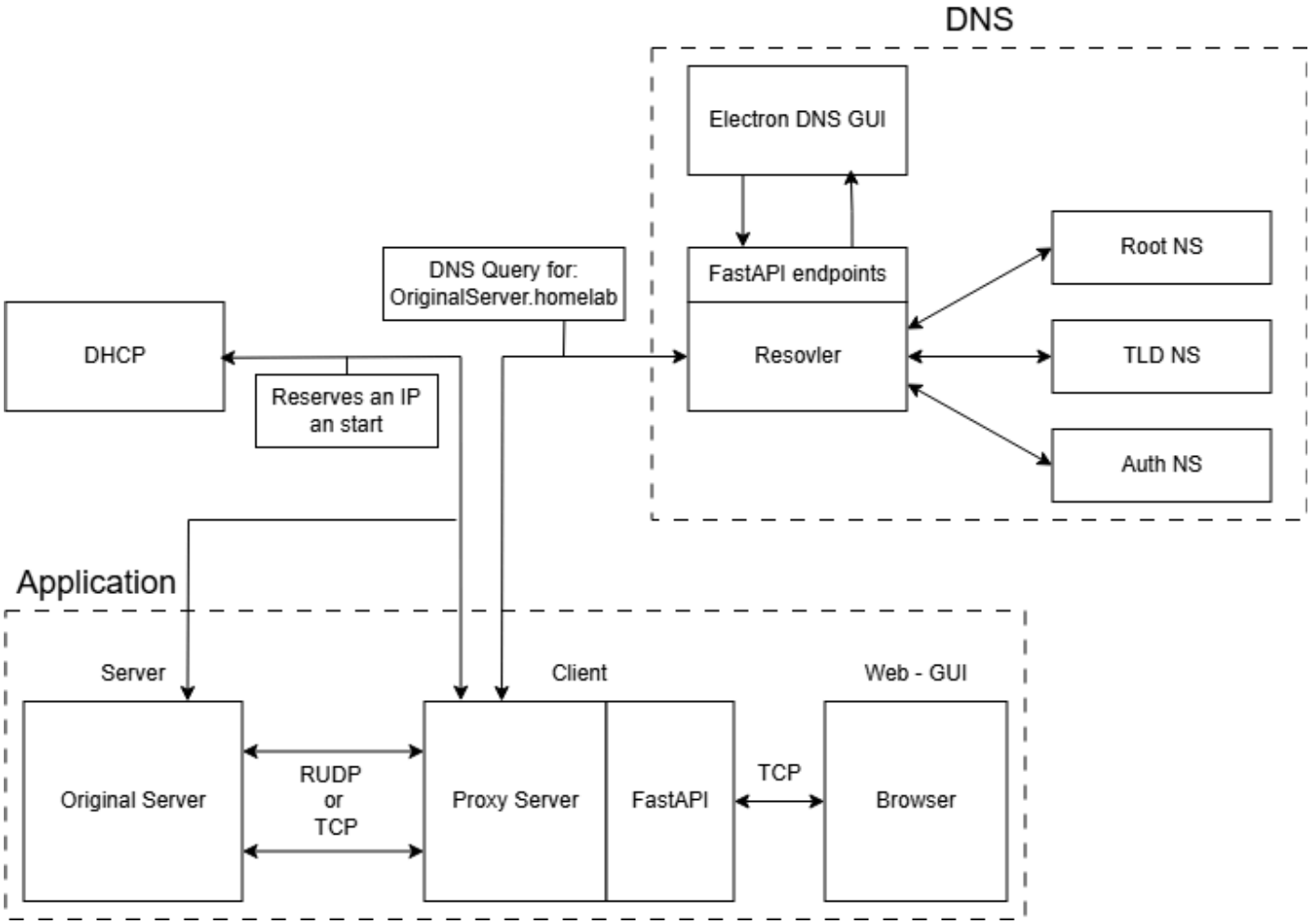

- במידה ובחרנו בפרוקטול RUDP נוכל לבחור לקבוע את איכות הסרטון לאחת מהאופציות הבאות
  - Auto-Adaptive - בעת תחילת הסרטון תתבצע בדיקה לתנאי הרשת - ועל פיה תיקבע אוטומטית האיכות
  - איכות ידנית (1080p Ultra, 720p High, 480p Low).
- במידה ובחרנו בפרוטוקול TCP לא תינתן אופציה לבחירת איכות.

---

# מידע כללי

## האפליקציה שבחרנו:

לפרויקט זה בחרנו לממש Media Player, שניתן להריץ יהיה בדפדפן המחובר למחשב.
לשם הסבר הפרויקט נסתכל על הדיאגרמה המופשטת הבאה:




## הסבר שלבים:

1) יאותחל DHCP וגם DNS (כל השרתים שלו)
2) כעת יתחיל Original Server ויבקש את הכתובת 127.0.0.13, הכתובת הזאת היא כתובת שמורה עבורו בDHCP.
3) לאחר מכן יופעל Proxy Server וגם כן יבקש כתובת מה DHCP ויקבל בטווח של **127.0.0.30-100**
4) לאחר מכן, Proxy Server ישלח Recursive Query ל Resolver לקבלת הכתובת של Original Server.
5) ה Resolver בעת קבלת Message מתחיל בתהליך ה"Walk The Tree" של ה DNS, ועובר על כל העץ עד שהוא מוצא את הA records של Original Server (אנחנו מחזיקים רק כתובות IPV4 ב zone).
6) לאחר מכן ה Resolver מאתחל את התשובה ב cache ומחזיר תשובה לProxy Server
7) שרת ה Proxy Server פותח חיבור ב RUDP בעת אתחול, אבל ניתן להחליף את החיבור ל TCP.
8) מתבצע תהליך של fragmentation של הסרטון לחלקים מהOrigin לProxy, ומהפרוקסי ל- Browser.

# הסבר ארכיטקטורה:

## הסבר קצר על מימוש DHCP

עבור שרת ה DHCP, ביצענו שרת בסיסי, יש לו שני מצבים:

1. מצב רגיל
2. מצב LoopBack

**במצב רגיל** - אנחנו משתמשים בPool של כתובות ip בטווח 192.168.1.**120-200** אלו הם כתובות IP שמכשירים ברשת המקומית יכולים לקבל, זאת בתנאי ששרת ה DHCP יהיה יותר מהיר מהשרת הDHCP אמיתי אשר נמצא ברשת שלכם.

**במצב LoopBack** - אנחנו משתמשים בPool של כתובות ip בטווח 127.0.0.**1-100** אלו הם כתובות LoopBack, שום מכשיר לא יכול להשתמש בהם בתור כתובת אמיתית ברשת מקומית, אבל זה הדרך שאנחנו בחרנו להתגבר על מירוצים עם שרתי DHCP אמיתיים ברשתות מקומיות, מבחינה רעיונית זה לא שונה מלחלק כתובת אמיתית וזה מקטין את הסיכוי שזה לא יעבוד המחשב של הבודק.

| **הערה: בעת הרצה של האפליקציה אנחנו מפעילים את DHCP במצב LoopBack** |
|---|

עבור הסבר מעמיק עבור השרת DHCP יש לעבור לקטגוריה DHCP לחץ כאן

---

## הסבר קצר על מימוש הDNS

עבור שרתי DNS ביצענו תהליך סימולציה של walk the tree של DNS Resolver, סה״כ יש לנו:

- שרת Resolver על הכתובת 127.0.0.2
- שרת Root על הכתובת 127.0.0.3
- שרת TLD על הכתובת 127.0.0.4
- שרת AUTH על הכתובת 127.0.0.5

בנוסף אנחנו מרצים שרת FastAPI עם endpoints על 127.0.0.1:8000 כדי לאפשר לGUI שיצרנו בElectron לתקשר עם שרת ה DNS, עבור כל שרת של DNS קיים קובץ קונפיג שניתן לערוך בעזרת ה GUI וקיים לכל Name Server קובץ Zone אשר מכיל את Records והכתובות שהוא אחראי עליהם.

עבור הסבר מעמיק עבור השרת DNS יש לעבור לקטגוריה של DNS לחץ כאן

---

## הסבר קצר על מימוש הApplication

האפליקציה שאנחנו מממשים היא Media Player, אבל הClient שלנו הוא לא הmedia player עצמו שנמצא בbrowser, אלא זה השרת הפרוקסי של המדיה פלייר אשר מבצע את כל התקשורת מול Original Server שהוא השרת שבעצם מחזיק את כל הסרטונים של המדיה, והמדיה פלייר פשוט מוצג באמצעות Fast API בBrowser ב TCP. כלומר, בפרויקט זה:

| Client: proxy_Server | Server: Original_Server |
|---|---|

# DHCP

פרוטוקול ה-**DHCP** מבוסס על ארכיטקטורת שרת-לקוח וייעודו להקצות כתובות IP והגדרות רשת למכשירים באופן אוטומטי. לצורך כך, הוגדר בתקני ה-RFC תהליך ה-**DORA**, המורכב מארבעה שלבים עוקבים:

| | |
|---|---|
| Discover | **(גילוי):** הלקוח שולח הודעת **Broadcast** לכתובת הכללית `255.255.255.255`, מאחר שטרם הוגדרה לו כתובת רשת. בשלב זה, כתובת ה-IP של השולח היא `0.0.0.0`.<br>בשכבת הנתונים (**Layer 2**), כתובת ה-MAC של היעד מוגדרת כולה כ-`1` (FF:FF:FF:FF:FF:FF) כדי להפיץ את ההודעה לכלל המכשירים ברשת. |
| Offer | **(הצעה):** שרת ה-DHCP מזהה את הבקשה ומשיב בחבילת הצעה הכוללת:<br>● כתובת IP פנויה<br>● מסיכת רשת (Subnet Mask)<br>● שער ברירת מחדל (Default Gateway)<br>● זמן השאלה (Lease Time)<br>השרת DHCP מנתב את התשובה חזרה ללקוח על בסיס כתובת ה-**MAC** וה-**Transaction ID** הייחודי שצורפו לבקשה המקורית. |
| Request | **(בקשה):** הלקוח מאשר את קבלת ההצעה ושולח בקשה רשמית (Formal Request) להקצאת כתובת ה-IP שהוצעה לו על ידי השרת. |
| Acknowledge | **(אישור):** השרת מאשר סופית את הבקשה (ACK) ושולח אישור ללקוח להתחיל להשתמש בכתובת ה-IP שהוקצתה לו תחת תנאי ההשאלה שנקבעו. |

---

## מנגנוני אישור, סירוב ובדיקת התנגשויות

תהליך הקצאת הכתובת אינו מסתיים בקבלת ההצעה (Offer), שכן קיימים מקרים בהם הלקוח או השרת יבטלו את ההצעה לטובת הבטחת תקינות הרשת:

- **בדיקת התנגשויות מצד הלקוח (Gratuitous ARP):** בהתאם להמלצות ה-RFC, על הלקוח לוודא שהכתובת המוצעת אינה בשימוש טרם אישורה. הבדיקה מתבצעת באמצעות שליחת בקשת **ARP**. אם מכשיר אחר ברשת משיב לבקשה, הלקוח מזהה כי קיימת התנגשות כתובות ושולח לשרת הודעת Message Type 4 **(DHCPDECLINE)**. בעקבות זאת, השרת מסמן את הכתובת כתפוסה ומונע את הקצאתה העתידית.
- **בחירה בין שרתים מרובים (Option 54):** בסביבה הכוללת מספר שרתי DHCP, הלקוח עשוי לקבל מספר הצעות במקביל. לרוב, הלקוח יבחר בהצעה הראשונה שהגיעה. עם הבחירה, הלקוח משדר הודעת **DHCPREQUEST** ב-Broadcast הכוללת את כתובת השרת הנבחר בתוך שדה **Option 54**. שאר השרתים, המאזינים לשידור זה, יזהו כי הצעתם לא התקבלה וישחררו את הכתובת ששמרו חזרה למאגר (Pool).
- **אימות מקדים מצד השרת (Server-side Check):** כמנגנון הגנה נוסף, מומלץ שגם השרת יבצע בדיקה אקטיבית לפני שליחת הצעה ללקוח. השרת שולח בקשת Ping **(ICMP ECHO)** לכתובת המיועדת אם מתקבל מענה, השרת מבין שהכתובת נתפסה באופן ידני או סטטי, מסמן אותה כלא-זמינה ובוחר כתובת אחרת מהמאגר.

- **מדיניות השאלה (Lease Policy):** מרגע שהוקצתה כתובת והתהליך הושלם (ACK), הכתובת שייכת למכשיר הלקוח במשך כל זמן ההשאלה המוגדר. השרת מבטיח כי לא תתבצע הקצאה כפולה של כתובת זו למכשיר אחר כל עוד ההשאלה בתוקף.

## מבנה פקטת הDHCP:

**DHCP Packet Format:**

32 bits

| | | | |
|---|---|---|---|
| Operation Code | Hardware Type | Hardware Address Length | Hops |
| Transaction ID | | | |
| Seconds | | Flags | |
| Client IP Address (CIAddr) | | | |
| "Your" (Client) IP Address (YIAddr) | | | |
| Server IP Address (SIAddr) | | | |
| Gateway IP Address (GIAddr) | | | |
| Client Hardware Address (CHAddr) (16 bytes) | | | |
| Server Host Name (Optional) (64 bytes) | | | |
| File (128 bytes) | | | |
| Options (variable size) | | | |

- Operation Code

- 1 - BOOTREQUEST (בקשה)
- 2 - BOOTREPLY (תשובה)

Hardware Type - על פי שהוגדר בRFC מספר 1700

```
Number Hardware Type (hrd)                          References
------ ----------------------------------           ----------
     1 Ethernet (10Mb)                                    [JBP]
     2 Experimental Ethernet (3Mb)                        [JBP]
     3 Amateur Radio AX.25                                [PXK]
     4 Proteon ProNET Token Ring                          [JBP]
     5 Chaos                                              [GXP]
     6 IEEE 802 Networks                                  [JBP]
     7 ARCNET                                             [JBP]
     8 Hyperchannel                                       [JBP]
     9 Lanstar                                             [TU]


Reynolds & Postel                                          [Pa
---------------------------------------------------------------
RFC 1700                    Assigned Numbers               Octob


    10 Autonet Short Address                             [MXB1]
    11 LocalTalk                                         [JKR1]
    12 LocalNet (IBM PCNet or SYTEK LocalNET)             [JXM]
    13 Ultra link                                        [RXD2]
    14 SMDS                                              [GXC1]
    15 Frame Relay                                        [AGM]
    16 Asynchronous Transmission Mode (ATM)              [JXB2]
    17 HDLC                                               [JBP]
    18 Fibre Channel                            [Yakov Rekhter]
    19 Asynchronous Transmission Mode (ATM)      [Mark Laubach]
    20 Serial Line                                        [JBP]
    21 Asynchronous Transmission Mode (ATM)              [MXB1]
```

**Hardware Address Length** - אורך הכתובת הפיזית באוקטטות.

**Hops** - מסמל את כמות הקפיצות שלקח להגיע לשרת הDHCP הלקוח ימלא בבקשה הראשונית ב-0 וכל נתב מתווך (שנקרא relay agent) יעלה את הערך ב-1. שדה זה נועד למקרים שבהם שרת הDHCP אינו ברשת המקומית של הלקוח. בהגעה לערך hop מסויים (תלוי בהגדרה) החבילה תיזרק למניעת לולאה אינסופית.

**Transaction ID** - זיהוי ייחודי למכשיר המדבר, עוזר לזיהוי חוזר של חבילת ההצעה שמגיעה חזרה משרת הDHCP

**Seconds** - שדה שמסמל את כמות השניות שעברו מאז הלקוח התחיל את התהליך (בקשה לכתובת חדשה\חידוש הכתובת).

מתי זה יכול להיות שימושי?

לפעמים יהיו מספר שרתי DHCP כדי לאפשר עמידה בעת קריסת השרת המקורי.

שרתים אחרים יוכלו לראות את הבקשות, ואם יעבור זמן מסוים (על פי שהוגדר אצלהם בקוד) יוכלו להבין שכנראה השרת הראשי קרס ולתת תשובה במקומו.

**Flags** - שדה דגלים בגודל 16 ביטים. הדגל הראשון (ביט 0) הוא דגל broadcast. שאר הביטים חייבים להיות 0 על פי המוגדר ב-RFC והם שמורים לשימוש עתידי.

**CIAddr** - במידה וללקוח כבר יש כתובת חוקית (למשל אם הוא מבקש לחדש את החכירה של הכתובת) ימלא את הכתובת שהוא מכיר

**YIAddr** - כתובת הIP של הלקוח, שדה שממולא על ידי השרת, בכתובת שמקצה ללקוח. הלקוח ישאיר את השדה הזה מאופס (כי הוא לא מכיר את הכתובת של עצמו מראש)

**SIAddr** - כתובת השרת להמשך התהליך. יכול להיות שאת כתובת ה-IP קיבלנו משרת אחד, ואת שאר ההגדרות נקבל משרת אחר. בעת שליחת הבקשה הלקוח ימלא שם 0.0.0.0 אם יש המשך - השרת ימלא את הכתובת הרלוונטית בתשובה, אחרת ימלא 0.0.0.0. נשים לב שלמרות שמו המבלבל - השדה הזה אינו נועד לזיהוי שרת ה-DHCP

**GIAddr** - כתובת IP של relay agent - במידה ומשמשיכים את תהליך ההגדרה אל מול מכשיר אחר

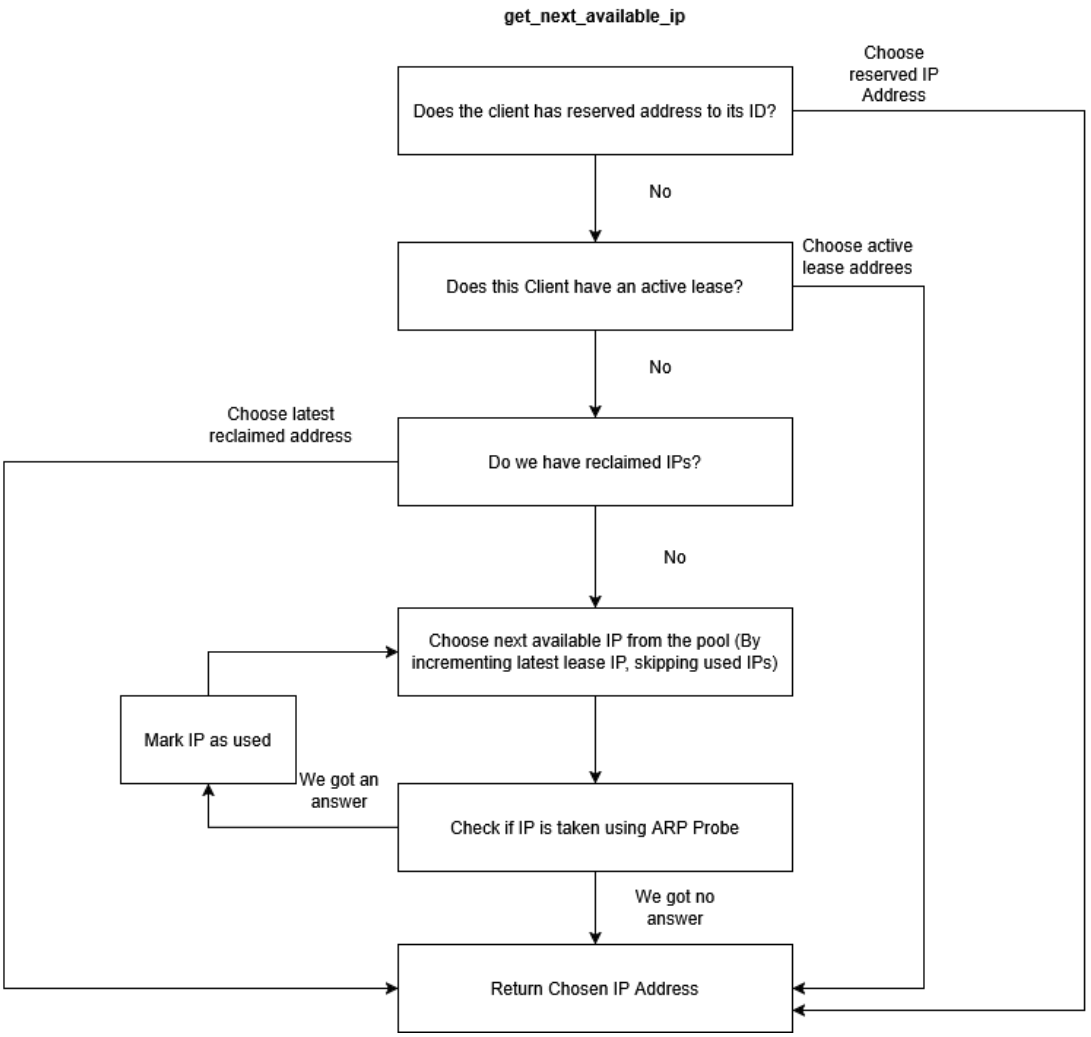
**CHAddr** - כתובת פיזית של הלקוח (כתובת MAC).

**Server Host Name** - שדה אופציונלי - לשם הhost של השרת. נגמר בnull terminator

**File** - שם קובץ ההגדרה - נגמר בnull terminator (המשך לשדה GIAddr).

**Options** (מפורט בRFC 1533) - שדה באורך משתנה המגדיר אפשרויות שונות. מבנה האפשרות יהיה אוקטטה ראשונה שמסמלת את מספר האופציה, ולאחר מכן האוקטטות הבאות בהתאמה יהיו הערך. יש 2 הגדרות שהן חובה:

- אפשרות 53 - סוג הודעת הDHCP.
  - 1 - DHCPDISCOVER - נשלח על ידי לקוח בתחילת התהליך.
  - 2 - DHCPOFFER - נשלח על ידי השרת ללקוח ובחבילה הצעה לכתובת IP
  - 3 - DHCPREQUEST - נשלח כדי לבקש סופית את כתובת הIP שהוצעה או כדי לחדש חכירה.
  - 4 - DHCPDECLINE - נשלח על ידי הלקוח כאשר הכתובת המוצעת תפוסה
  - 5 - DHCPACK - אישור מהשרת על חכירת הכתובת וכוללת את שאר ההגדרות
  - 6 - DHCPNAK - נשלח על ידי השרת לסמן שהכתובת המבוקשת אינה חוקית, או שהחכירה הסתיימה
  - 7 - DHCPRELEASE - נשלח על ידי הלקוח לסמן לשרת שסיים להשתמש בכתובת ואפשר לשחרר אותה
  - 8 - DHCPINFORM - נשלח על ידי לקוח שכבר מוגדרת לו כתובת IP - ומשמש לבקשת הגדרות נוספות (כמו למשל DNS)
- אפשרות 255 - תפקידה לסמן את סוף רשימת האופציות. כל מה שיופיע לאחר מכן השרת יתעלם ממנו.




## בנוסף יש הגדרות רשות שמימשנו:

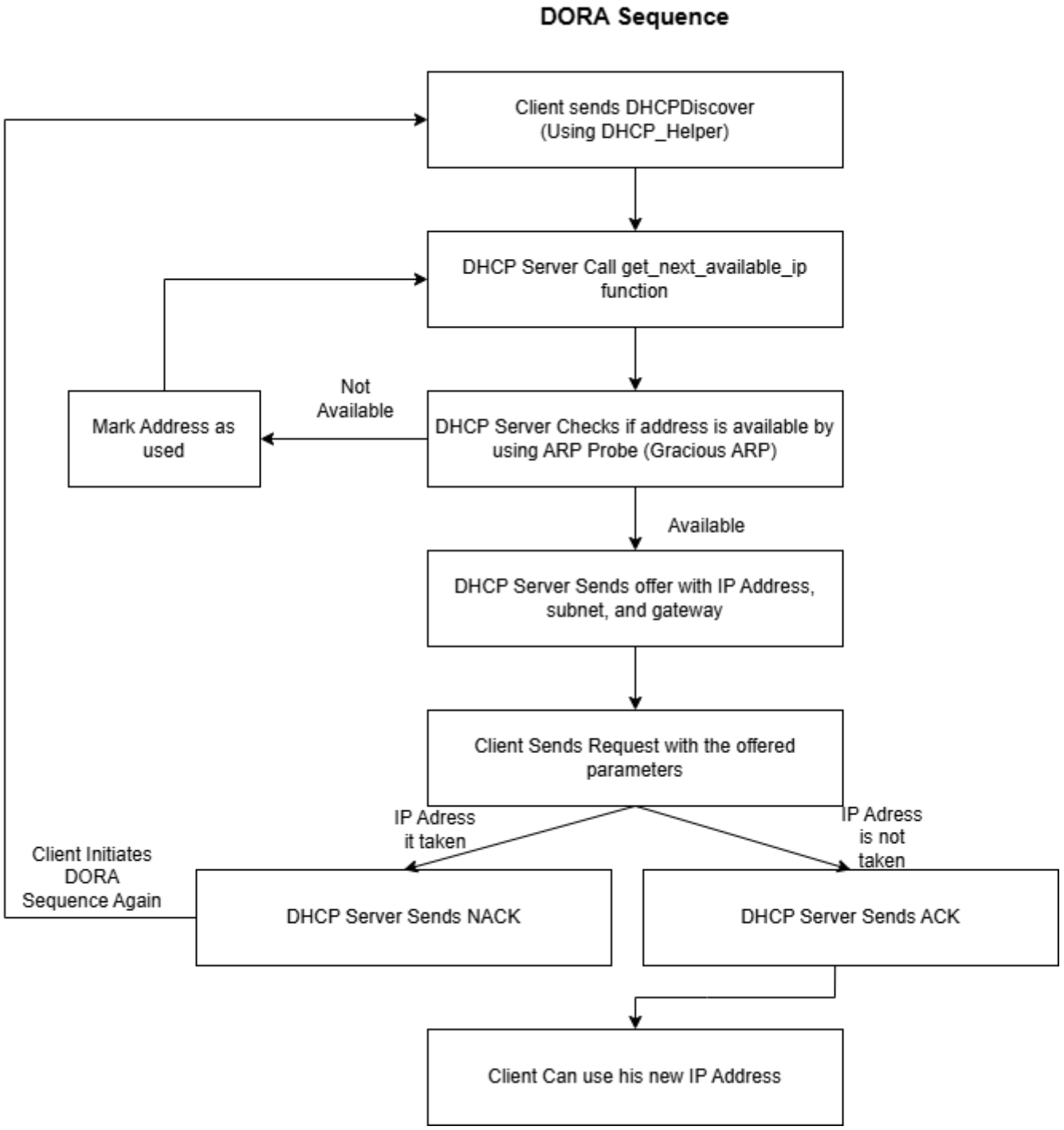
- אפשרות 54 - Server Identifier - כתובת ה-IP של שרת ה-DHCP. זה חובה ב-OFFER וב-ACK.
- אפשרות 51 - IP address Lease Time - הזמן בשניות שהכתובת ניתנת ללקוח.
- אפשרות 1 - Subnet Mask - מסכת הרשת ללקוח - קריטי כדי שהלקוח יידע את מרחב הכתובת של הרשת - וכך יידע מי נמצא איתו ברשת.
- אפשרות 3 - Router Option - כתובת ה-Default Gateway. הכתובת של הנתב - אליו מנתבים את התקשורת שנועדה לצאת מחוץ לרשת הפנימית
- אפשרות 6 - Domain Name Server Option - כתובת שרת ה-DNS שמוגדרת על ידי שרת הDHCP
- אפשרות 61 - Client Identifier - מזהה ייחודי של הלקוח. השתמשנו ב PID

## אפשרויות נוספות שלא מימשנו וכדאי לדבר עליהם:

- אפשרות 28 - Broadcast Address הכתובת שאליה שולחים broadcast. ניתן לחשב אותה על פי הsubnet ולכן אין צורך לכתוב אותה מפושרות
- אפשרות 50 - Requested IP Address - האפשרות הזו תתמלא בעת Request (בקשה רשמית של הכתובת) ובעת INIT-REBOOT - כאשר מחשב נדלק מחדש הוא מנסה לבקש את הכתובת שהייתה לו.

את האפשרות הזו לא מימשנו כדי לאפשר הרצה ללא הרשאות Admin. כדי שנוכל לממש את מחלק כתובות הDHCP כפי שתכננו, היינו צריכים לחרוג מקצת מהתקן.

## 1. תרשים זרימה: תהליך ה-DORA והלוגיקה הפנימית




---

## 2. לוגיקת ניהול הכתובות בשרת (IP Allocation)

השרת שבנינו פועל לפי סדר עדיפויות קשיח כדי להבטיח עקביות ברשת:

1. **Static Reservations:** בדיקה בקובץ ה-YAML האם ה-PID של הלקוח זכאי לכתובת קבועה. למשל את כתובת השרת קבענו להיות 127.0.0.13.

```
reservations_loopback:
  "API": "127.0.0.1"
  "RESOLVER": "127.0.0.2"
  "ROOT": "127.0.0.3"
  "TLD": "127.0.0.4"
  "AUTH": "127.0.0.5"
  "105": "127.0.0.6"
  "106": "127.0.0.7"
  "107": "127.0.0.8"
  "108": "127.0.0.9"
  "109": "127.0.0.10"
  "110": "127.0.0.11"
  "OriginServer": "127.0.0.13"
  "113": "127.0.0.14"
  "114": "127.0.0.15"
  "115": "127.0.0.16"
  "116": "127.0.0.17"
  "117": "127.0.0.18"
  "118": "127.0.0.19"
  "119": "127.0.0.20"
  "120": "127.0.0.21"
  "121": "127.0.0.22"
```

2. **Sticky Leases:** בדיקה בזיכרון (ובקובץ ה-Persistence) האם ללקוח כבר יש כתובת קיימת.
3. **Reclaimed IPs:** שימוש בכתובות שחזרו למאגר (כדי לא לבזבז כתובות חדשות). לשם כך יש את פונקציית השרת lease_cleanup_thread שתרוץ ברקע, ובכל פעם שתראה כתובת שזמן הסיום שלה עבר, תמחק את הכתובת מקובץ ההשאלות, ובכך נוודא שאנחנו לא מאבדים כתובות.
4. **Dynamic Pool:** הגרלת כתובת חדשה מהטווח שהוגדר. לשם כך יש את הפונקציה של השרת בשם get_next_available_ip. ראשית אם יש כתובת שמורה (כמו לשרת), נחזיר אותה. אחרת, נבדוק האם יש כתובת שמוקצה בזה הרגע למכשיר על פי הPID. במידה ואין נקצה כתובת חדשה (במידה והקצנו את כל הכתובות הקיימות בPool נחזיר None (מכיוון שאין לנו כתובת פנויה להקצאה).

```python
def get_next_available_ip(self, client_id):
        """Returns the next available IP for client_id,
         making sure to check reservations and reclaimed IPs."""
         with self.lock:
             if client_id in self.reservations:
                 return self.reservations[client_id]
             if client_id in self.client_to_ip:
                 return self.client_to_ip[client_id]
             if self.available_reclaimed_ips:
                 return self.available_reclaimed_ips.pop(0)
             reserved_ips = set(self.reservations.values())
```

```
            # Skip reserved IPs in the pool
        while self.current_pool_ip in reserved_ips:
            octets = list(map(int, self.current_pool_ip.split('.')))
            if octets[3] <= int(self.end_ip.split('.')[3]):
                octets[3] += 1
                self.current_pool_ip = ".".join(map(str, octets))
            else:
                return None

        # Simple IP increment logic
        octets = list(map(int, self.current_pool_ip.split('.')))
        if octets[3] < int(self.end_ip.split('.')[3]):
            assigned = self.current_pool_ip
            octets[3] += 1
            self.current_pool_ip = ".".join(map(str, octets))
            print(f" |---> [POOL] Assigned {assigned} to PID: {client_id}")
            return assigned
        return None
```

מצד הלקוח הגדרנו שהלקוח יבצע בקשה לחידוש האישור לשימוש בכתובת, כאשר נגיע לחצי מהזמן הקצוב. כך נוודא שכל האפשר נשאר עם אותה הכתובת.

---

### 3. מנגנוני הגנה וגיבוי (Reliability)

במסגרת הפרויקט הוספנו שלושה מנגנונים גיבוי קריטיים:

1. **Persistence (התמדה):** השרת שומר את מצב הכתובות בקובץ json כדי למנוע כפילויות במקרה של קריסת שרת.
2. **APIPA Fallback:** אם הלקוח לא מקבל מענה (Timeout), הוא לא קורס אלא מקצה לעצמו כתובת זמנית בטווח x.x.169.254.
3. **Active Release:** שימוש ב-atexit בלקוח כדי להודיע לשרת על עזיבה, מה שמאפשר שימוש חוזר מיידי בכתובת.

---

# המימוש שלנו של פרוטוקול הDHCP:

## דגשים במימוש פרוטוקול ה-DHCP בפרויקט

במימוש הנוכחי, הוחלט על מספר התאמות המאפשרות הרצה סטרילית של הפרויקט מבלי להשפיע על תהליכי ה-DHCP הקיימים ברשת הפיזית:

- **בחירת פורט (Port 6700):** השרת הוגדר להאזין בפורט **6700** (במקום פורט 67 הסטנדרטי).
  בחירה זו נועדה למנוע התנגשויות עם שירותי מערכת ההפעלה, להבטיח עבודה בטווח הפורטים החופשיים, ולשמור על דמיון ויזואלי לפורט המקורי לטובת קריאות הקוד.

המערכת מיועדת לניהול והקצאה של כתובות בטווח ה-**Loopback**.

- **ממשק ה-(VirtualNetworkInterface):** לצורך אינטגרציה פשוטה עם רכיבי המערכת השונים, מימשנו את המחלקה `VirtualNetworkInterface` (בקובץ **DHCP_HELPER.py**). מחלקה זו מאפשרת שימוש אחיד לתהליכים חיצוניים בפרויקט לקבל כתובות ip.

- **ניהול זהויות לקוח (Client Identification):**
  - במהלך ה-Initialization (תהליך ה-`__init__`), ניתן למלא את הפרמטר `client_name` (ברירת מחדל: "Device") ו-`fixed_id`.
  - **מנגנון ה-ID:** אם סופק `fixed_id` (למשל עבור Original Server), המערכת תשתמש בו כדי להבטיח הקצאת IP קבועה (Static DHCP). במידה ולא סופק מזהה, המערכת תייצר ID ייחודי המבוסס על שם הלקוח, מזהה תהליך (**PID**) ורכיב רנדומלי.

```
    def __init__(self, client_name="Device", fixed_id=None):
            self.client_name = client_name
            self.my_pid = os.getpid()
          # unique id that combines name, PID and a random number to make sure
          there are no duplicates in the pool

            if fixed_id is None:
                self.unique_id =
                f"{client_name}-{self.my_pid}-{random.randint(1000,9999)}"

                self.client_id_opt = ("client_id", self.unique_id.encode())
            else:
                self.unique_id = fixed_id
                self.client_id_opt = ("client_id", self.unique_id.encode())

            self.sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            self.sock.bind(("0.0.0.0", 0))
            self.sock.settimeout(3)

            self.ip = None
            self.is_apipa = False
            self.lease_time = 60
            self.running = True
            self._probe_listener = ProbeListener(lambda: self.ip)
            atexit.register(self.release_ip)
```

בפונקציה setup_network נבצע את הליך הבקשה המבוסס על תהליך ה-DORA - ובסופו נקבע את כתובת ה-IP שקיבלנו משרת הDHCP. במידה ולא נקבל כתובת, נקבע אותה בתור כתובת APIPA.

```
    def perform_dora(sock, client_id_opt, server_addr=("127.0.0.1", 6700)):
        """Performs DISCOVER → OFFER → REQUEST → ACK. Returns (assigned_ip,
    lease_time)."""
        xid = random.getrandbits(32)

        # D - Discover
        disc = BOOTP(xid=xid)/DHCP(options=[("message-type", "discover"),
```

```
client_id_opt, "end"])
        sock.sendto(raw(disc), server_addr)

        # O - Offer
        data, _ = sock.recvfrom(2048)
        offer = BOOTP(data)

        # R - Request
        req = BOOTP(xid=xid, yiaddr=offer.yiaddr)/DHCP(options=[("message-type",
"request"), client_id_opt, "end"])
        sock.sendto(raw(req), server_addr)

        # A - Ack
        data, _ = sock.recvfrom(2048)
        ack = BOOTP(data)

        opts = {opt[0]: opt[1] for opt in ack[DHCP].options if isinstance(opt,
tuple)}
        lease_time = opts.get('lease_time', 60)
        return ack.yiaddr, lease_time, opts
```

```
def setup_network(self):
        try:
            # Call the shared DORA helper
            self.ip, self.lease_time, _ = perform_dora(self.sock,
self.client_id_opt)

            print(f"[V] DHCP Success: {self.unique_id} assigned {self.ip}")
            self._probe_listener.start()
            threading.Thread(target=self._maintain_lease, daemon=True).start()

        except Exception:
            self.ip = f"169.254.{random.randint(1,254)}.{random.randint(1,254)}"
            self.is_apipa = True
            print(f"[!] DHCP Failed, using APIPA: {self.ip}")

        return self.ip
```

מכיוון שכל התהליך שלנו רץ על כתובות loopback החלטנו לבצע שליחה של בקשת discover ישירות לכתובת הידועה של שרת ה DHCP במקום לכתובת הbroadcast כלומר ל- **127.0.0.1**

במימוש לשרת אמיתי על פי ה-RFC השליחה של הבקשה תתבצע כתובת הbroadcast שהיא:

**255.255.255.255**

בנוסף יש לנו את פונקציית maintain_lease שתפקידה לבצע חידוש אוטומטי לכתובת ה-IP שקיבלנו.

```
    def _maintain_lease(self):
        while self.running:
            time.sleep(self.lease_time / 2)
            if not self.is_apipa and self.ip:
                try:
                    req = BOOTP(yiaddr=self.ip)/DHCP(options=[("message-type",
"request"), self.client_id_opt, "end"])
                    self.sock.sendto(raw(req), ("127.0.0.1", 6700))
                    self.sock.recvfrom(2048)
                except: pass
```

**נשים לב:**

שנבצע את הבקשה רק אם כבר קיבלנו כתובת IP וכתובת הIP היא לא כתובת APIPA (במידה ויש כתובת APIPA לא קיבלנו אותה מהשרת, ועל כן אין צורך לבקש ממנו חידוש)

## נעבור לdhcp_server.py :

תחילה נאתחל את השרת ונקרא לפונקציית run:

```
if __name__ == "__main__":
    DHCPServer().run()
```

## אתחול השרת:

נקרא את הconfig מהקובץ, ונפעיל את השרת על פורט 6700 נאתחל את גבולות הכתובות על פי שנקרא מה config את ההקצאות הקיימות נקרא מקובץ הyaml. מטרת מנגנון זה לאפשר התאוששות בעת קריסת השרת ללא איבוד ההקצאות.

```
class DHCPServer:
    def __init__(self):
        self.project_root = Path(__file__).resolve().parent.parent
        self.config_path = self.project_root / "dhcp" / "configs" /
"dhcp_config.yaml"
        with open(self.config_path, 'r') as f:
            self.conf = yaml.safe_load(f)['dhcp_server']
        self.PORT_S, self.running = 6700, True
        self.lock, self.work_queue = threading.Lock(), queue.Queue()

        # Determine which IP pool to use based on the 'loopback' setting in the
config
        pool = self.conf['loopback_pool'] if self.conf['loopback'] else
self.conf['outside_pool']
```

```
        self.current_pool_ip, self.end_ip = pool['start_ip'], pool['end_ip']
        self.available_reclaimed_ips = []
        self.active_leases = self.load_leases()
        self.client_to_ip = {data['client_id']: ip for ip, data in
self.active_leases.items()}
        self.reservations = self.conf['reservations_loopback'] if
self.conf['loopback'] else self.conf['reservations_outside']
        self.persistence_file = self.conf['persistence_file']
        #if there were already leased IPs, we need to make sure the pool starts at
the right place.
        pool_start = list(map(int, self.current_pool_ip.split('.')))
        for ip in self.active_leases:
            octets = list(map(int, ip.split('.')))
            if octets[:3] == pool_start[:3] and octets[3] >= pool_start[3]:
                pool_start[3] = max(pool_start[3], octets[3] + 1)
        #change the current pool accordingly
        self.current_pool_ip = ".".join(map(str, pool_start))
        #if all IPs are taken the get_next_ip will return None (untill we get back
addresses)
```

לקובץ הקצאות קיימות פונקציות לקריאה ושמירה - בנפרד כדי שנוכל להריץ אותם במקביל לthread הכללי.

```
def save_leases(self):
            """Saves a snapshot of leases to avoid thread-blocking during I/O."""
            try:
                with self.lock:
                    data_to_save = dict(self.active_leases)
                with open(self.persistence_file, 'w') as f:
                    json.dump(data_to_save, f, indent=4)
            except Exception as e:
                print(f"[!] Save Error: {e}")

    def load_leases(self):
        """Helper to load leases from disk at startup"""
        if os.path.exists(self.conf['persistence_file']):
            try:
                with open(self.conf['persistence_file'], 'r') as f: return json.load(f)
            except: return {}
        return {}
```

נעבור לפונקציה run:

נריץ את שני הפונקציות lease_cleanup_thread ו sender_thread לטובת ריצת השרת

במידה והפורט תפוס נציג הודעה ונפסיק את הריצה

אם השרת רץ, נעצור את הריצה שלו בעת לחיצה על ctrl+c.

```
    def run(self):
        """Start background threads for lease cleanup and packet sending"""
```

```
        threading.Thread(target=self.lease_cleanup_thread, daemon=True).start()
        threading.Thread(target=self.sender_thread, daemon=True).start()
        try:
            # If can't bind, port is taken
            recv_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            recv_sock.bind(("0.0.0.0", self.PORT_S))
            recv_sock.settimeout(1.0)
            print(f"[*] DHCP Server active on Port {self.PORT_S}")
            try:
                while self.running:
                    try:
                        data, addr = recv_sock.recvfrom(2048)
                        pkt = BOOTP(data)
                        if DHCP in pkt: self.process_packet(pkt, addr)
                    except (socket.timeout, Exception): continue
            except KeyboardInterrupt: sys.exit(0)
        except Exception as e:
            print(f"[!] Port Taken")
```

הפונקציה lease_cleanup_thread :

```
    def lease_cleanup_thread(self):
            """Background thread to reclaim IPs when the lease time is up."""
            while self.running:
                time.sleep(5)  # Check every 5 seconds
                now = time.time()
                expired_ips = []

                # 1. LOCK FAST: Just identify what needs to be deleted
                with self.lock:
                    expired_ips = [ip for ip, data in self.active_leases.items() if
now > data['expiry']]
                    if not expired_ips:
                        continue

                # 2. PROCESS: Handle the deletions
                    for ip in expired_ips:
                        client_id = self.active_leases[ip]['client_id']
                        print(f"\n[EXPIRE] Reclaiming {ip} from {client_id}")

                        if ip in self.active_leases: del self.active_leases[ip]
                        if client_id in self.client_to_ip: del
self.client_to_ip[client_id]

                        # Add back to reclaimed list so it can be reused immediately
                        if ip not in self.available_reclaimed_ips:
                            self.available_reclaimed_ips.append(ip)
```

```
                # 3. SAVE OUTSIDE: Writing to disk is slow; don't block the network
while doing it
                try:
                    self.work_queue.put(("save", None))
                except Exception as e:
                    print(f"[!] Error saving leases: {e}")
```

תפקידה לעבור על רשימת הכתובות המוקצות ולשחרר כתובות להקצאה חוזרת במידה והחכירה הסתיימה.

בכל 5 שניות נבדוק האם יש כתובות IP שצריך לנקות.

במידה ויש נדפיס הודעה בהתאם שפג תוקף, ונמחוק את ההקצאה הרלוונטית. עכשיו הכתובת פנוייה ועל כן נוסיף אותה לכתובות הפנויות. לבסוף נקרא לפונקצייה save_leases כדי לשמור את השינויים לקובץ (כתיבה לדיסק היא איטית אז נפריד את השמירה)

פונקציית sender_thread

פותחת את הsocket ושולחת את המידע מהשרת במידת הצורך.

```
    def sender_thread(self):
        """Background thread to send packets and save files from the work queue"""
        send_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        while self.running:
            try:
                task = self.work_queue.get(timeout=1)

                # Check if the task is a request to save to disk
                if task[0] == "save":
                    self.save_leases()
                else:
                    # Otherwise, it's a packet that needs to be sent
                    pkt_raw, target = task
                    send_sock.sendto(pkt_raw, target)

                self.work_queue.task_done()
            except queue.Empty: continue
```

פונקציית **process_packet**:

תפקידה לנתח את חבילות המידע הנכנסות, לזהות את סוג הבקשה ולהגיב בהתאם לפרוטוקול ה-DORA ולמצב הניהולי של הכתובות בשרת, הסבר מעמיק לאחר הקוד.

```
    def process_packet(self, pkt, addr):
        """Main packet processing for DISCOVER, REQUEST, RELEASE."""
        options = pkt[DHCP].options
        msg_type = next((opt[1] for opt in options if isinstance(opt, tuple) and
```

```
opt[0] == 'message-type'), None)
        client_id = next((opt[1].decode() if isinstance(opt[1], bytes) else opt[1]
                          for opt in options if isinstance(opt, tuple) and opt[0] ==
'client_id'), None)

        if not client_id:
            print(f"[DHCP] Packet without client_id from {addr}")
            return

        if msg_type == DHCPMessageType.DISCOVER:
            print(f"\n[DISCOVER] From {client_id} at {addr}")
            offer_ip = self.get_next_available_ip(client_id)

            if offer_ip:
                print(f"[OFFER] Sending {offer_ip} to {client_id}")
                reply = BOOTP(op=2, yiaddr=offer_ip, xid=pkt.xid) / DHCP(options=[
                    ("message-type", "offer"),
                    ("server_id", "127.0.0.1"),
                    ("subnet_mask", self.conf['subnet_mask']),
                    ("router", self.conf['gateway']),
                    ("name_server", self.conf['dns_servers'][0]),
                    ("lease_time", self.conf['lease_time']),
                    ("client_id", client_id.encode()),
                    "end"
                ])
                self.work_queue.put((raw(reply), (addr[0], addr[1])))

        elif msg_type == DHCPMessageType.REQUEST:
            print(f"\n[REQUEST] From {client_id}")
            target_ip = pkt.yiaddr if pkt.yiaddr != "0.0.0.0" else None

            if not target_ip:
                with self.lock:
                    target_ip = self.client_to_ip.get(client_id)

            if not target_ip:
                print(f"[NAK] No IP for {client_id}")
                nack = BOOTP(op=2, xid=pkt.xid) / DHCP(options=[
                    ("message-type", "nak"),
                    ("server_id", "127.0.0.1"),
                    "end"
                ])
                self.work_queue.put((raw(nack), (addr[0], addr[1])))
                return

            # Lease the IP
            with self.lock:
                self.active_leases[target_ip] = {
                    "expiry": time.time() + self.conf['lease_time'],
```

```
                "client_id": client_id
            }
            self.client_to_ip[client_id] = target_ip

        print(f"[ACK] Lease confirmed for {target_ip} → {client_id}")
        reply = BOOTP(op=2, yiaddr=target_ip, xid=pkt.xid) / DHCP(options=[
            ("message-type", "ack"),
            ("server_id", "127.0.0.1"),
            ("lease_time", self.conf['lease_time']),
            "end"
        ])
        self.work_queue.put((raw(reply), (addr[0], addr[1])))
        self.work_queue.put(("save", None))

    elif msg_type == DHCPMessageType.RELEASE:
        try:
            self.release_ip(client_id, pkt.yiaddr)
        except Exception as e:
            print(f"[!] Error processing RELEASE from {client_id}:
{type(e).__name__}: {e}")
        # Save outside the lock
        self.work_queue.put(("save", None))
```

הפונקציה אחראית לבצע מספר דברים:

**חילוץ נתונים ואימות:** הפונקציה סורקת את ה-Options של חבילת ה-DHCP כדי לחלץ את סוג ההודעה (**Message Type**) ואת מזהה הלקוח (**Client ID**). במידה ולא נמצא מזהה לקוח, החבילה נזרקת כחבילה לא תקינה.

**ניתוב לפי סוג ההודעה:**

- **DISCOVER:** השרת מאתר כתובת IP פנויה עבור הלקוח. במידה ונמצאה כתובת, נוצרת חבילת **OFFER** הכוללת את כל הגדרות הרשת הרלוונטיות (Subnet Mask, Router, DNS, Lease Time) והודעה זו נשלחה לתור העבודה (**Work Queue**).

  **REQUEST:** השרת מוודא שהכתובת המבוקשת אכן שמורה עבור הלקוח.
  - אם הכתובת נמצאה במערכת: מתבצע עדכון של ה-**Active Leases** (תחת מנגנון נעילה למניעת Race Conditions), נשלח אישור סופי (**ACK**) והמערכת מבצעת רישום (Save) של המצב החדש.
  - אם הכתובת אינה רלוונטית או לא קיימת: נשלחת הודעת סירוב (**NAK**).
- **RELEASE:** השרת קורא לפונקציית העזר `release_ip` כדי לשחרר את הכתובת שהוקצתה חזרה למאגר הכתובות הפנויות, ומעדכן את בסיס הנתונים.

**ניהול תורים וסנכרון:** הפונקציה אינה שולחת את התשובה ישירות אלא משתמשת ב-`work_queue`. הפרדה זו מאפשרת לשרת להמשיך להאזין לבקשות חדשות בזמן שתהליך אחר מטפל בשליחה הפיזית ובעדכון הקבצים.

## פונקציה get_next_available_ip:

```
def get_next_available_ip(self, client_id):
```

```
        """Get next available IP for client, checking reservations and reclaimed IPs."""
        # CHECK RESERVATIONS FIRST (no probing needed)
        if client_id in self.reservations:
            reserved_ip = self.reservations[client_id]
            print(f"[DHCP] Reserved IP for {client_id}: {reserved_ip}")
            return reserved_ip

        # Check if client already has an IP
        with self.lock:
            if client_id in self.client_to_ip:
                existing_ip = self.client_to_ip[client_id]
                print(f"[DHCP] Existing lease for {client_id}: {existing_ip}")
                return existing_ip

        # For pool IPs, try reclaimed first, then new ones
        MAX_TRIES = 10
        for attempt in range(MAX_TRIES):
            with self.lock:
                # Try reclaimed IPs first
                if self.available_reclaimed_ips:
                    candidate =self.available_reclaimed_ips.pop(0)
                    print(f"[DHCP] Using reclaimed IP: {candidate}")
                else:
                    candidate = self._advance_pool_ip()


                if candidate is None:
                    print(f"[DHCP] No IPs available in pool")
                    return None

            # Probe outside the lock (blocking network call)
            if not self._probe_ip(candidate):
                print(f"[DHCP] Assigned pool IP {candidate} to {client_id} (attempt {attempt
+ 1}/{MAX_TRIES})")
                return candidate

            print(f"[DHCP] Conflict on {candidate} -- responded to probe. Skipping.")

        print(f"[DHCP] Could not find free IP after {MAX_TRIES} attempts")
        return None
```

תפקידה בעת בקשת כתובת להקצות כתובת מתאימה.

היא עובדת על פי סדר עדיפויות:

תחילה אם למכשיר זה יש כתובת שמורה אז הוא יקבל אותה (בעצם הקצאה של כתובת קבועה)

אם למכשיר יש כתובת מוקצית בזה הרגע - הוא יקבל אותה

נבדוק לבסוף האם יש כתובות שהוקצו בעבר והתפנו - אם כן נקצה מהן.

רק אם אף אחת מהאופציות התקיימה נקצה למכשיר כתובת חדשה מה-pool.

הקידום יתבצע על ידי קידום כתובת ה-IP ב-1.

לאחר בחירת כתובת נוודא שהיא אכן פנוייה בעזרת שימוש בפונקציה probe_ip

במידה ולא נשתמש בפונקציה advance_pool_ip שתקדם לנו לכתובת הבאה שאינה תפוסה על פי הסימונים שלנו, ותדלג על כתובות שחולקו

---

פונקציה advance_pool_ip:

תקדם את המצביע של הכתובת הפנויה לכתובת הבאה אשר פנויה (ונדלג על כתובות פנויות).

```
def _advance_pool_ip(self):
    """Get next IP from pool, skipping reserved IPs. Called with lock held."""
    reserved_ips = set(self.reservations.values())
    end_octet = int(self.pool_end.split('.')[3])

    while True:
        octets = list(map(int, self.current_pool_ip.split('.')))

        # Check if we've exceeded the pool range
        if octets[3] > end_octet:
            print(f"[DHCP] Pool exhausted, wrapping around")
            self.current_pool_ip = self.pool_start
            return None

        # Check if current IP is reserved
        if self.current_pool_ip in reserved_ips:
            octets[3] += 1
            self.current_pool_ip = ".".join(map(str, octets))
            continue

        # Return this IP and advance pointer
        ip_to_return = self.current_pool_ip
        octets[3] += 1
        self.current_pool_ip = ".".join(map(str, octets))

        return ip_to_return
```

---

## הקצאת DNS אוטומטית באמצעות DHCP

רוב המכשירים ברשת אינם מוגדרים עם כתובת DNS ידנית. במקום זאת, הם מקבלים את כתובת ה-DNS כחלק מ"חבילת ההגדרות" שמספק שרת ה-**DHCP** בזמן ההתחברות לרשת.

**התהליך הטכני:**

1. **שלב ה-DORA:** במהלך לחיצת היד של פרוטוקול ה-DHCP (בשלבי ה-**Offer** וה-**ACK**), השרת שולח ללקוח רשימה של שדות הנקראים **DHCP Options**.
2. **Option 6 (Domain Name Server):** זהו השדה הספציפי בחבילת ה-DHCP המכיל את כתובות ה-IP של שרתי ה-DNS הזמינים. השרת פשוט "מצרף" את הכתובת הזו לבקשה.
3. **הטמעה במערכת ההפעלה:** ברגע שהמחשב (הלקוח) מקבל את אישור הכתובת (ACK), מערכת ההפעלה (Windows, Linux או macOS) מעדכנת את הגדרות ה-Resolver הפנימיות שלה בכתובת שהתקבלה ב-Option 6.

תהליך דומה לזה אנחנו מממשים בפונקציה שכבר עברנו עליה process_packet:

```
if msg_type == DHCPMessageType.DISCOVER:
            print(f"\n[DISCOVER] From {client_id} at {addr}")
            offer_ip = self.get_next_available_ip(client_id)

            if offer_ip:
                print(f"[OFFER] Sending {offer_ip} to {client_id}")
                reply = BOOTP(op=2, yiaddr=offer_ip, xid=pkt.xid) / DHCP(options=[
                    ("message-type", "offer"),
                    ("server_id", "127.0.0.1"),
                    ("subnet_mask", self.conf['subnet_mask']),
                    ("router", self.conf['gateway']),
                    ("name_server", self.conf['dns_servers'][0]),
                    ("lease_time", self.conf['lease_time']),
                    ("client_id", client_id.encode()),
                    "end"
                ])
                self.work_queue.put((raw(reply), (addr[0], addr[1])))
```

- השם `"name_server"` הוא הכינוי (Alias) שספריית Scapy נותנת ל-**DHCP Option 6**.
- הערך `conf['dns_servers'][0]` לוקח את הכתובת הראשונה מרשימת שרתי ה-DNS שהגדרת בקונפיגורציה של השרת שלך.

ברגע שהלקוח (ה-PC) יקבל את חבילת ה-OFFER הזו, הוא יקרא את השדה הזה ויבין:

*"אוקיי, אם אני מקבל את הכתובת הזו, זה ה-DNS שאני צריך לעבוד איתו"*

# DNS

בפשטות: שירות ה-DNS משמש כ"ספר הטלפונים" של הרשת. תפקידו המרכזי הוא לבצע תרגום (Resolution) בין שמות מתחם (**Domains**) קריאים לאדם המבוססים על מבנה היררכי, לבין כתובות **IP** נומריות המשמשות את המכונות לתקשורת בפועל.

כאשר אנחנו מנסים לגשת לDomain מסויים דרך הדפדפן שלנו (למשל www.google.com) מאחורי הקלעים מתבצעת שליחה של פקטה בקשת DNS לשרת שנקרא Resolver המוגדר בהגדרות המכשיר.

פורמט בקשת DNS יראה כך:

Message Format:

| |
|---|
| Header |
| Question |
| Answer |
| Authority |
| Additional |

הסבר קצר על מבנה ההודעה:

ה Header תמיד נמצא בכל סוגי ההודעות, הוא מכיל את המידע הנחוץ לנו בשביל לדעת את גודל ההודעה ואת סוגה ההודעה (תשובה או בקשה). גודלו הוא קבוע, והוא תמיד בגודל של 12 **בתים**. מבנה ה header נראה כך:

Header Format:

16 bit

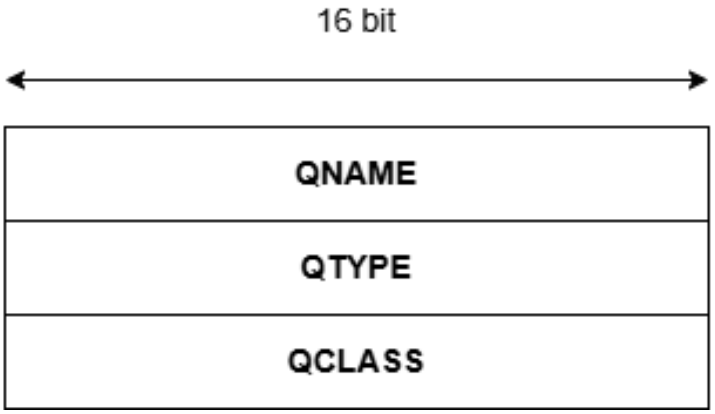
| | | | | | | | |
|---|---|---|---|---|---|---|---|
| ID | | | | | | | |
| QR | Opcode | AA | TC | RD | RA | Z | RCODE |
| QDCOUNT | | | | | | | |
| ANCOUNT | | | | | | | |
| NSCOUNT | | | | | | | |
| ARCOUNT | | | | | | | |

נפרק את המבנה:

1. השורה העליונה באיור - ID - זהו מזהה ייחודי להודעה, כל חבילה שעוברת ברשת מכילה מספר מזהה שלפיה השולח ידע עבור איזה חבילה הוא מקבל מענה
2. שורת הדגלים - מכילה את הדגלים הבאים:
   - QR - ביט אחד אשר מסמן האם ה message
   - הנוכחי הוא תשובה או בקשה. (Question\Response)
   - Opcode - ארבעה ביטים שמסמנים לנו את סוג ההודעה:
     - 0 - בקשה רגילה (query) - שמטרתה לתרגם לנו Domain לכתובת IP
     - 1 - בקשה הפוכה (inverse query) שמטרתה לתרגם לנו כתובת IP לdomain
     - 2 בקשת סטטוס (Status Request) - מדובר ב-Legacy Opcode - שמטרתו לבדוק את הסטטוס של שרת הDNS.
       הקוד הזה לא נמצא בשימוש ובחלק מהמערכות מודרניות אף לא ממומש
     - 3-15 שמורים לשימוש עתידי (אין בהם שימוש לפחות כרגע)
   - AA - קיצור של Authoritative Answer, הדגל הזה מסמן האם קיבלנו את התשובה שלנו משרת שהוא Authoritative (אמין) או שרת ביניים.
   - TC - מסמן לנו האם ההודעה נחתכה כי עברה את הגודל המותר המוגדר
   - RD - הדגל מסמן האם מדובר בקשה רקורסיבית.
     במידה והדגל מסומן הResolver יבצע את החיפוש בעץ ההירכי בעצמו, ויחזיר לנו את התשובה הסופית.
     במידה ולא מסומן - החיפוש לא יבוצע על ידי הResolver ובמידה ולא תהיה לו תשובה, הוא יחזיר לנו הפנייה לשרת הבא בעץ שאיתו נוכל לבדוק.
   - RA - דגל Recursion Available - מסמל לנו האם הResolver יודע לבצע שאילתא רקורסיבית
   - Z - דגל שמור לשימוש עתידי, על פי הגדרת הRFC חייב להכיל אפסים **בכל** סוגי ההודעות
   - RCODE - קוד תשובה (Response Code)
     - 0 - אין שגיאה
     - 1 - שגיאת פורמט. השרת לא הצליח לפענח את השאילתא.
     - 2 - שגיאת שרת - השרת לא הצליח לעבד את השאילתא בגלל בעיה בשרת
     - 3 - שגיאת שם - את השגיאה הזו נקבל רק משרת מסוג authoritative server (שרת מורשה).
       השגיאה מסמלת לנו ששם הDomain המבוקש לא קיים
     - 4 - not implemented (לא מומש) - השרת לא תומך בסוג השאילתא שהתקבלה
     - 5 - סירוב - השרת סירב לתת שירות לבקשה (למשל אם המבקש הוא מבקש בעל כתובת שאסרו עליה להשתמש בשירות)
     - 6-15 שמורים לשימוש עתידי
3. QDCOUNT - מספר לא מסומן (unsigned) בגדול 16 ביטים שמראה לנו את מספר השאלות בquestion section
4. ANCOUNT - מספר לא מסומן (unsigned) בגדול 16 ביטים שמראה לנו את מספר התשובות בanswer section
5. NSCOUNT - מספר לא מסומן (unsigned) בגדול 16 ביטים שמראה לנו את מספר הרשומות של שרתי המשאבים בauthority records section
6. ARCOUNT - מספר לא מסומן (unsigned) בגדול 16 ביטים שמראה לנו את מספר הרשומות המשאבים בadditional records section

# מבנה Question Section

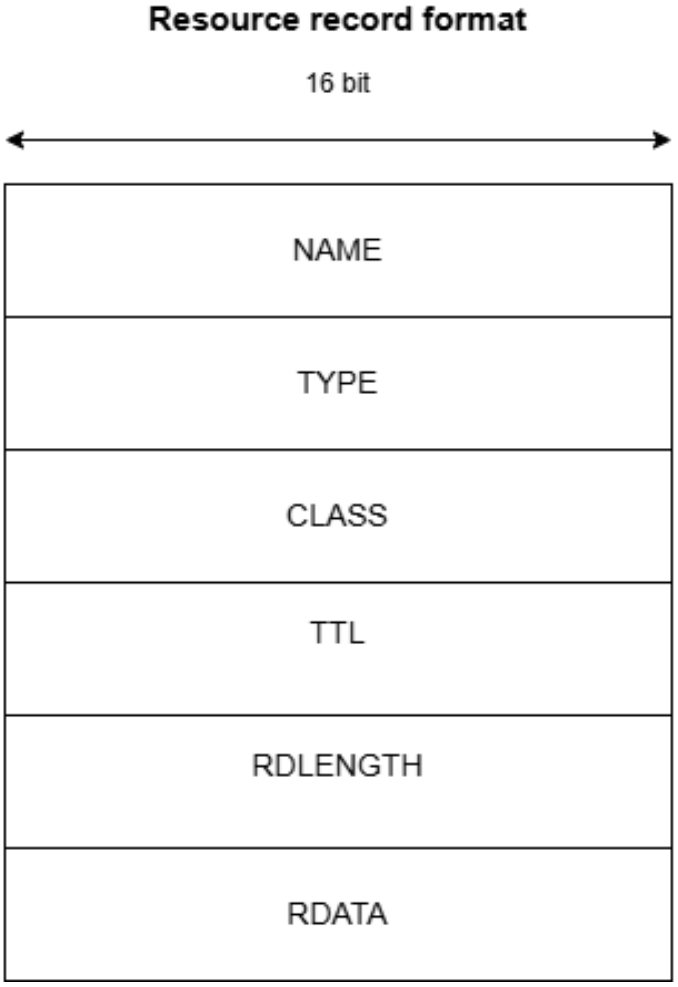
## Question Section Format:




סעיף זה מכיל את פרטי השאילתה שנשלחת לשרת. בניגוד לשדות ה-Header, חלק מהשדות כאן הם באורך משתנה.

- **(QNAME Query Name):** שדה באורך משתנה המייצג את שם המתחם (Domain) המבוקש. השם מיוצג כסדרה של תגים (**Labels**):
  - כל תג מתחיל ב**אוקטטת אורך** (אחד בית) המציינת כמה תווים יופיעו בהמשכו.
  - לאחר אוקטטת האורך, מופיעים התווים של אותו חלק בשם (למשל, עבור "google", אוקטטת האורך תהיה 6).
  - השם מסתיים ב**אוקטטת אפס (0x00)**, המשמשת כ-Null Terminator ומסמנת את סוף כתובת ה-Root.
  - בשדה זה לא מתבצע שימוש ב-Padding (ריפוד).
- **(QTYPE Query Type):** שדה בן שתי אוקטטות (16 ביט) המגדיר את סוג הרשומה המבוקשת. קוד זה קובע איזה סוג מידע אנו מצפים לקבל מהשרת. ערכים נפוצים כוללים:
  - **1 (A Record):** בקשה לכתובת IPv4.
  - **28 (AAAA Record):** בקשה לכתובת IPv6.
  - **5 (CNAME):** בקשה לשם קנוני (Alias).
  - **15 (MX):** בקשה לשרתי הדואר של ה-Domain.
- **(QCLASS Query Class):** שדה בן שתי אוקטטות המגדיר את משפחת הפרוטוקולים של הרשת.
  - במרבית המוחלטת של המקרים (ובפרויקט זה בפרט), הערך יהיה **1**, המייצג את מחלקת ה-Internet (**IN**).

## מבנה רשומת המשאב (Resource Record):




רשומות המשאב מופיעות בסעיפי ה-**Authority** ,**Answer**, ו-**Additional** של חבילת ה-DNS. הן מכילות את התשובות לשאילתה שנשלחה ואת המידע שנלווה אליהן.

- **NAME:** שם המתחם אליו שייכת הרשומה. השם יכול להופיע בפורמט מלא (Labels) או כמצביע (Pointer) לצורך דחיסה.
- **TYPE:** שדה בן שתי אוקטטות המגדיר את סוג המשאב. שדה זה קובע כיצד יש לפרש את המידע שיופיע בהמשך בשדה ה-RDATA.
- **CLASS:** שדה בן שתי אוקטטות המגדיר את מחלקת הרשת (בפרויקט זה נעשה שימוש ב-**IN** לציון Internet).
- **(TTL Time To Live):** מספר שלם חיובי (**Unsigned 32-bit**) המציין את משך הזמן (בשניות) שבו מותר לתחנות ברשת לשמור את הרשומה בזיכרון המטמון (**Cache**). ערך **0** מציין כי אין לשמור את הרשומה ב-Cache ויש להשתמש בה באופן חד-פעמי עבור השאילתה הנוכחית.
- **RDLENGTH:** מספר שלם (**Unsigned 16-bit**) המגדיר את אורך שדה ה-RDATA באוקטטות (בתים).
- **RDATA:** שדה באורך משתנה המכיל את נתוני המשאב עצמם (למשל, כתובת ה-IP). פורמט המידע בשדה זה נקבע באופן בלעדי על פי ערכי ה-**Type** וה-**Class**.

---

## מנגנון דחיסת שמות (DNS Compression)

כדי לצמצם את נפח החבילה ולמנוע חזרתיות של שמות מתחם ארוכים, פרוטוקול ה-DNS עושה שימוש במצביעים:

במקום לחזור על שם ה-Domain במלואו, נעשה שימוש בשתי אוקטטות מיוחדות. שתי הביטים הראשונים באוקטטה הראשונה יהיו תמיד **11** (ערך המיוצג כ-**`0xc0`** בהקסדצימלי), סימון המורה למפענח שמדובר במצביע. שאר 14 הביטים הנותרים מהווים את ה-**Offset** (היסט) – הם מצביעים על המיקום המדויק (בבתים) מתחילת הפקטה שבו מופיע שם ה-Domain לראשונה.

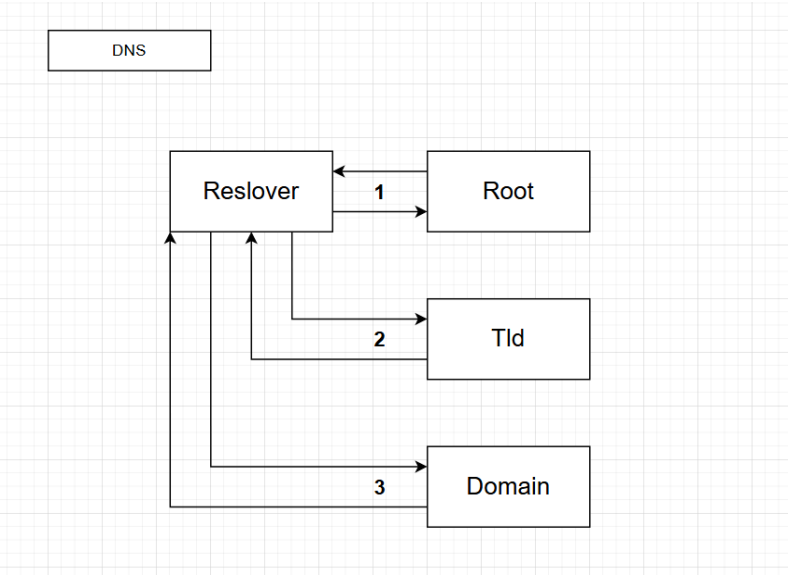
---

דוגמה מה-RFC לפקטת תשובה:

```
                +-----------------------------------------+
   Header       |         OPCODE=RESPONSE, ID=997         |
                +-----------------------------------------+
  Question      |QTYPE=A, QCLASS=IN, QNAME=VENERA.ISI.EDU |
                +-----------------------------------------+
   Answer       |  VENERA.ISI.EDU  A IN 10.1.0.52         |
                +-----------------------------------------+
 Authority      |                  <empty>                |
                +-----------------------------------------+
Additional      |                  <empty>                |
                +-----------------------------------------+
```

את הפקטה שנרכיב נשלח לResolver שלנו (לרוב הראוטר הביתי) ונקבל חזרה תשובה.
במידה והראוטר מכיר את הכתובת (כמובן שאם הTTL של הcache נגמר הוא לא יזכור אותה) הוא יחזיר אותה.
במידה ולא, אנחנו נצטרך לטייל בעץ שרתי הDNS לקבלת תשובה.
במידה ומדובר בהליך רקורסיבי - הResolver שלנו יבצע שליחה וקבלת תשובות, ויחזיר לנו רק את התשובה הסופית.
במידה ומדובר בהליך איטרטיבי - הResolver יפנה אותנו לשרת הרלוונטי והפנייה לשרתים תתבצע מהמחשב שלנו (ועל כן נראה יותר פקטות בwireshark).

**תרשים של היירכיית החיפוש כאשר מדובר בחיפוש רקורסיבי:**




נשים לב הפנייה לשרתים השונים מתבצעת על ידי הResolver - מכיוון שמדובר בבקשה רקורסיבית.
מכיוון שהname servers השונים (Root,TLD,Domain) מאוד דומים במהותם ביכולתם החלטנו על הגדרת מחלקה אבסטרקטית שתשמש את השרתים האלו.

נתחיל מלדבר על האתחול:
אנחנו נקבל מהקלט את שם השרת (להדפסות השונות) את כתובת הIP שתשמש אותנו בהמשך לטובת פעילות השרת, את שם קובץ הconfig ודגל שמסמן לנו האם מצב dnssec מופעל (יפורט בהמשך).
נשים לב שכדי להגיע לקובץ הconfig נצטרך לחזור אחורה לroot של הפרוייקט ולעבור לתיקיית config.
לאחר מכן נקרא לפונקציות load_config שתקבל את כתובת הIP של השרת
ולפונקציה load_zone_data שתפקידה לטעון את המידע על הzone.
השרת עדיין לא רץ ועל כן נגדיר אותו כך וללא socket.

```
class AbstractNameServer(ABC):
    def __init__(self, server_name, default_ip, config_filename, dnssec_enabled=False):
        print(f"[*] Booting {server_name}...")
        self.dnssec_enabled = dnssec_enabled

        # Path Resolution
        self.project_root = Path(__file__).resolve().parent.parent
        self.config_path = self.project_root / "configs" / config_filename

        # Hot-Reloading State
        self._zone_mtimes = {}
        self._watcher_event = threading.Event()

        # Load Config and Zone Data
        self._load_config(default_ip)
        self.zone_records = self._load_zone_data()

        self.running = False
```

```
        self.server_sock = None
```

## פונקציה load_config:

בהתאם לconfig שהוגדר בתהליך האתחול, נפתח את הקובץ המתאים ונקח ממנו את ההגדרות המופיעות.
נשים לב שלכל אחת מההגדרות יש fallback למקרה שלא מוגדר בקובץ, את הIP גם קיבלנו בתהליך האתחול.

```
def _load_config(self, default_ip):
        if not self.config_path.exists():
            raise FileNotFoundError(f"[FATAL] Config file missing: {self.config_path}")

        with open(self.config_path, 'r') as f:
            config = yaml.safe_load(f)
            self.ip = config['server'].get('bind_ip', default_ip)
            self.port = config['server'].get('bind_port', 53)
            self.buffer_size = config['server'].get('buffer_size', 512)
            self.max_workers = config['server'].get('max_workers', 100)
            self.zone_directory_path = self.project_root /
config['data'].get('zone_directory', 'zones/auth/')
```

## *פונקצייה load_zone_data:*

נקח את הנתיב שהוגדר בתהליך התאחול. נבצע בדיקה במידה והנתיב לא קיים או שלא מדובר בנתיב - נציג שגיאה בהתאם.
נתאחל את הערכים נמשיך על ידי מעבר על כל קבצי הzone. בנתיב. נבדוק האם מדובר בקובץ חתום - שמסמן לנו שמדובר בdnssec. במידה ואנחנו במצב dnssec והקובץ הוא לא קובץ חתום, נדלג עליו במידה ואנחנו לא במצב dnssec והקובץ הוא קובץ חתום, נדלג עליו. במידה והקובץ רלוונטי לנו, נפתח אותו ונעבור עליו. נעבור על כל רשומה רלוונטית שקיימת ונכניס אותה לzone_db שיחזיק לנו את הרשומות. בסיום נדפיס הודעה. אחרי סיום טעינת כל הרשומות בקובץ נעדכן בהודעה ונחזיר את zone_db.

```
def _load_zone_data(self) -> dict:
        zone_dir = self.zone_directory_path
        if not zone_dir.exists() or not zone_dir.is_dir():
            print(f"[FATAL] Zone directory not found at {zone_dir}.")
            sys.exit(1)

        zone_db = {}
        new_mtimes = {}
        loaded_files = 0
        total_records = 0

        for zone_file in zone_dir.glob("*.zone"):
            is_signed_file = zone_file.name.endswith(".signed.zone")

            # Filter logic for DNSSEC
            if self.dnssec_enabled and not is_signed_file:
                continue
            if not self.dnssec_enabled and is_signed_file:
                continue
```

```
            try:
                # Track file modification time for hot-reloading
                new_mtimes[str(zone_file)] = zone_file.stat().st_mtime

                with open(zone_file, 'r') as f:
                    zone_text = f.read()

                parsed_records = RR.fromZone(zone_text)
                loaded_files += 1
                total_records += len(parsed_records)

                for rr in parsed_records:
                    name = str(rr.rname)
                    rtype = rr.rtype
                    zone_db.setdefault(name, {}).setdefault(rtype, []).append(rr)

                print(f"[*] Loaded {len(parsed_records)} records from {zone_file.name}")
            except Exception as e:
                print(f"[ERROR] Failed to parse {zone_file.name}: {e}")

        # Atomically update the watched timestamps
        self._zone_mtimes = new_mtimes
        print(f"[*] Successfully loaded {total_records} total records across
{loaded_files} files.")
        return zone_db
```

מכיוון שמדובר פה ב-3 שרתים שונים שכל אחד מפענח בקשה בצורה אחרת,
נגדיר פונקציה אבסטרקטית בשם handle_query

```
    @abstractmethod
    def handle_query(self, data, addr, sock):
        """Each server type implements its own resolution logic here."""
        pass
```

## פונקציית start:

פונקצייה להקמת השרת והפעלתו, תרוץ בשרתים עצמם שיורשים מהמחלקה.
נגדיר את הsocket וננסה להריץ את השרת.
נריץ בנוסף את הפונקצייה listening_loop בthread נפרד שתחכה לקבל בקשות.
במידה וקיבלנו שגיאת OS - סימן שלא הצלחנו להקצות את השרת בכתובת ובפורט שרצינו.
ובדומה לאפליקציות רבות אחרות, כאשר נלחץ ctrl+c נפעיל את פונקציית stop שתסגור את השרת.

```
def start(self):
        self.server_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        self.server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        if hasattr(socket, 'SO_REUSEPORT'):
            self.server_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEPORT, 1)
```

```
        try:
            self.server_sock.bind((self.ip, self.port))
            self.running = True

            listener = threading.Thread(target=self._listening_loop)
            listener.daemon = True
            listener.start()

            # Start the hot-reload watcher thread
            watcher = threading.Thread(target=self._zone_watcher_loop)
            watcher.daemon = True
            watcher.start()

            # Persistent sleep event
            sleep_event = threading.Event()
            while self.running:
                sleep_event.wait(1)

        except KeyboardInterrupt:
            self.stop()
        except OSError as e:
            print(f"[FATAL] Could not bind server: {e}")
```

## פונקציה listening_loop:

הפונקצייה תנסה לקרוא את המידע שנשלח. אותו נעביר בthread נוסף לפונקציה handle_query (שכמו שאמרנו היא לא ממומשת במחלקה האבסטרקטית).

```
def _listening_loop(self):
        print(f"[*] Server Active on {self.ip}:{self.port} (Max Workers:
{self.max_workers})")

        with ThreadPoolExecutor(max_workers=self.max_workers) as executor:
            while self.running:
                try:
                    data, addr = self.server_sock.recvfrom(self.buffer_size)
                    executor.submit(self.handle_query, data, addr, self.server_sock)
                except OSError:
                    break
```

## פונקציה stop:

מטרת הפונקציה היא לאפשר כיבוי "לפי הספר". בדומה לנהוג במספר תוכנות אחרות הגדרנו את הפונקציה לרוץ כאשר לוחצים על Ctrl+C.
בקריאה לפונקציה תוצג הודעה בהתאם, נכבה את הערך הבוליאני שמייצג את ריצת השרת.
ונבצע בדיקה האם ה-socket פתוח. מכיוון שהפונקציה יכולה לרוץ בכל רגע נתון, יכול להיות מצב שנקרא לה גם כאשר הsocket אינו פתוח ועל כן חשובה הבדיקה.
במידה וה-socket פתוח נשחרר אותו על מנת לסמן למערכת ההפעלה שהשימוש בפורט הסתיים ואפשר להקצות אותו לדברים אחרים.

```
def stop(self):
        print(f"\n[*] Shutting down server...")
        self.running = False
        self._watcher_event.set() # Unblock the watcher thread instantly
        if self.server_sock:
            self.server_sock.close()
        sys.exit(0)
```

## פונקציה zone_watcher_loop:

תפקידה לדגום את הzone-ים כל 5 שניות על מנת לאפשר שינויים של הקבצים (ידנית או דרך הGUI שיצרנו) בלי הצורך להריץ מחדש את השרת. במידה ונמצאו שינויים נבצע טעינה מחודשת של הפרטים.

```
def _zone_watcher_loop(self):
        """Polls zone files every 5 seconds and hot-reloads them if changes are
detected."""
        while self.running:
            # Wait for 5 seconds, but wake up instantly if the server stops
            self._watcher_event.wait(5.0)
            if not self.running:
                break

            try:
                current_mtimes = {}
                zone_dir = self.zone_directory_path

                if zone_dir.exists() and zone_dir.is_dir():
                    for zone_file in zone_dir.glob("*.zone"):
                        is_signed_file = zone_file.name.endswith(".signed.zone")
                        if self.dnssec_enabled and not is_signed_file:
                            continue
                        if not self.dnssec_enabled and is_signed_file:
                            continue

                        current_mtimes[str(zone_file)] = zone_file.stat().st_mtime

                # If a file was added, deleted, or modified, trigger a hot reload
                if current_mtimes != self._zone_mtimes:
                    print("\n[*] Zone file change detected! Hot-reloading records...")
```

```
                self.zone_records = self._load_zone_data()
        except Exception as e:
            print(f"[ERROR] Zone watcher failed: {e}")
```

מימוש השרתים עצמם

נתחיל מRoot_Name_Server:

תהליך האתחול שלו יכלול העברת הפרטים שמתקבלים בעת יצירת השרת לinit של האבא. בנוסף נקבע את שם השרת ואת הIP שלו שנעביר גם.

```
class LocalRootServer(AbstractNameServer):
    def __init__(self, config_filename="root_config.yaml", dnssec_enabled=False):
        super().__init__("Local Root Server",'127.0.0.3',config_filename,dnssec_enabled)
```

פונקצייה handle_query:

זו הפונקצייה האבסטרקטית שתמומש בצורה שונה לכל שרת.
נגדיר מספר הגדרות לחבילה שתישלח.
לאחר מכן נחלץ את הTLD על ידי שימוש בפונקצייה extract_tld.
נבדוק האם השרת מכיר את הTLD המדובר. במידה וכן נדפיס הודעה בהתאם
נעבור על כל הרשומות המתאימות מסוג NS (name server) ונחפש את כתובת הIP של אותו שרת.

```
    def handle_query(self, data, addr, sock):
        try:
            request = DNSRecord.parse(data)
            qname = str(request.q.qname)

            reply = request.reply()
            reply.header.ra = 0
            reply.header.aa = 1

            tld = self.extract_tld(qname)

            if tld in self.zone_records and getattr(QTYPE, 'NS') in
self.zone_records[tld]:
                print(f"[*] Delegating {qname} to .{tld} nameservers.")

                # 1. Add NS records to Authority Section
                for ns_rr in self.zone_records[tld][getattr(QTYPE, 'NS')]:
                    reply.add_auth(ns_rr)

                    # 2. Find the Glue A record for this NS and add to Additional
Section
                    target_ns = str(ns_rr.rdata)
                    if target_ns in self.zone_records and getattr(QTYPE, 'A') in
self.zone_records[target_ns]:
                        for a_rr in self.zone_records[target_ns][getattr(QTYPE, 'A')]:
                            reply.add_ar(a_rr)

                if self.dnssec_enabled and getattr(QTYPE, 'TXT') in
self.zone_records[tld]:
```

```
                        for txt_rr in self.zone_records[tld][getattr(QTYPE, 'TXT')]:
                            txt_data = str(txt_rr.rdata).strip('"')
                            if txt_data.startswith("DS|") or
txt_data.startswith("RRSIG|DS|"):
                                reply.add_auth(txt_rr)
                                print(f"    [+] DNSSEC: Attached DS and RRSIG for
{tld}")
            else:
                print(f"[*] NXDOMAIN: Unknown TLD '{tld}' for query {qname}")
                reply.header.rcode = getattr(RCODE, 'NXDOMAIN')

            sock.sendto(reply.pack(), addr)
        except Exception as e:
            print(f"[ERROR] Handling query: {e}")
```

כאשר מצב dnssec מופעל מלבד ההפנייה לשרת הבא, נצרף גם רשומת DS, הרשומה הזו מייצגת לנו את הhash של המפתח של השרת הבא בתור. (השרת הנוכחי "מכיר" את כל השרתים שהוא סומך עליהם ואת המפתח שלהם).
את הרשומה השרת יחתום באמצעות רשומת RRSIG.
במידה ולאחר העברה נראה חוסר תאימות בין המפתח שהעברנו (המצופה) לבין המפתח של השרת בפועל, סימן שנערכה ההפנייה לשרת אחר, ועל כן נשברה שרשרת האמון. כך יידע הresolver האם בוצעה הפנייה לשרת אחר.
במידה והשרת שלנו לא מכיר את הTLD נציג הודעה בהתאם ונוסיף לתשובה שלנו שגיאת NXDOMAIN (Non-Existent Domain)

## פונקציה extract_tld:

פונקציה פשוטה שתפקידה לחלץ את שרת הTLD מתוך הטקסט של הכתובת שקיבלנו.

```
    def extract_tld(self, qname: str) -> str:
        parts = qname.strip(".").split(".")
        if len(parts) > 0:
            return parts[-1] + "."
        return ""
```

## Tld_name_server:

הוא השרת הבא בתור, מאותחל בצורה דומה לשרת הROOT אך כמובן ששמו אחר ויש לו כתובת אחרת 127.0.0.4

```
class LocalTLDServer(AbstractNameServer):
    def __init__(self, config_filename="tld_config.yaml", dnssec_enabled=False):
        super().__init__(
            "Local TLD Server (.homelab)", "127.0.0.4", config_filename, dnssec_enabled
        )
```

## פונקציית handle_query:

בדומה לroot נחלץ את החלק של הכתובת - הפעם את הdomain.
נחפש את השרת הרוולנטי ונעביר אליו את הבקשה על ידי חילוץ כתובת ה-IP שלו.
בדומה, אם מצב dnssec מופעל נעביר את המפתח לשרת הבא, כדי שהresolver יידע אם נשברה שרשרת האמון.

```python
def handle_query(self, data, addr, sock):
        try:
            request = DNSRecord.parse(data)
            qname = str(request.q.qname)

            reply = request.reply()
            reply.header.ra = 0
            reply.header.aa = 0  # TLD is not the final authority for A records

            domain = self.extract_domain(qname)

            if domain in self.zone_records and QTYPE.NS in self.zone_records[domain]:
                print(f"[*] Delegating {qname} to {domain} nameservers.")

                # 1. Add NS records to Authority Section
                for ns_rr in self.zone_records[domain][QTYPE.NS]:
                    reply.add_auth(ns_rr)

                    # 2. Find the Glue A record
                    target_ns = str(ns_rr.rdata)
                    if (
                        target_ns in self.zone_records
                        and QTYPE.A in self.zone_records[target_ns]
                    ):
                        for a_rr in self.zone_records[target_ns][QTYPE.A]:
                            reply.add_ar(a_rr)

                    if self.dnssec_enabled and QTYPE.TXT in self.zone_records[domain]:
                        for txt_rr in self.zone_records[domain][QTYPE.TXT]:
                            txt_data = str(txt_rr.rdata).strip('"')
                            if txt_data.startswith("DS|") or txt_data.startswith(
                                "RRSIG|DS|"
                            ):
                                reply.add_auth(txt_rr)
                                print(f"    [+] DNSSEC: Attached DS and RRSIG for
{domain}")
            else:
                print(f"[*] NXDOMAIN: Domain '{domain}' not registered in this TLD.")
                reply.header.rcode = RCODE.NXDOMAIN

            sock.sendto(reply.pack(), addr)
        except Exception as e:
            print(f"[ERROR] Handling query: {e}")
```

בדומה לפונקציה extract_tld כאן יש לנו פונקציה בשם extract_domain שמחלצת לנו את הdomain

```python
def extract_domain(self, qname: str) -> str:
        """
        Extracts the domain and TLD from a query.
        E.g., 'www.test.homelab.' -> 'test.homelab.'
        """
```

```
        parts = qname.strip(".").split(".")
        if len(parts) >= 2:
            return f"{parts[-2]}.{parts[-1]}."
        return qname
```

## לבסוף auth_name_server:

הכתובת של השרת היא 127.0.0.5

```
class LocalAuthServer(AbstractNameServer):
    def __init__(self, config_filename="auth_config.yaml", dnssec_enabled=False):
        super().__init__(
            "Local Auth Server", "127.0.0.5", config_filename, dnssec_enabled
        )
```

## פונקציה handle_query:

מכיוון שהשרת הזה הוא הכתובת האחרונה, נסמן את דגל aa שמאשר לנו שזו תשובה אמינה. נבדוק מה סוג השאילתה שהתקבלה.

אופציה 1: יש לנו התאמה. נבדוק האם יש לנו גם רשומה מהסוג המבוקש. במידה וכן נוסיף את התשובה לחבילה.

לאחר מכן נבצע גם בדיקה האם dnssec מופעל. במידהה וכן נחפש רשומת RRSIG ונצרף אותה לתשובה.

```
    def handle_query(self, data, addr, sock):
        try:
            request = DNSRecord.parse(data)
            qname = str(request.q.qname)
            qtype = request.q.qtype

            reply = request.reply()
            reply.header.ra = 0
            reply.header.aa = 1

            # Use our new zone_records (renamed from a_records)
            if qname in self.zone_records:
                node = self.zone_records[qname]

                # CASE 1: They asked for a specific record type (like A or NS)
                if qtype in node:
                    for rr in node[qtype]:
                        reply.add_answer(rr)
                        print(f"[*] ANSWER: Appended {QTYPE[qtype]} record for {qname}")

                    if self.dnssec_enabled and QTYPE.TXT in node:
                        for txt_rr in node[QTYPE.TXT]:
                            txt_data = str(txt_rr.rdata).strip('"')
        # If the TXT record is an RRSIG for the record type we just answered, attach it!
                            if txt_data.startswith(f"RRSIG|{QTYPE[qtype]}|"):
                                reply.add_answer(txt_rr)
```

```
                                print(f"    [+] DNSSEC: Attached RRSIG for
{QTYPE[qtype]}")
```

מצב 2: הלקוח ביקש רשומה מסוג A (ipv4) וקיימת רק רשומה מסוג CNAME (כינוי). השרת יוסיף את רשומת ה-CNAME שקיימת אצלו, ובמידה וקיימת לנו רשומת RRSIG נוסיף גם אותה.
מנגנון הchasing, כדי לחסוך למשתמש בקשה נוספת נבדוק האם יש לנו רשומה מסוג A שמתאימה לCNAME שמצאנו. אם כן נוסיף אותה גם לתשובה בצירוף החתימה (אם יש).

```
# CASE 2: CNAME resolution (They asked for A, but we only have CNAME)
                elif QTYPE.CNAME in node and qtype == QTYPE.A:
                    for cname_rr in node[QTYPE.CNAME]:
                        reply.add_answer(cname_rr)

                        # Attach CNAME Signature
                        if self.dnssec_enabled and QTYPE.TXT in node:
                            for txt_rr in node[QTYPE.TXT]:
                                if
str(txt_rr.rdata).strip('"').startswith("RRSIG|CNAME|"):
                                    reply.add_answer(txt_rr)
                                    print(f"    [+] DNSSEC: Attached RRSIG for CNAME")

                        # CNAME Chasing: Do we also have the A record for the target?
                        target_name = str(cname_rr.rdata)
                        if (
                            target_name in self.zone_records
                            and QTYPE.A in self.zone_records[target_name]
                        ):
                            target_node = self.zone_records[target_name]
                            for target_a_rr in target_node[QTYPE.A]:
                                reply.add_answer(target_a_rr)
                                print(f"[*] CNAME CHASE: Appended A record for
{target_name}")

                            # Attach Chased A Record Signature
                            if self.dnssec_enabled and QTYPE.TXT in target_node:
                                for txt_rr in target_node[QTYPE.TXT]:
                                    if
str(txt_rr.rdata).strip('"').startswith("RRSIG|A|"):
                                        reply.add_answer(txt_rr)
                                        print(f"    [+] DNSSEC: Attached RRSIG for
Chased A record")
```

מצב 3: השם קיים, אך אין לנו רשומה מהסוג הלוונטי. נציג הודעה בהתאם

```
# CASE 3: The name exists, but not the requested type
                else:
                    print(f"[*] NODATA: Name '{qname}' exists, but no {QTYPE[qtype]}
records.")
```

מצב 4: לא קיים הדומיין.
נציג הודעת NXDOMAIN בהתאם

```
            else:
                print(f"[*] NXDOMAIN: Domain '{qname}' does not exist.")
                reply.header.rcode = RCODE.NXDOMAIN
```

במידה והשרת לא החזיר תשובה (מצבים 3 ו4) השרת יוסיף רשומת SOA שתוכיח שאכן אין נתונים. תפקידה של הרשומה להגיד ללקוח - אני השרת בעל הסמכות לzone הזה, והרשומה שאתה מחפש לא קיימת. כדי למצוא את רשומת ה-SOA נצטרך לחפש אותה בעץ, אך בכיוון ההפוך ממה שחיפשנו עד עכשיו (נעלה חזרה למעלה). את הSOA שנמצא נוסיף לחלק הAuthority בחבילה. נשים לב שזו לא תשובה, אך יש לנו אישור שאכן אין תשובה. בהתאם נוסיף את מפתח הRRSIG של רשומת הSOA (אם קיים)

```
# If we didn't add any answers, we must provide the SOA record in the Authority section
            if len(reply.rr) == 0:
                parts = qname.strip(".").split(".")
                # Climb up the domain tree (x.test.homelab. -> test.homelab. ->
homelab.)
                for i in range(len(parts)):
                    apex = ".".join(parts[i:]) + "."
                    if apex in self.zone_records and QTYPE.SOA in
self.zone_records[apex]:
                        # 1. Add the SOA record
                        for soa_rr in self.zone_records[apex][QTYPE.SOA]:
                            reply.add_auth(soa_rr)
                            print(f"    [*] Attached SOA record for {apex} to Authority
section.")

                        # 2. Add the RRSIG for the SOA record (if DNSSEC is enabled)
                        if self.dnssec_enabled and QTYPE.TXT in self.zone_records[apex]:
                            for txt_rr in self.zone_records[apex][QTYPE.TXT]:
                                if
str(txt_rr.rdata).strip('"').startswith("RRSIG|SOA|"):
                                    reply.add_auth(txt_rr)
                                    print(f"    [+] DNSSEC: Attached RRSIG for SOA
record")
                        break

            sock.sendto(reply.pack(), addr)

        except Exception as e:
            print(f"[ERROR] Handling query: {e}")
```

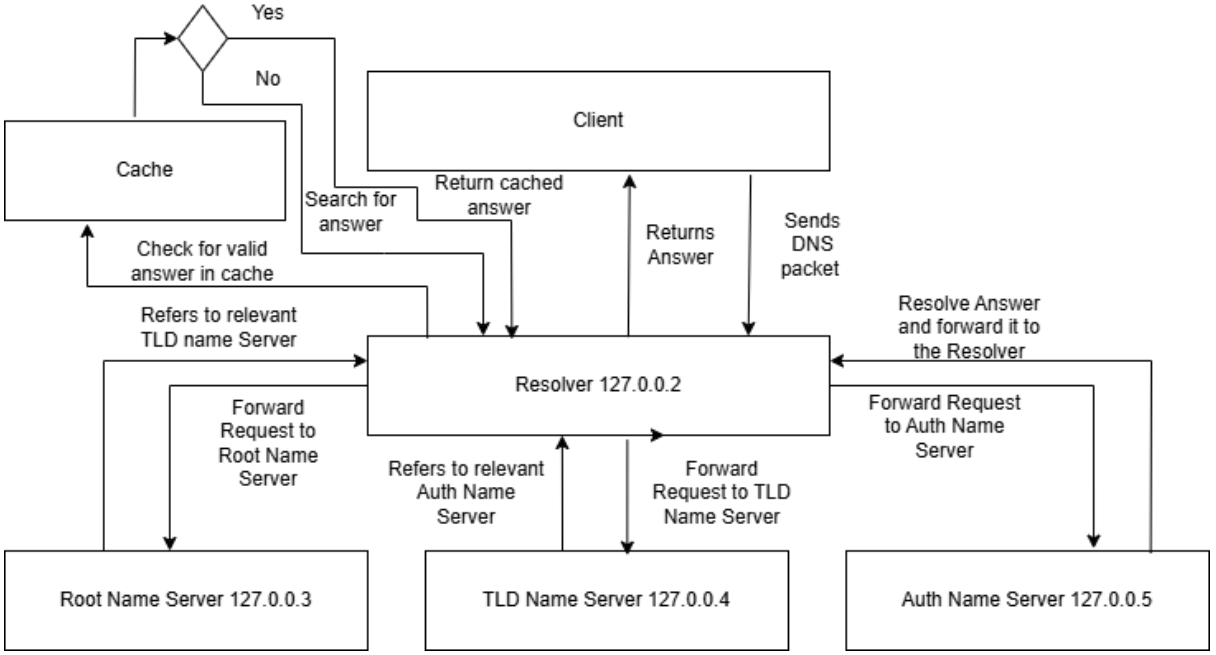
הסבר על התהליך:
כאשר נרצה לבדוק את המערכת שלנו נוכל לשלוח בקשת Resolve-DnsName דרך הpowershell. נשים לב ששרת הresolver שלנו אינו השרת שמוגדר במחשב, ועל כן נרשום את כתובת השרת ידנית.

הפקודה הזו יודעת להדפיס את התשובה, על פי תוכן החבילה שמוחזרת.

```
PS C:\Users\stets> Resolve-DnsName -Name server1.test.homelab -Server 127.0.0.2 -DnssecOk

Name                                           Type   TTL   Section    IPAddress
----                                           ----   ---   -------    ---------
server1.test.homelab                           A      86400 Answer     192.168.1.100

Name      : server1.test.homelab
QueryType : TXT
TTL       : 86400
Section   : Answer
Strings   : {RRSIG|A|MEQCIFmjq+CQcuA+WM2/PqDR7/UdwvTpWa+wIBasTB/0642YAiAvp9duIZsUT5Go+SNdGd2SQFUTC7a1jkUJZNvTlZDrkw==}

PS C:\Users\stets>
```

איך זה עובד מאחורי הקלעים:
את הפנייה ששלחנו לResolver נעביר את השלבים בהתאם בעזרת הפונקציה resolve_iterative.

המימוש בפרוייקט שלנו (הסבר):
גרף התהליך של הDNS:




נשים לב שלא הוספנו בגרף על dnssec שמפורט בהמשך. גם כשמצב dnssec מופעל התהליך זהה - רק נוספות בדיקות על החתימות.

בפרוייקט שלנו ביצענו הדמייה להיררכיית הDNS על ידי הקמת שרתים מקומיים לכל אחד מרמות העץ.
כדי לאפשר עבודה בפורט הסטנדרטי המוגדר 53, השתמשנו בכתובות הloopback המוגדרות על פי rfc 5735. על פי המוגדר בrfc ובשונה מהנחת היסוד, יש מספר כתובות loopback אשר נמצאות במרחב הכתובות 127.0.0.0/8.
בעזרת הגדרה זו נוכל להריץ מספר שרתים **באותו הפורט** על המחשב, ובכל זאת לקבל כתובת ייחודית לכל שרת - כך שהsocket-ים יהיו שונים.
בנוסף, השארנו מנגנון fallback למקרים בהם השרתים שלנו לא מגיבים או מסרבים לתת שירות או פשוט לא מכירים את הכתובת המבוקשת, ובמקרים האלה הforwarder יעביר את הבקשה לשרת DNS הציבורי של Google בכתובת 8.8.8.8.

נראה דוגמה לתהליך:
כדי לאפשר עבודה נוחה יותר על השרתים, יצרנו UI בelectron שייאפשר לנו עריכה.
נחפש למשל את הכתובת server1.test.homelab. נשים לב שמכיוון שלא הגדרנו את שרת הResolver להיות השרת ברירת המחדל של המחשב, בעת הפנייה עם הפונקציה nslookup נציין לאחר כתובת הDomain את כתובת הResolver שהגדרנו

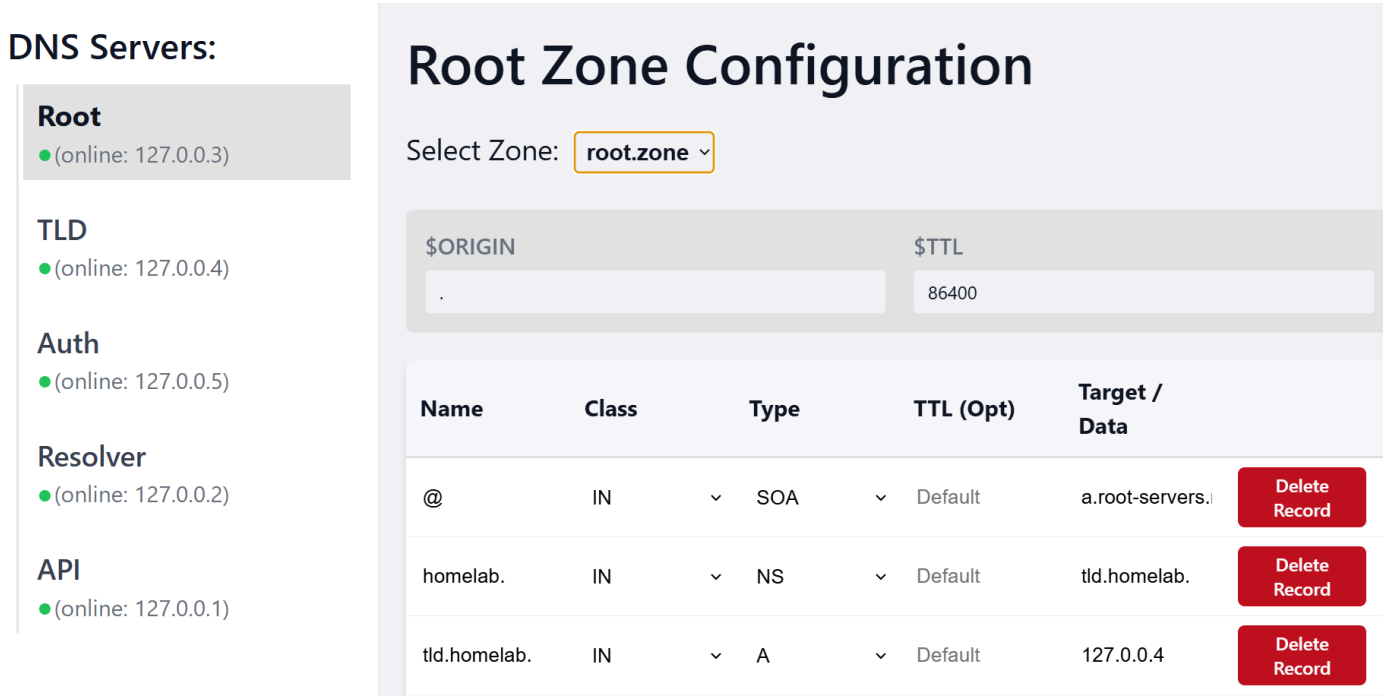
אותה קבועה בכתובת 127.0.0.2.
(מכיוון שקיבלנו את התשובה מה-Resolver התשובה לא מסומנת כ-nslookup.(authoritative

```
server1.test.homelab 127.0.0.2
Server:  UnKnown
Address:  127.0.0.2

Non-authoritative answer:
Name:    server1.test.homelab
Address:  192.168.1.100
```

בצד ה-UI נוכל לראות את חלוקת השרתים.
ראשית תתבצע פנייה מה-Resolver לשרת ה-Root בכתובת 127.0.0.3 (מכיוון שהכתובת הזו לא שמורה ב-cache)

```
[CACHE MISS] Resolving A server1.test.homelab....
    [Iterative Jump 1] Asking 127.0.0.3 for server1.test.homelab.
[*] Delegating server1.test.homelab. to .homelab. nameservers.
    [*] Delegation received. Moving to: 127.0.0.4
```

נשים לב לשורה השנייה בUI בה ניתן לראות את הNS (name server) בשמו homelab.
השורה תפנה אותנו לשורה מסוג A שתתן לנו את כתובת ה-IP של שרת ה-TLD בכתובת 127.0.0.4

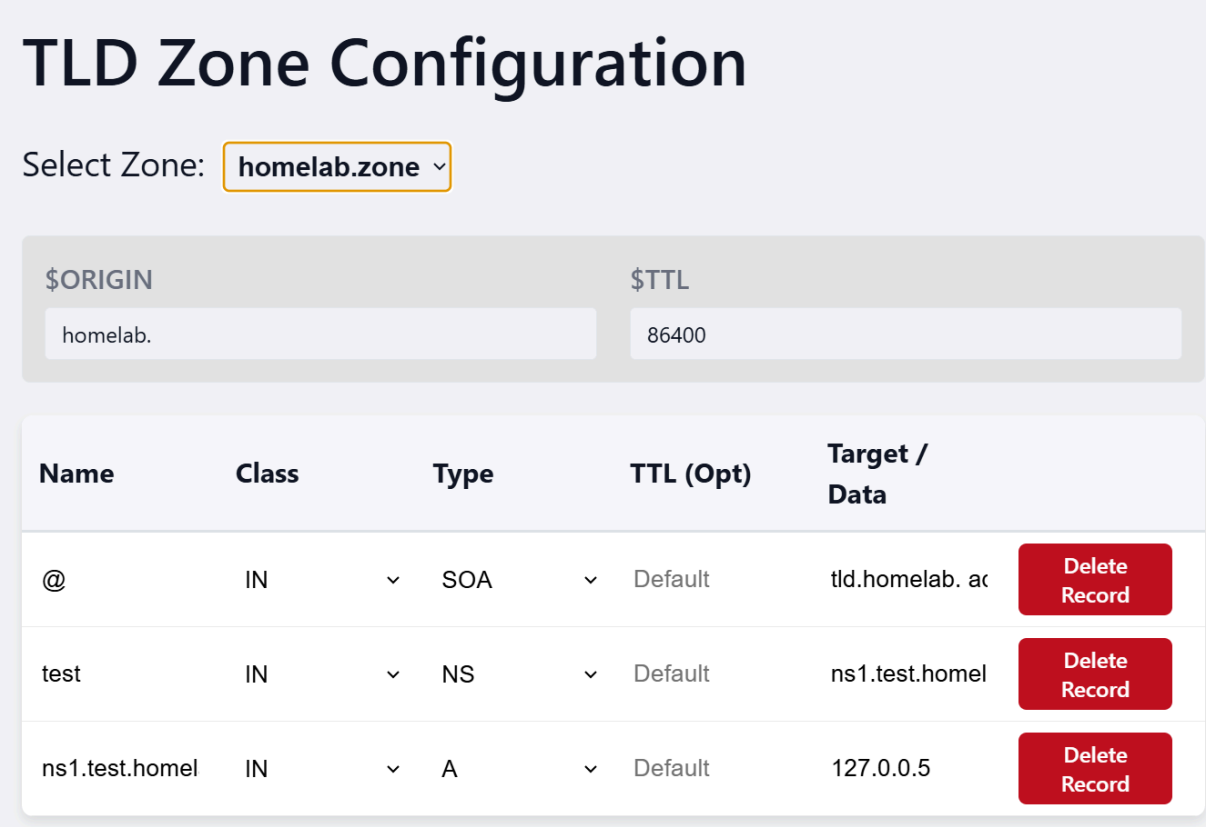



נשאל את שרת ה-TLD את השאלה והוא יפנה אותנו לשרת Auth בכתובת 127.0.0.5

```
 [Iterative Jump 2] Asking 127.0.0.4 for server1.test.homelab.
[*] Delegating server1.test.homelab. to test.homelab. nameservers.
```

```
    [*] Delegation received. Moving to: 127.0.0.5
```

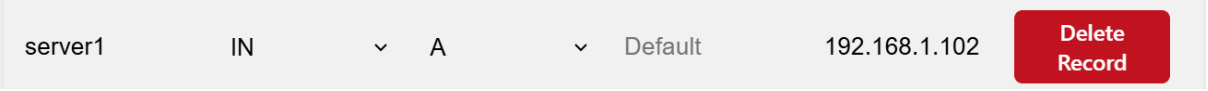
נשים לב שאכן מופיעה ההגדרה לשרת הauth בNS המתאים test:




לבסוף נקבל תשובה משרת ה-AUTH

```
[*] ANSWER: Appended A record for server1.test.homelab.
    [*] Final exact answer found at 127.0.0.5!
```

ואכן אצלו נמצאת התשובה הסופית שקיבלנו:




הוספנו גם מנגנון Fallback.
במידה ומדובר בDomain שלא שייך לשרתים שלנו (למשל אתר האוניברסיטה https://www.ariel.ac.il/) הבקשה תועבר לשרת הDNS של google בכתובת 8.8.8.8 כדי שנוכל לקבל כתובת גם במקרים של כתובות שלא מתוחזקות בשרתים הפנימיים שלנו

```
nslookup www.ariel.ac.il 127.0.0.2
Server:  UnKnown
Address:  127.0.0.2

Non-authoritative answer:
```

```
Name:    www.ariel.ac.il
Address:  31.214.213.20
```

וגם בלוגים נראה שאכן שרת ה-Root שלנו אינו מכיר את הTLD ולכן יעביר את הבקשה ל8.8.8.8

```
[CACHE MISS] Resolving A www.ariel.ac.il....
    [Iterative Jump 1] Asking 127.0.0.3 for www.ariel.ac.il.
[*] NXDOMAIN: Unknown TLD 'il.' for query www.ariel.ac.il.
    [*] NXDOMAIN from 127.0.0.3. Domain does not exist.
[*] Forwarding to Public DNS (8.8.8.8)...
```

לresolver גם קיים מנגנון (LRU CACHE (ordered dict שממומש בקובץ dns_cache.py.

```
class DNSCache:
    def __init__(self, filename: str = "cache.db", capacity: int = 1000, save_interval:
int = 5) -> None:
        """
        :param filename: Where to save the cache on disk.
        :param capacity: Max items in memory.
        :param save_interval: How often (in seconds) to check for autosave.
        """
        self.capacity: int = capacity
        self.filename: str = filename
        self.save_interval: int = save_interval

        self._store: OrderedDict[Tuple[str, int], Tuple[DNSRecord, float]] = OrderedDict()

        # --- THREADING LOCKS ---
        self._lock: threading.Lock = threading.Lock()     # Protects the in-memory
dictionary
        self._io_lock: threading.Lock = threading.Lock()  # Protects the disk to prevent
overlapping writes

        # Persistence flags
        self._dirty: bool = False  # Has the cache changed since last save
        self._running: bool = True

        # Load existing data immediately
        self._load_from_disk()

        # Start the Background Saver Thread
```

```
        self._saver_thread = threading.Thread(target=self._auto_save_loop, daemon=True)
        self._saver_thread.start()
```

השמירה לקובץ מתבצעת בעזרת הפונקציה save_to_disk

```
def _save_to_disk(self) -> None:
        with self._io_lock:
            with self._lock:
                if not self._dirty:
                    return

                snapshot = self._store.copy()
                self._dirty = False # Reset flag

            try:
                # 1. Write to a temporary file first (ATOMIC WRITE)
                temp_filename = f"{self.filename}.tmp"
                with open(temp_filename, 'wb') as f:
                    pickle.dump(snapshot, f)

                os.replace(temp_filename, self.filename)

            except Exception as e:
                # If the disk write fails, we MUST restore the dirty flag
                # so the data isn't silently lost forever.
                with self._lock:
                    self._dirty = True
                print(f"[!] Auto-save failed: {e}")
```

והקריאה בהתאם מתבצעת מהפונקציה load_from_disk

```
    def _load_from_disk(self) -> None:
        """Called only on startup."""
        if not os.path.exists(self.filename):
            return

        try:
            with open(self.filename, 'rb') as f:
                data = pickle.load(f)

                # Cleanup: Filter out expired items immediately on load
                now = time.time()
                valid_data = OrderedDict()
                for key, (record, exp) in data.items():
                    if exp > now:
                        valid_data[key] = (record, exp)

                with self._lock:
                    self._store = valid_data
```

```
            print(f"[*] Loaded {len(self._store)} records from {self.filename}")
        except Exception as e:
            print(f"[!] Failed to load cache: {e}")
```

את השמירה נריץ באופן קבוע  בפונקציה auto_save_loop מנת למנוע איבוד מידע.

```
def _auto_save_loop(self) -> None:
        """Background worker function."""
        while self._running:
            time.sleep(self.save_interval)
            self._save_to_disk()
```

נשים לב שלכל רשומה יש TTL - הרשומות לא תקפות לנצח ונוודא אותן על ידי בדיקה מחודשת אם פג התוקף של הרשומה.
כאשר ה-resolver יבצע query הוא קודם  כל יבצע בדיקה בcache.
רק במידה ולא תימצא תשובה יבצע את התהליך המלא

```
            # --- LEADER PROCESSING LOGIC ---
            try:
                # STEP 1: CACHE LOOKUP
                if self._handle_cache_lookup(request, qname, qtype, client_wants_dnssec,
addr, socket_ref):
                    print(f"[CACHE HIT] returning {QTYPE[qtype]} {qname} from cache.")
                    return

                print(f"[CACHE MISS] Resolving {QTYPE[qtype]} {qname}...")
```

מנגנון נוסף לשיפור הביצועים הוא זיהוי של בקשה כפולה בעת ביצוע. מכיוון שמדובר על בקשה בעת ביצוע - היא עוד לא קיימת ב-cache.
במידה וה-resolver מזהה מצב כזה, במקום לשלוח בקשה חדשה, הוא ימתין לתשובה וישלח את התשובה שתתקבל על ידי התהליך המקורי - גם לדורש הנוכחי.

```
            with self.lock:
```

```
                if query_key in self.in_flight_queries:
                    # Someone else is already fetching this! We will wait.
                    wait_event = self.in_flight_queries[query_key]
                else:
                    # We are the first to ask. We are the Leader.
                    wait_event = threading.Event()
                    self.in_flight_queries[query_key] = wait_event
                    is_leader = True
```

בהתאם, יחזיר את התשובה. במידה והבקשה המקורית נכשלה, נחזיר SERVFAIL

```
if not is_leader:
                print(f"[DEBUG] Coalescing duplicate query for {QTYPE[qtype]} {qname}.
Waiting for leader...")
                # Wait for the leader thread to finish fetching and save to cache
                wait_event.wait(timeout=self.config.timeout + 2.0)

                # The leader finished! Serve our identical request instantly from the
cache
                if self._handle_cache_lookup(request, qname, qtype, client_wants_dnssec,
addr, socket_ref):
                    return

                # If leader failed and cache is still empty, return a SERVFAIL
                error_reply = request.reply()
                error_reply.header.rcode = getattr(RCODE, 'SERVFAIL')
                socket_ref.sendto(error_reply.pack(), addr)
                return
```

מטרת המנגנון לאפשר שמירה של queries קודמים על מנת לחסוך פניות חוזרות. את הcache אנחנו נשמור בקובץ כדי לאפשר עמידות שלו בעת קריסה של השרת.

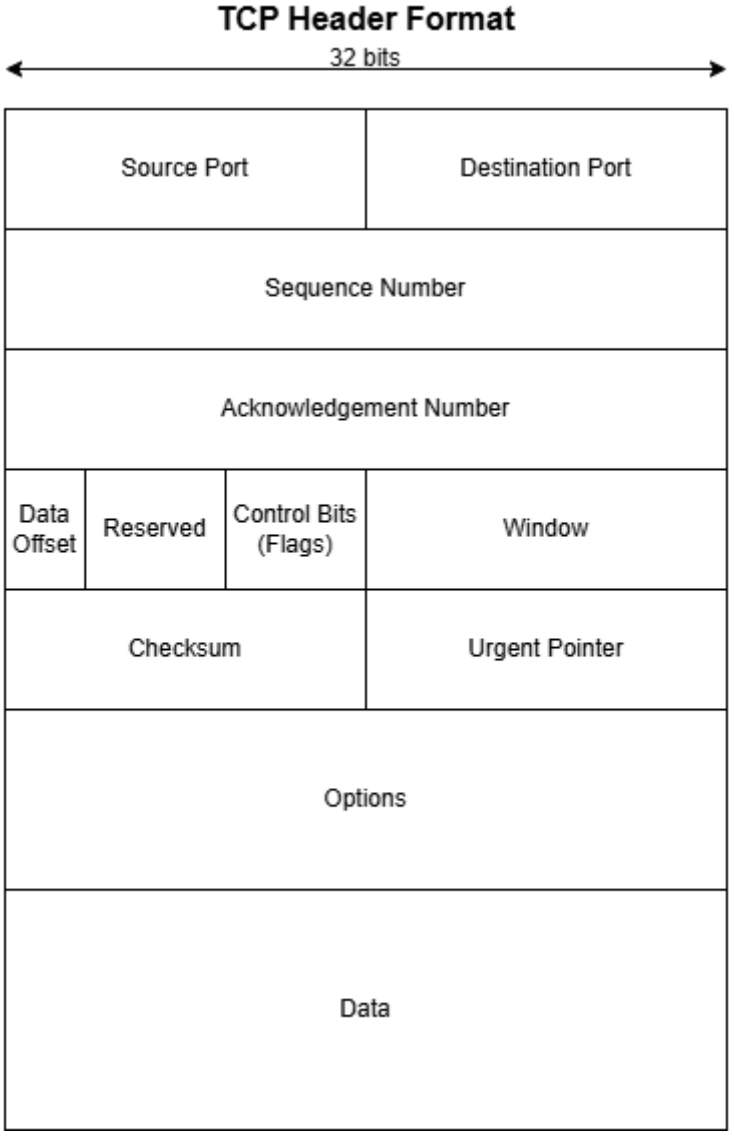
# Application

<u>פרוטוקול TCP:</u>

פרוטוקול TCP הינו פרוטוקול בשכבת התעבורה. תפקידו להעביר סגמנטים של מידע בין שני מכשירים ולאפשר ביניהם תקשורת שוטפת.

הפרוטוקול משמש אותנו לתקשורת של המשתמש אל מול אתר ה-proxy ובנוסף, לתקשורת של שרת ה-proxy אל מול שרת הסרטונים - אם בחרנו בפרוטוקול TCP.

מבנה הHeader של TCP:




1) Source Port - פורט מקור שממנו המכשיר שולח את המידע
2) Destination Port - פורט יעד אליו המכשיר שולח את המידע
3) Sequence Number - מספר הבית הראשון בdata. משמש לנו כדי שנדע לבנות את ההודעה בצד המקבל על פי הסדר. הערך הראשוני לא בהכרח יהיה 0 - והוא נקבע בתהליך הthree way handshake בין הצדדים
4) Acknowledgment Number - ממן את הsequence number הבא שאנחנו מצפים לקבל
5) Data Offset - ממן לנו איפה מתחיל החלק של הData (נשים לב שמכיוון שהחלק של הoptions הוא לא בגודל קבוע, תחילת הData משתנה). הגודל נמדד במילים באורך 32 ביט.
6) Reserved - שמור לשימוש עתידי - חייב להיות מלא באפסים.
7) Control Bits - הדגלים שנסמן

- URG - דגל Urgent מסמן למקבל שהסגמנט הזה "דחוף" ועל כן יש לתת לו עדיפות
- ACK - מסמן לנו שיש להתייחס לערך הקיים בשדה Acknowledgment Number
- PSH - מסמן למקבל שיש להעביר את המידע לאפליקציה מיידית, ללא צורך לחכות לbuffer שיתמלא.
- RST - כדי לבצע reset לחיבור
- SYN - דגל Syncronize נועד לבצע סנכרון של sequence number בין שני הצדדים (ייסומן בשני השלבים הראשונים של תהליך ההתחברות)
- FIN - מסמן לנו סיום של שליחת המידע.

8) Window - מספר האוקטטות שהשולח מאשר לקבל בחלון הדינמי שלו.
9) Checksum - שדה חישובי שמבצע חישוב על הHeader, הData וה-Pseudo Header (שכולל את כתובות ה-IP משכבת הרשת). תפקידו לוודא שהמידע לא השתבש (אם המקבל מחשב את הchecksum ורואה ערך שונה ממה שכתוב בשדה - סימן שהיה שיבוש במידע).
10) Urgent Pointer - מסמן לנו את סיום המידע ה"דחוף". הערך ייבדק רק אם מסומן דגל URG.
11) Options - שדה באורך משתנה שכולל בתוכו אפשרויות. במידה וגודל השדה אינו מתחלק ב-32 ביט יוסף padding של אפסים.
12) Data - שדה בגודל משתנה שכולל את המידע שאנחנו מעבירים בסגמנט.

התקשורת מתבצעת באמצעות socket - שהוא tuple שמכיל ארבעה ערכים - כתובת IP ופורט - לשרת וללקוח. ה-Socket משמש כשני נקודות הקצה (Endpoint) של 'צינור התקשורת' בין שני המכשירים.
נשים לב שלמכשיר יחיד - בעל אותה כתובת IP יכולים להיות מספר רב של תהליכים שונים הרצים על גבי פרוטוקול TCP, ועל שם כך כל תהליך יקבל פורט שונה, ועל כך יוקצה לו socket שונה. פרוטוקול TCP בהגדרה הוא פרוטוקול אמין. ובשם כך מובטח לנו הגעה וודאית ועל פי סדר של פקטות המידע, בשני צידי התקשורת.
איך מובטחת לנו אמינות המידע?
בכל שלב לתהליך התקשורת יש שלבים מוגדרים על מנת לוודא את אמינות ותקינות הקשר.
בתחילת התהליך יתבצע תהליך אשר מוכר בשם three way handshake
התהליך כשמו מורכב ב-3 שלבים:

1) SYN - יוזם הקשר (לרוב ייקרא הלקוח) ישלח פקטה עם דגל SYN ל-Socket ידוע מראש הנקרא welcoming socket. תפקידו של socket זה להיות יעד קבוע ומוכר אשר תפקידו לקבל את בקשת ההתחברות ולהקצות לחיבור socket ייעודי
2) SYN-ACK - לאחר קבלת הפקטה, השרת ייצור socket חדש לשם התקשרות, ויישלח פקטה עם דגלים SYN ו ACK שמסמנים את אישור השרת לקבלת בקשת הSYN.
3) ACK - הלקוח שיקבל פקטה עם דגלים SYN וACK יישלח חזרה פקטה עם דגל ACK לעדכון השרת שאישר את החיבור שקיבל

בסוף 3 השלבים אנחנו מוכנים להתחלת תהליך הקשר, גם השרת וגם הלקוח מכירים את ערוץ התקשורת שלהם בעזרת הSocket שהוגדר.

בעת התקשורת השוטפת בין הצדדים על כל סגמנט שיישלח מצד אחד צפוייה להתקבל בהמשך תשובה עם דגל ACK שתאשר לנו את קבלת המידע מהצד השני. איך נדע על איזה סגמנט קיבלנו אישור (ACK)? בעזרת שדה בסגמנט הTCP אשר נקרא sequence number.
לתהליך ה-TCP יש טיימר
שמופעל בעת שליחת הסגמנט, במידה ולא קיבלנו אישור על קבלת המידע, בעת פקיעת הטיימר הסגמנט יישלח מחדש.
כדי לוודא את אמינות המידע שהגיע, ושלא השתבש בדרך יש את שדה Checksum שתפקידו לבצע חישוב כלשהו שתלוי בביטים שהועברו, אם נראה שיש סתירה בין הערך הקיים שם לבין הערך שמחושב ביעד, נדע שהיה שיבוש כלשהו בהעברת המידע, ולכן לא נשלח על הסגמנט הזה ACK.
בפקיעת הטיימר הסגמנט יישלח מחדש.
נשים לב שבעזרת הsequence number נוכל לדעת גם את סדר השליחה ועל כן נוכל לסדר את הפקטות שהתקבלו על פי סדר השליחה.

Sequence number שומר לנו את המיקום של העברת המידע בבתים, כאשר יישלח ACK מספר הSequence Number ייסמן לנו אני מצפה לX byte (וכיוצא בזאת קיבלתי עד x-1 byte).

גם סיום התקשרות TCP מבוצעת בתהליך מוגדר אשר נקרא four way Handhake. אלו הם שלבי התהליך:

1) שליחת סגמנט עם דגל FIN מצד הלקוח - אשר מסמנת לשרת שהלקוח סיים מבחינתו את שליחת המידע.
2) החזרת סגמנט עם דגל ACK מצד השרת - לסימון קבלת ההודעה מהסעיף הראשון. בשלב הזה החיבור "חצי סגור" הלקוח סיים לשלוח מידע, אך השרת עצמו עדיין לא סיים את התקשורת מהצד שלו.
3) שליחת סגמנט עם דגל FIN מצד השרת - לסימון ללקוח שגם השרת סיים את העברת המידע - ורוצה לסיים את הקשר
4) שליחת סגמנט עם דגל ACK מהלקוח - למענה לסגמנט מסעיף 3 שמסמן שהלקוח קיבל את בקשת סיום ההתקשרות מהשרת.
נשים לב שהלקוח ימתין בשלב הזה לפני סגירת התקשורת, מכיוון שיכול להיות שהודעת ה-ACK שלו לא תגיע. אם בזמן שהוא מחכה יקבל פקטה עם דגל FIN - סימן שהשרת לא קיבל את תשובתו וישלח תשובה בשנית.

בסיום התהליך ההתקשרות הסתיימה, הsocket נסגר, ושני הצדדים סיימו את התקשורת. מכיוון שה-socket נסגר יהיה אפשר להשתמש בו לקשר אחר בהמשך בנוסף למנגון ה-ACK שדיברנו עליו, יש שני מנגנונים בתהליך השליחה, ששולטים על קצב שליחת המידע.

## מנגנון Flow Control

בחלק קודם של המסמך דיברנו על מנגנון האישורים שמשתמש בדגלי ACK. אם אכן היינו מחכים לאישור על כל פקטה שנשלח, השליחה הייתה איטית מאוד שכן היינו מחכים בכל פעם ל-ACK לפני שליחת מידע חדש. לשם ייעול התהליך משתמשים ב-Sliding Window. תפקיד ה-Sliding Window הוא להגדיר את כמות המידע בבתים שניתן להעביר בלי צורך לחכות לאישור.

המטרה של מנגנון Flow Control היא למנוע הצפה של מקבל המידע (שהוא אחד ה-Endpoints ב-Socket). כאשר נקבל אישור על סגמנט מתחילת החלון נוכל "להזיז" את בסיס החלון ולשלוח סגמנט חדש. גודל החלון ייקבע על ידי שדה `rwnd`. כל צד בתקשורת יחזיק Buffer שישמור את הסגמנטים שלא מעובדים בזה הרגע (אם נקבל סגמנטים מאוחרים יותר, או פשוט כמה סגמנטים במקביל). במידה ושדה `rwnd` מגיע ל-0, השולח מפסיק לשלוח מידע מכיוון שה-Buffer התמלא, ורק שולח סגמנטים זעירים כדי לדגום מתי אפשר לחזור לשלוח.

## מנגנון Congestion Control

מנגנון זה נועד להגן על הרשת עצמה. אם במנגנון Flow Control המטרה הייתה לוודא ששני צדדי התקשורת לא מוצפים, מטרת המנגנון הזה היא לוודא שלא מציפים את הרשת עצמה (בעצם התחנות שהמידע עובר ביניהן – שאנחנו לא חשופים אליהן ישירות). לשם כך יתוחזק שדה `cwnd`. בתחילת התקשורת איננו יודעים את מצב הרשת, ועל כן נתחיל בערך קטן.

על כל ACK שנקבל נדע שהמידע עבר ועל כן נגדיל את גודל החלון ב-1. בסך הכל נקבל הכפלה פי 2 (למשל, אם שלחנו 4 חבילות שהגיעו, נגדיל ב-4 את גודל החלון). כאשר נעבור סף מסוים של גודל החלון שנקרא Slow Start Threshold (ssthresh), השולח יקטין את קצב הגדלת החלון ויגדיל אותו ב-1 לא על כל ACK שקיבל, אלא על קבלת ACK על כל החבילות שנשלחו בחלון (גידול ליניארי).

**איך נדע האם עברנו את קצה גבול היכולת?**

- **Timeout:** אם הטיימר לקבלת ה-ACK פקע, זה נחשב עומס גדול, ועל כן נתחיל את התהליך מחדש בערך קטן. את ערך ה-Slow Start Threshold נחלק ב-2 בהתאם, כדי שבניסיון הבא נעבור להגדלת חלון איטית באזור רלוונטי יותר.
- **3 Duplicate ACKs:** אם קיבלנו ACK כפול על אותו סגמנט, סימן שהגיע סגמנט מאוחר יותר והיה דילוג על הקודמים. המקבל שולח אישור, אך ה-ACK מסמן את הסגמנט האחרון ברצף שהתקבל. אם השולח סופר בסך הכל 3

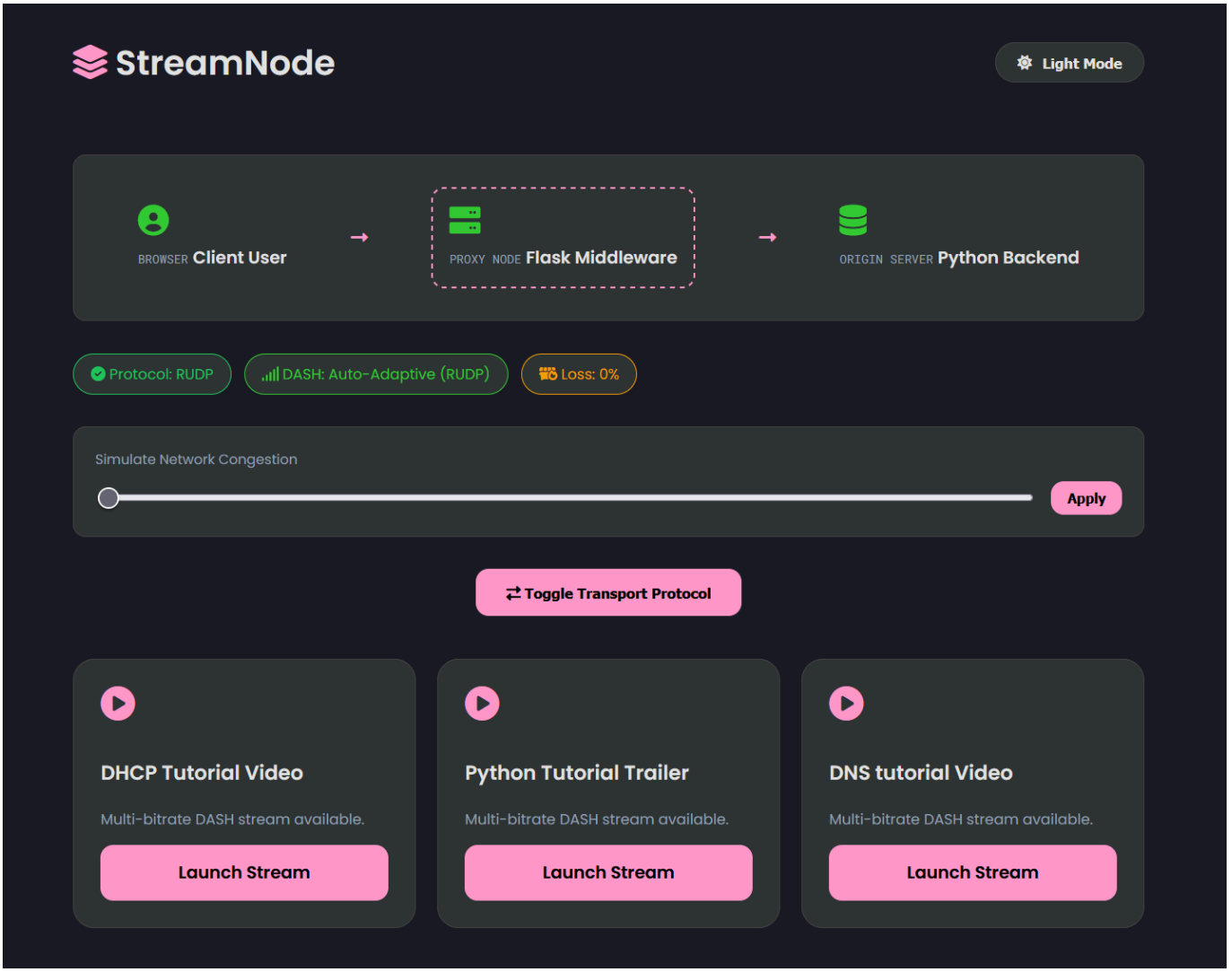
ACKים כפולים (4 אישורים זהים סך הכל), הוא מבצע שליחה מחודשת של הסגמנט החסר. זה נחשב מצב קל יותר שכן התקשורת עדיין מתפקדת, ולכן נחלק את ערך ה-ssthresh ב-2 ואת ערך ה-cwnd נחלק ב-2 (ולא נחזור להתחלה כמו במקרה הקשה של Timeout).

נעבור לדבר על הproxy: הproxy שלנו הוא הלקוח של שרת הווידאו - והמקשר בין המשתמש בדפדפן לבין שרת הסרטונים. בעת פתיחת הproxy בקובץ proxy_app.py נקבל כתובת IP בעזרת הAPI שיצרנו בקובץ dhcp_helper.py בפונקציה perform_dora. לאחר מכן נפנה לDNS לקבל את הכתובת הdomain ששייכת לproxy על פי הקיים הclient_id שרשום מבעוד מועד בתוך הzone-ים. התהליך הזה מבצע את כל ההליך בצורה אוטומטית לחלוטין ומאפשר למריץ הפרויקט לגשת ישירות לurl שמתקבל מהפלט.

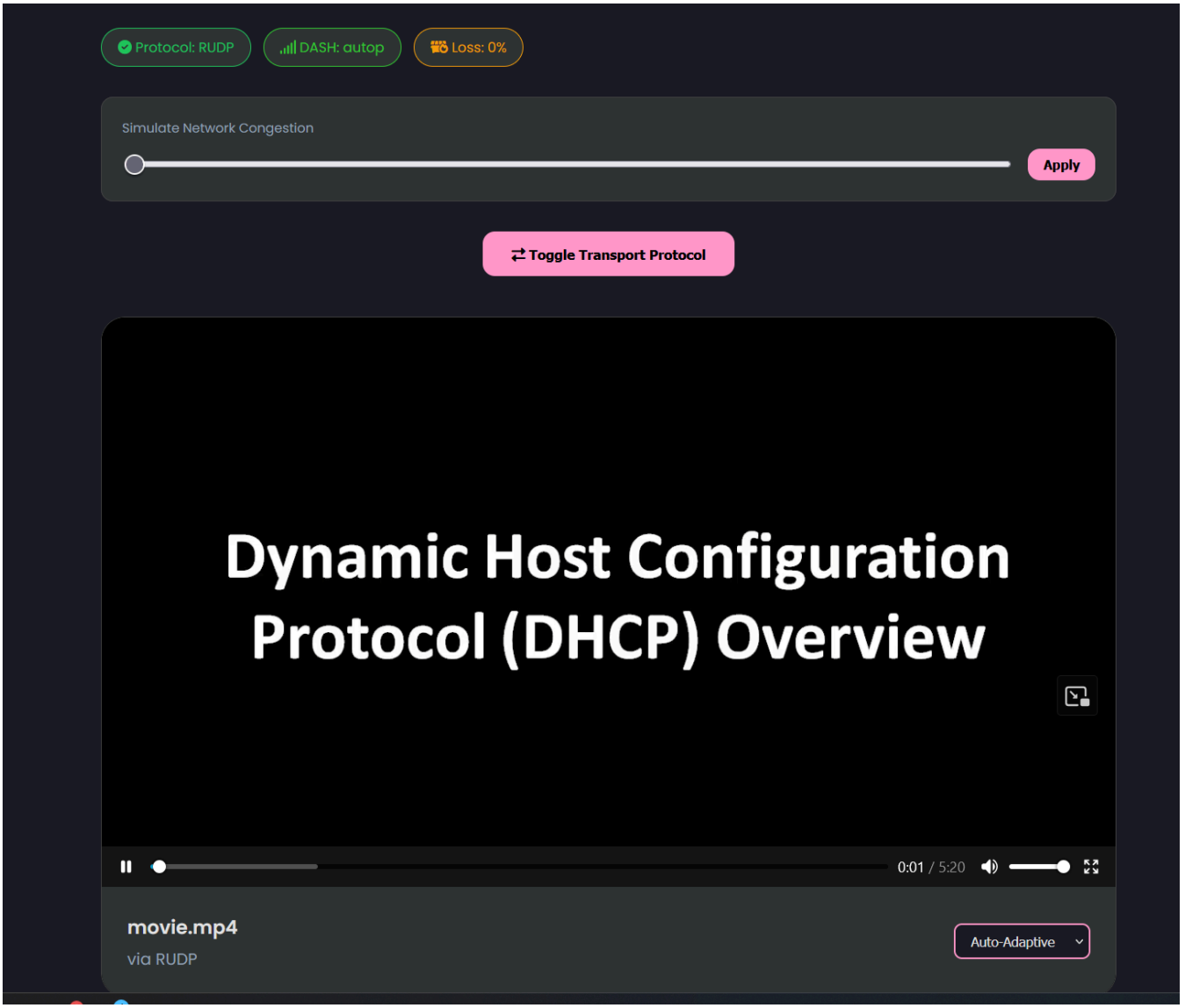
```
if __name__ == "__main__":
    if VirtualNetworkInterface:
        try:
            v_net = VirtualNetworkInterface(client_name="ProxyNode")
            MY_IP = v_net.setup_network()

        except:
            MY_IP = "127.0.0.1"

    # Resolve origin server via DNS
    resolve_origin_server()

    print(f"""
╔═══════════════════════════════════════════════════════════════════════════╗
║═══════════════════════════════════════════════════════════════════════════╕
│ UNIFIED EDGE PROXY - WITH DNS & PACKET LOSS LOGGING        │
╘═══════════════════════════════════════════════════════════════════════════╛
╠═══════════════════════════════════════════════════════════════════════════╣
│ Listening: {MY_IP}:5000
│ Origin Server: {WEB_SERVER_ADDR[0]}:{WEB_SERVER_ADDR[1]} (DNS resolved)
│ Protocol: {PROTOCOL.upper()}
│ Packet Loss: {PACKET_LOSS_PCT}%
│
│ Features:
│  ✓ DNS resolution for origin server
│  ✓ Detailed packet loss visibility
│  ✓ Real-time DASH quality selection
│  ✓ HTTP 200 for RUDP
│  ✓ HTTP 206 for TCP
│ Watch the logs to see packet loss events!
╚═══════════════════════════════════════════════════════════════════════════╝
""")

    _noise_filter = _SuppressKnownNoise()
    logging.getLogger("uvicorn.error").addFilter(_noise_filter)
    logging.getLogger("asyncio").addFilter(_noise_filter)

    uvicorn.run(
        app,
        host=MY_IP,
        port=5000,
```

```
        log_level="warning",
        workers=1
    )
```

באתר נציג את התהליך הכללי (המעבר דרך הפרוקסי)
נציג את הפרוטוקול הנבחר ואת אחוז הpacket loss (שנועד לדמות איבוד חבילות בשימוש אמיתי)




כפי שהתבקשנו במטלה אפשרנו בעזרת כפתור להחליף בין פרוטוקול TCP לRUDP.
כדי שנוכל להדגים מספר מצבים החלטנו שרירותית שהאיכות של הסרטון בRUDP יהיה אוטומטי עם אפשרות לבחירת איכות. לעומת זאת בTCP אין אפשרות להחליף איכות או לבחור, בהתאם לתנאי החיבור (מכיוון שמדובר באפליקציה על המחשב שלנו החיבור תמיד יהיה איכותי). הסרטון יוצג לאחר בחירת סרטון מהמאגר (יש לנו 3 סרטונים לבחור מהם). בכל רגע נתון נוכל להחליף בין איכויות הסרטון כאשר משתמשים בRUDP בדומה ליוטיוב.

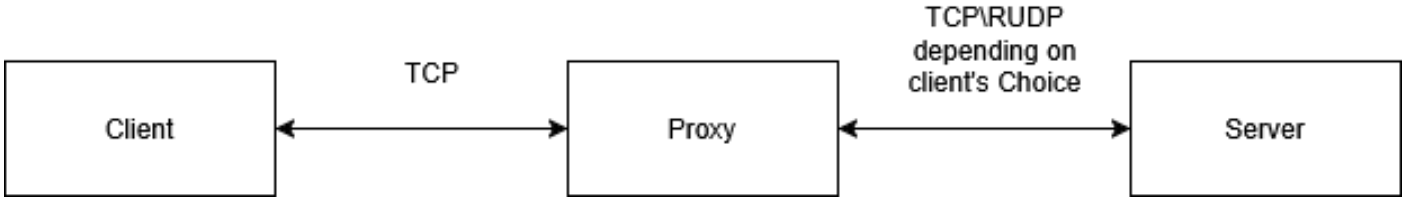



מאחורי הקלעים מוחזקים עותקים של הסרטון באיכויות השונות. בעת מעבר איכות נעבור לקובץ המתאים.

השרת: השרת שמחזיק את הסרטונים קיבל כתובת קבועה 127.0.0.13
תפקידו של השרת להחזיק את הסרטונים שה-proxy מציג ללקוח.
כדי לאפשר עבודה של ה-proxy אל מול השרת ב-RUDP ו-TCP השרת רץ בו זמנית גם על UDP וגם על TCP באותו הפורט. הדבר אפשרי מכיוון שמרחב הפורטים של הפרוטוקולים האלו נפרד.

```
def start(self):
        threading.Thread(target=self._run_tcp, daemon=True).start()
        self._run_rudp()

if __name__ == "__main__":
    server = UnifiedServer()
    server.start()
```

ארכיטקטורת החיבור בין הלקוח הproxy והשרת:




השרת הוא זה שמחזיק את הסרטונים - לפי בקשת הלקוח הProxy פונה לשרת שמקשיב בו זמנית על אותו פורט לבקשות TCP וגם UDP, על פי החלטת הלקוח נפנה באחד הפרוטוקולים לבקש את קובץ הסרטון הנבחר.
הקשר בין הלקוח לproxy הוא בעזרת פרוטוקול http בדפדפן - אשר יושב על פרוטוקול TCP.
תחילה נדבר על השרת:
כאשר מפעילים את השרת, נקרא את הconfig לקבלת הפורט (9000). את כתובת ה-ip נבקש משרת הDHCP דרך הhelper -> לשרת שלנו כתובת שמורה בDHCP והיא 127.0.0.13. במידה ולא הצלחנו לקבל כתובת נשתמש ב127.0.0.1.
נאפשר עד 100 חיבורים לכל היותר.

# פרוטוקול DASH:

פרוטוקול DASH (או בשמו המלא MPEG-DASH: Dynamic Adaptive Streaming over HTTP), הוא פרוטוקול להזרמת מדיה אדפטיבית שיושב על גבי פרוטוקול HTTP.
כדי לאפשר להשתמש בפורטוקול זה השרת צריך להכין את אותו וידאו במספר איכויות שונות. כל איכות תהיה בקובץ **שונה**. כל אחד מהקבצים ייחתך למקטעים (chunks), כדי שאפליקציית הלקוח תוכל להחליף למקטע מקביל בקובץ אחר בעת שינוי האיכות.
השרת יחזיק קובץ xml בשם (MPD Media Presentation Description) שתפקידו להחזיק פרטים על אפשרויות האיכות השונות, וכתובת להורדת כל אחד מהמקטעים.
את ההחלטה איזה איכות לבחור תבצע אפליקציית הלקוח. במידה ונבחרה האפשרות לאיכות אוטומטית התהליך מתבצע בדרך הבאה:

תחילה כדי לטעון את הסרטון במהירות תיבחר איכות נמוכה-בינונית.
במהלך הורדת מקטע נוכל לחשב את מהירות רוחב הפס על פי כמות הזמן שלקחה ההורדה ביחס לגודל. במידה ונראה שיש רוחב פס פנוי - נוכל לבקש את המקטע הבא באיכות גבוהה יותר (וכמובן גם להוריד איכות במידת הצורך).

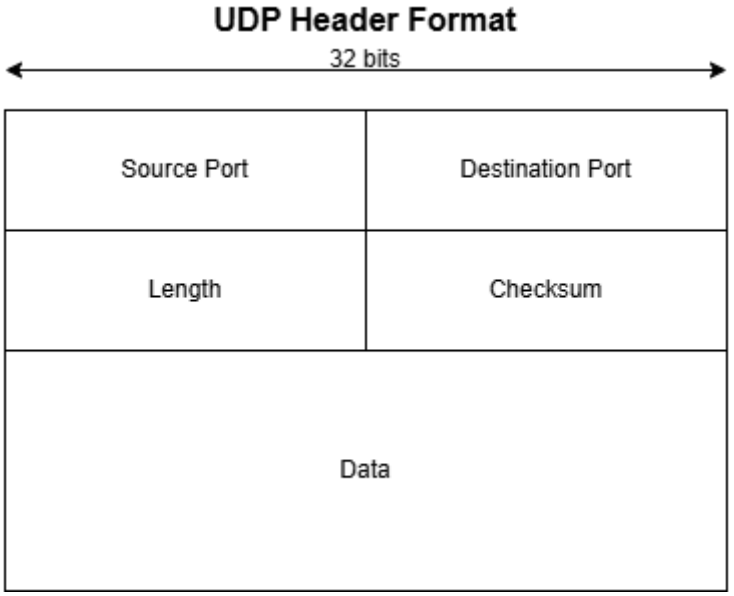
מדוע בחרו להשתמש בפרוטוקול HTTP במקום לבנות פרוטוקול ייעודי?
הסיבה העיקרית היא תאימות - הראוטר השולט על הרשת יכול לחסום תעבורה בפרוטוקול לא מוכר, למשל בארגונים, נוטים לחסום שירותים לא מוכרים לטובת אבטחת מידע.
אך בHTTP התעבורה תעבור - מדובר בפרוטוקול בסיסי שנמצא בשימוש נרחב בכל רחבי האינטרנט, ויש לו פורטים ייעודים 80 או 443. פורטים אלה לא ייחסמו, מכיוון שיש צורך בשימוש בפרוטוקול לכמעט כל שימוש בדפדפן האינטרנט.
בכך אין צורך באימוץ של הפרוטוקול, נוכל להשתמש במה שכבר עובד.
הדבר רלוונטי גם לשמירה בcache-ים של ספקי אינטרנט כך שנוכל לאפשר הורדת עומס מהשרת.

המימוש שלנו:

כדי לאפשר מימוש יותר פשוט החלטנו לבצע בדיקה יחידה למהירות בפתיחת הסרטון - הדבר רלוונטי לפרוטוקול RUDP בלבד - שכן על פי שפורט מעלה TCP יישמור על קצב קבוע.
מכיוון שהתקשורת מתבצעת בתוך המחשב (אפילו לא בתוך הרשת), אנחנו לא צופים לאיבוד של פקטות , ועל כן יש באתר אפשרות לקבוע ידנית את אחוז איבוד החבילות - לדימוי מצב אמיתי אפשרי בעבודה ברשת האינטרנט.

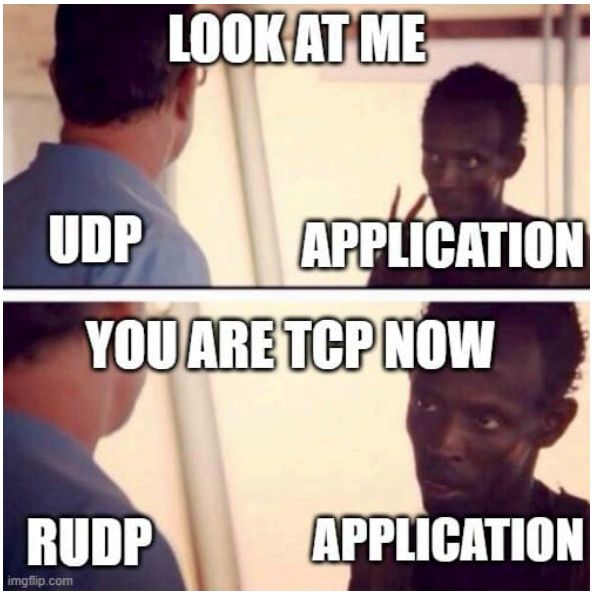
# RUDP

לפני שנדבר על RUDP נדבר תחילה על UDP. נעבור תחילה על מבנה ה-Header:




1) Source Port - פורט מקור שממנו המכשיר שולח את המידע
2) Destination Port - פורט יעד אליו המכשיר שולח את המידע
3) Length - גודל הDatagram (כולל הheader בגודל קבוע של 8 בתים והData)
4) Checksum - שדה חישובי שמבצע חישוב על הHeader, הData וה-Pseudo Header (שכולל את כתובות ה-IP משכבת הרשת). תפקידו לוודא שהמידע לא השתבש (אם המקבל מחשב את הchecksum ורואה ערך שונה ממה שכתוב בשדה - סימן שהיה שיבוש במידע). נשים לב שבשונה מTCP שדה זה הוא שדה רשות. במידה ונחליט לא להשתמש בו נמלא אותו באפסים.
5) Data - שדה בגודל משתנה שכולל את המידע שאנחנו מעבירים בdatagram.

לעומת TCP שדיברנו עליו UDP אינו פרוטוקול אמין, ועל שם כך אין לו את המנגנונים שדיברנו עליהם: תחילת קשר, אישור חבילות, ניהול קצב שליחת החבילות, וסיום הקשר. הפרוטקול פשוט שולח מידית וזהו.בשם כך גם מבנה המידע הנשלח דורש פחות בתים לעומת TCP, מכיוון שיש צורך בפחות שדות. אז מה אפשר לעשות במקרים שבהם יש צורך בחלק מהמנגנונים שדיברנו עליהם אך לא כולם?



בהמשך לmeme:

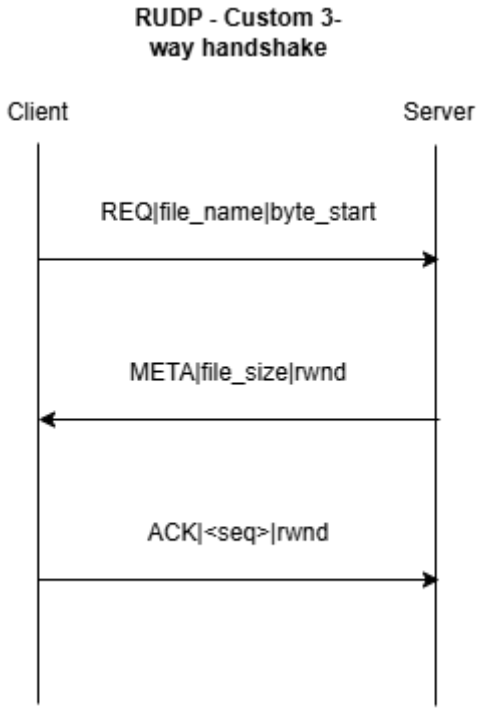
# מפרט פרוטוקול RUDP (Reliable UDP)

שימוש ב-UDP מאפשר לנו להימנע מן התהליכים המורכבים והעברת המידע המסיבית שפרוטוקול TCP כופה. ה-RUDP שפיתחנו מאפשר מהירות גבוהה משמעותית מ-TCP (אמנם איטית במעט מ-UDP "נקי") תוך מימוש מנגנוני אמינות הכרחיים.

## 1. הקמת קשר (3-Way Handshake)

בשונה מ-UDP סטנדרטי, המימוש שלנו דורש תהליך הקמה של שלושה שלבים כדי לסנכרן מטדאטה וחלונות זרימה:

- **REQ:** הלקוח שולח בקשה הכוללת את שם הקובץ והמיקום ההתחלתי (byte offset).
- **META:** השרת מגיב עם גודל הקובץ המלא וגודל חלון הקבלה שלו (`rwnd`).
- **ACK (selective ACK):** הלקוח מאשר את קבלת מטאדאטה ומפרסם את חלון הקבלה המקומי שלו. רק לאחר מכן מתחילה העברת הנתונים.




נשים לב שלמרות מספר השלבים הזהה, עדיין יש יתרון לתהליך הזה על TCP, שכן אנחנו מעבירים מידע עוד לפני סיום תהליך החיבור (file_name, file_size).

---

## 2. מבנה החבילה (Packet Format)

ה-Header של RUDP תוכנן להיות יעיל וקומפקטי. בעוד ש-Header של TCP שוקל לפחות 20 בתים, ה-Header שלנו (היושב מעל ה-UDP) מוסיף **6 בתים בלבד**:

- **Sequence Number (4 Bytes):** לסידור החבילות וזיהוי אובדן.
- **Length (2 Bytes):** אורך הפלודי (Payload).
- **Payload:** נתוני הווידאו הגולמיים (עד 65,501 בתים).

כדי שה-Proxy והשרת יפענחו נכון את המידע, הגדרנו פורמטים קבועים לכל סוג הודעה. רוב הודעות הבקרה עוברות כטקסט (String) מופרד בתו הצינור ( | ), בעוד שחבילות הנתונים עוברות בפורמט בינארי דחוס.

מבנה ה-Payload משתנה בהתאם לסוג ההודעה. נשים לב שמבנה ה-Header זהה למבנה אשר הוגדר ל-UDP ב-RFC.
השדות שהוספנו כולם מועברים בחלק של ה-DATA, והפענוח מתבצע בשכבת האפליקציה על ידי שני הצדדים.
כמפורט בטבלה:

| סוג הודעה | פורמט | דוגמה | תיאור |
|---|---|---|---|
| Req | REQ\|<file_name>\|<byte_start> | REQ\|movie_720.mp4\|0 | בקשה לתוכן מקובץ עם היסט (כל פעם נבקש חלק מהסרטון) |
| Meta | \|META\|<file_size>\|<rwnd> | \|META\|15025028\|1048576 | תגובה של השרת לבקשהת ה-REQ עם Metadata כוללת את גודל החלון הדינמי |
| DATA | <seq - 4 bytes>\|<len - 2 bytes>\|data | \|0\|0\|0\|5\|0\|5\|<data> | פקטת מידע גולמית, seq ייסמן לנו את היסט הבתים מההתחלה, ו-len את גודל שדה המידע. בשדה המידע יהיו לנו בתים שמעבירים חלק מהסרטון |
| ACK | \|ACK\|<seq>\|<rwnd> | ACK\|5\|1048576 | חבילה שמאשרת קבלת מידע (בדומה ל-ACK ב-TCP) בנוסף מעדכנת את גודל החלון |
| ALIVE | ALIVE\|<rwnd> | ALIVE\|1048576 | כדי להתגבר על זה ש-UDP הוא חסר חיבור - נשלח חבילות שמטרתם לוודא שהצד השני אכן מקשיב. בנוסף נעדכן את גודל החלון |
| FIN | FIN\|DONE\|<rwnd> | FIN\|DONE\|0 | כדי שהצד השני יסיים את החיבור נצטרך לעדכן אותו על סיום מהצד שלנו. גודל החלון יעודכן. |

מבנה ה-payload בחבילת data של הסרטון: שדה Len יעזור לנו לדעת עד איפה לקרוא מידע.

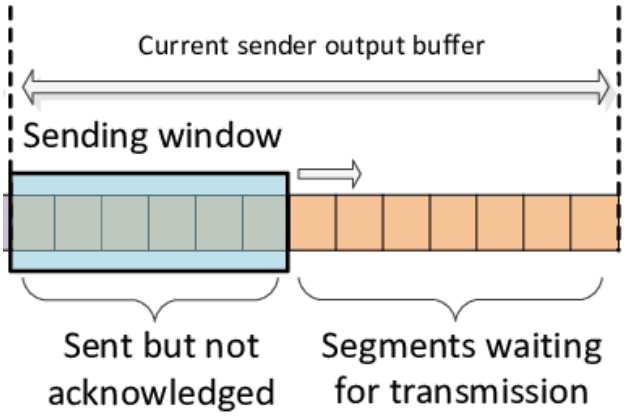
| | | |
|---|---|---|
| Seq (Sequence Number) - 4 Bytes | Len - 2 Bytes | Data (Len size) |

## 3. אמינות וחלון החלקה (Sliding Window ARQ)

כדי להבטיח אמינות, אימצנו את מנגנון ה-ACKs. במימוש שלנו, כל אישור מתייחס **לחבילה ספציפית** ולא לרצף מצטבר.

- השולח מחזיק חלון המכיל חבילות שנשלחו וטרם אושרו.
- בצד המקבל, המערכת יודעת לסדר את החבילות לפי ה-Seq ולזהות חורים ברצף.
- בשונה מ-TCP, ב-RUDP נוכל להחליט ברמת האפליקציה כיצד לטפל בחבילות שאבדו (למשל, דילוג על פריים פגום בווידאו) מבלי לעצור את כל הסטרים.

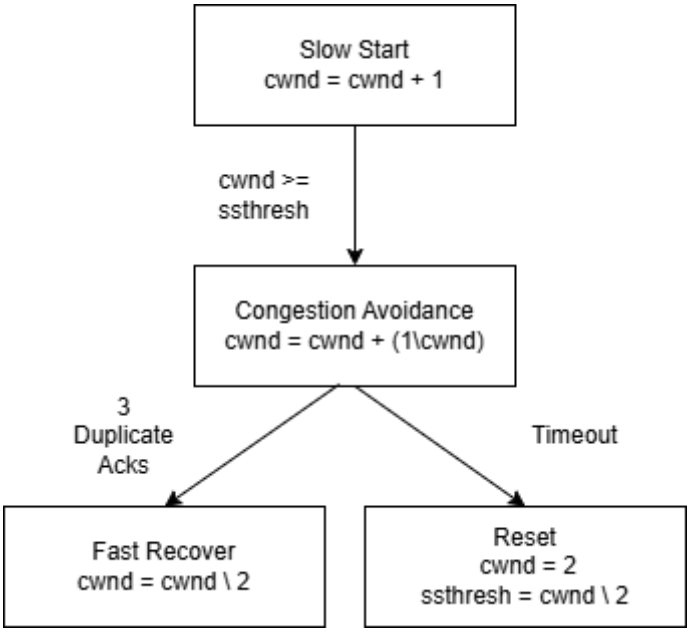
מבנה ה-sliding window.
הבסיס של החלון יזוז בהתאם ל-ACKים שנקבל על תחילת הבסיס של החלון. במידה ויהיה דילוג על חבילות לא נוכל להזיז את הבסיס של החלון ונשלח אותן מחדש.




## 4. בקרת עומס (Congestion Control - TCP Reno)

בחרנו לממש את מנגנון **TCP Reno** כדי לוודא שאיננו מציפים את הרשת:

- **Slow Start:** צמיחה אקספוננציאלית של חלון השליחה (cwnd) עד להגעה לסף ה-`ssthresh`.
- **Congestion Avoidance:** צמיחה ליניארית למניעת עומס.
- **Fast Recovery:** במקרה של קבלת 3 אישורים כפולים (Duplicate ACKs), המערכת מבצעת התאוששות מהירה במקום "להתרסק" למצב איטי.




---

### 5. בקרת זרימה (Flow Control)

כדי לא להציף את חוצץ הקליטה (Buffer) של הלקוח, אנו משתמשים ב-**Receiver Window (RWnd)**. הלקוח מעדכן בכל ACK מהו השטח הפנוי אצלו, והשרת מגביל את הקצב לפי הערך המינימלי שבין חלון העומס לחלון הקבלה

**3.5 Flow Control**

**Receiver Window (RWnd):**

- Client advertises available buffer space in every ACK
- Server limits sending to `min(cwnd, rwnd/packet_size)`
- Prevents overwhelming receiver's buffer

```python
# Combined limit calculation
max_packets = min(
    congestion.get_sending_limit(),  # cwnd (network limit)
    flow.get_sending_limit()          # rwnd (receiver limit)
)
```

**3.6 Sequence Number Wraparound**

32-bit sequence numbers wrap at $2^{32}$ (4,294,967,296):

```python
def seq_less_than(a: int, b: int) -> bool:
    """RFC 1323 circular sequence space comparison."""
    return ((a - b) & 0x80000000) != 0

Examples:
  seq_less_than(100, 200) → True
  seq_less_than(0xFFFFFFFE, 5) → True  # Wraparound!
  seq_less_than(200, 100) → False
```

### 6. תחזוקת הקשר וסיום

- **Keep-Alive:** מכיוון ש-UDP הוא Connection-less, מימשנו מנגנון "דופק" (Heartbeat) – הודעת `ALIVE` נשלחת בכל כמה שניות כדי לוודא שהצד השני עדיין מחובר.
- **סיום (FIN):** בסיום ההעברה נשלחת הודעת `FIN` שלוש פעמים (כדי להתגבר על אובדן חבילות). ה-Proxy מאזין להודעה זו ומסיים את לולאת תחזוקת הקשר.

---

### 7. Wraparound של מספרי ה-Sequence

בשל השימוש ב-32 סיביות למספרי רצף, המערכת יודעת לטפל במצב שבו המספר מגיע למקסימום וחוזר ל-0, תוך שימוש בלוגיקה של **RFC 1323** להשוואת מספרים במרחב מעגלי.

---

```
class UnifiedServer:
    def __init__(self, config_file = os.path.join(PARENT_DIR, "shared", "configs",
"movies.yaml")):

        self.active = True
        signal.signal(signal.SIGINT, self._on_sigint)

        try:
            with open(config_file, "r") as f:
                    config = yaml.safe_load(f)
                    self.port = config.get("server_config", {}).get("origin_port", 9000)
            current_script_dir = os.path.dirname(os.path.abspath(__file__))

            # Build path to shared/videos relative to the script location
            default_video_dir = os.path.abspath(os.path.join(current_script_dir, "..",
"shared", "videos"))

            # Use the config value if it exists, otherwise use the absolute path we just
built
            video_dir = config.get("server_config", {}).get("base_directory",
default_video_dir)
        except Exception as e:
            print(f"[!] Failed to load config, using defaults: {e}")
            self.port, video_dir = 9000, os.path.join(BASE_DIR, "shared", "videos")

        self.my_ip = VirtualNetworkInterface(client_name="OriginServer",
fixed_id="OriginServer").setup_network() if VirtualNetworkInterface else "127.0.0.1"
        self.data_loss_rate = float(os.environ.get("DATA_LOSS_RATE", DATA_LOSS_RATE))
        self.ack_loss_rate = float(os.environ.get("ACK_LOSS_RATE", ACK_LOSS_RATE))
        self.latency_ms = int(os.environ.get("LATENCY_MS", LATENCY_MS))



        try:
            self.file_repo = LocalFileRepository(default_video_dir)
        except Exception as e:
            print(f"[!] Failed to initialize file repository: {e}")
            self.file_repo = None

        self.session_manager = SimpleSessionManager(max_sessions=100)

        def cleanup_loop():
            while self.active:
                time.sleep(30)
                self.session_manager.cleanup_idle_sessions(timeout_seconds=60.0)
        threading.Thread(target=cleanup_loop, daemon=True).start()

        print(f"╟ UNIFIED ORIGIN SERVER LISTENING ON {self.my_ip}:{self.port} ╢")
```

בהתחלת השרת נפעיל שני threads להאזנה. אחד בפרוטוקול TCP והשני ב-RUDP

```
    def start(self):
        threading.Thread(target=self._run_tcp, daemon=True).start()
        self._run_rudp()
```

פונקציה _run_tcp:
פותחת socket TCP ומאזינה לבקשות

```
def _run_tcp(self):
        srv = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        srv.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:
            srv.bind((self.my_ip, self.port))
            srv.listen(20)
            handler = TCPHandler(self)
            while self.active:
                try:
                    conn, addr = srv.accept()
                    threading.Thread(target=handler.handle, args=(conn, addr),
daemon=True).start()
                except socket.timeout: continue
        except OSError: pass
        finally: srv.close()
```

את הבקשות נפענח בעזרת הפונקצייה handle:
את תוכן הסרטון המועבר נפצל לחלקים בגדלים זהים (לפי chunk_size) הדבר יאפשר העברת הסרטון בחלקים והזרמה מיידית.

```
def handle(self, conn: socket.socket, addr: Tuple[str, int]):
        try:
            conn.settimeout(10.0)
            req_data = conn.recv(4096).decode(errors="ignore")
            parts = req_data.split("|")
            if len(parts) < 3 or parts[0] != "REQ":
                return conn.sendall(b"ERR|BAD_REQUEST\n")

            filename = os.path.basename(parts[1])
            byte_start = max(0, int(parts[2]))

            try:
                filepath = self.server.file_repo.get_file_path(filename, quality="auto")
                file_size = self.server.file_repo.get_file_size(filename,
quality="auto")
            except FileNotFoundError:
                return conn.sendall(b"ERR|NOT_FOUND\n")

            conn.sendall(f"META|{file_size}\n".encode())
            with open(filepath, "rb") as fh:
                fh.seek(byte_start)
                while True:
                    chunk = fh.read(131_072)
                    if not chunk: break
                    conn.sendall(chunk)
                    if self.server.data_loss_rate > 0 and random.random() <
self.server.data_loss_rate:
                        conn.sendall(b"DROP|packet_dropped\n")  # Signal drop
                        print(f"[SIMULATED DROP] TCP packet dropped for {addr}")
                        continue
        except Exception:
            pass
        finally:
            conn.close()
```

בדומה קיימות פוקנציות מקבילות לRUDP.
פונקציה _run_udp:

```
    def _run_rudp(self):
        sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
        sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
        try:
            sock.bind((self.my_ip, self.port))
            sock.settimeout(0.01)
            while self.active:
```

```
                try:
                    data, addr = sock.recvfrom(4096)
                    msg = data.decode(errors="ignore")
                    parts = msg.split("|")

                    if parts and parts[0] == "LOSS_RATE":
                        self.data_loss_rate = float(parts[1])
                        continue
                    if not parts or parts[0] != "REQ":
                        continue

                    filename = os.path.basename(parts[1])
                    byte_start = int(parts[2])
                    request = StreamRequest(filename=filename, byte_start=byte_start,
protocol=TransportProtocol.RUDP)
                    filepath = self.file_repo.get_file_path(filename, quality="auto")

                    # 🔥 RESTORED: Create the session WITHOUT passing the dispatcher
sock!
                    session_id = f"rudp_{addr[0]}_{addr[1]}_{int(time.time()*1000)}"
                    rudp_session = RUDPSession(
                        filepath=filepath,
                        client_addr=addr,
                        data_loss_rate=self.data_loss_rate,
                        ack_loss_rate=self.ack_loss_rate,
                        latency_ms=self.latency_ms,
                    )
                    self.session_manager.create_session(session_id,
TransportProtocol.RUDP, addr, rudp_session)

                    def run_and_cleanup():
                        try: rudp_session.handle_request(request, addr)
                        finally: self.session_manager.close_session(session_id)

                    threading.Thread(target=run_and_cleanup, daemon=True).start()
                except (socket.timeout, FileNotFoundError, ValueError):
                    continue
        finally:
            sock.close()
```

פונקציית run בRUDPSession:
תכין את החבילה לשליחה. בדומה למצב TCP גם כאן תתבצע חלוקה ל-Chunk-ים בגודל קבוע על מנת לאפשר הזרמה מיידית. בזכות החלוקה נוכל גם להחליף איכות כאשר החלפת האיכות בעצם "תחליף" את הורדת ה-chunck-ים הבאים לקובץ אחר באיכות שונה.

```
    def run(self):
        self._metrics.connection_state = StreamState.STREAMING
        try:
```

```
            meta_pkt = encode_control_message("META", self.file_size, RECV_WINDOW_SIZE)
            for _ in range(5):
                self._send(meta_pkt)
            time.sleep(0.03)

            with open(self.filepath, "rb") as fh:
                fh.seek(self.byte_start)

                while self.active:
                    if time.monotonic() - self.last_ack_time > KEEPALIVE_SECS:
                        self._send(encode_control_message("ALIVE", RECV_WINDOW_SIZE))
                        self.last_ack_time = time.monotonic()

                    if time.monotonic() - self.last_ack_time > SESSION_IDLE_TIMEOUT:
                        break

                    max_in_flight = calculate_combined_limit(
                        self.congestion.get_sending_limit(),
                        self.flow.get_sending_limit()
                    )

                    while self.active and not self.eof and
self.window.has_capacity(max_in_flight):
                        chunk = fh.read(CHUNK_SIZE)
                        if not chunk:
                            self.eof = True
                            break
                        try:
                            pkt = encode_data_packet(self.window.next_seq, chunk)
                            seq = self.window.add_packet(pkt)
                            if self._send(pkt, is_ack=False):
                                self.metrics.bytes_sent += len(chunk)
                        except ValueError:
                            self.active = False
                            break

                    self._drain_acks()
                    removed = self.window.slide_window()
                    self.metrics.window_max_size = max(self.metrics.window_max_size,
self.window.size)

                    timed_out = self.window.get_timed_out_packets(timeout_seconds=RTO)
                    if timed_out:
                        seq, packet = timed_out[0]
                        self.congestion.on_timeout()
                        self._send(packet, is_ack=False)
                        self.window.mark_retransmitted(seq)
                        self.metrics.packets_retransmitted += 1

                    if self.eof and self.window.is_empty():
                        break
```

```
                time.sleep(0.0001)
        except Exception as e:
            self._metrics.connection_state = StreamState.ERROR
            self._metrics.error_message = str(e)
            self.metrics.errors += 1
        finally:
            self._metrics.connection_state = StreamState.CLOSED
            for _ in range(5):
                self._send(encode_control_message("FIN", "DONE", 0))
                time.sleep(0.01)
            self._print_summary()
```

כעת נדבר על המנגנונים שמימשנו ב:RUDP
Congestion Control בוקבץ congestion_controller.py
חילקנו את המצבים לשלושה בעזרת :ENUM
Slow_Start - התחלה איטית - איננו יודעים עדיין מה הגבולות.
Congestion_Avoidance - במקרה שבו קיבלנו ACKים כפולים המצב פחות "גרוע" ועל כן לא נחזור להתחלה איטית.

```
class CongestionState(Enum):
    """Congestion control states (TCP Reno)."""
    SLOW_START = "slow_start"
    CONGESTION_AVOIDANCE = "congestion_avoidance"
```

בעת אתחול הinstance נגדיר את הערכים הראשוניים.

```
    def __init__(
        self,
        initial_cwnd: float = 2.0,
        initial_ssthresh: float = 64.0,
        min_cwnd: float = 1.0,
        max_cwnd: float = 1000.0,
    ):
        """
        Initialize congestion controller.

        Args:
            initial_cwnd: Initial congestion window (packets)
            initial_ssthresh: Initial slow-start threshold (packets)
            min_cwnd: Minimum cwnd (packets)
            max_cwnd: Maximum cwnd (packets)

        Example:
            >>> controller = CongestionController(
            ...     initial_cwnd=4.0,
            ...     initial_ssthresh=32.0
            ... )
```

```
        """
        self._initial_cwnd = float(initial_cwnd)
        self._initial_ssthresh = float(initial_ssthresh)
        self._min_cwnd = float(min_cwnd)
        self._max_cwnd = float(max_cwnd)

        # Current state
        self._cwnd = float(initial_cwnd)
        self._ssthresh = float(initial_ssthresh)
        self._state = CongestionState.SLOW_START

        # Statistics
        self._packets_since_increase = 0
        self._total_increases = 0
        self._total_decreases = 0
        self._min_cwnd_seen = self._cwnd
        self._max_cwnd_seen = self._cwnd

    # ── Properties
```

פונקציה update_state:
בהתאם למצב הנוכחי נשנה את המצב
במידה ולא הגענו לערך ה"בטוח" נהיה במצב SLOW_START
אחרת -> נעבור למצב CONGESTION_AVOIDANCE.

```
    def _update_state(self) -> None:
        """Update state based on cwnd and ssthresh."""
        if self._cwnd < self._ssthresh:
            self._state = CongestionState.SLOW_START
        else:
            self._state = CongestionState.CONGESTION_AVOIDANCE
```

פונקציה on_ack_recieved:
בעת קבלת ack נרצה להגדיל את גודל החלון
בהתאם למצב הנוכחי נבחר את קצב הגדילה - אם אנחנו בSLOW_START על כל ack שנקבל נגדיל את החלון ב-1.
אם אנחנו במצב CONGESTION_AVOIDANCE - נגדיל את גודל החלון ב-1 על אישור **כל** החבילות בחלון.
במידה ועברנו את המקסימום המוגדר -> נעדכן למקסימום בהתאם.
ונקרא לפונקציה update_state.

```
def on_ack_received(self) -> None:
        """
        Process ACK - increase cwnd according to current state.

        Slow Start: cwnd += 1 (doubles per RTT)
        Congestion Avoidance: cwnd += 1/cwnd (increases by 1 per RTT)

        Example:
```

```
        >>> controller.on_ack_received()
        >>> print(f"New cwnd: {controller.cwnd}")
    """
    self._packets_since_increase += 1

    if self._state == CongestionState.SLOW_START:
        # Exponential growth: cwnd += 1 per ACK
        self._cwnd += 1.0
        self._total_increases += 1

    elif self._state == CongestionState.CONGESTION_AVOIDANCE:
        # Linear growth: cwnd += 1/cwnd per ACK (1 per RTT)
        self._cwnd += 1.0 / self._cwnd
        self._total_increases += 1

    # Enforce maximum
    self._cwnd = min(self._cwnd, self._max_cwnd)

    # Update statistics
    self._max_cwnd_seen = max(self._max_cwnd_seen, self._cwnd)

    # Update state
    self._update_state()
```

פונקציה on_duplicate_ack:
בעת קבלה של 3 ackים כפולים - נחלק ב-2 את גדול החלון ה"בטוח" ונשנה את גודל החלון בהתאם. נעבור למצב CONGESTION_AVOIDANCE

```
def on_duplicate_ack(self) -> None:
    """
    Process duplicate ACK (fast retransmit trigger).

    TCP Reno behavior:
    - ssthresh = max(cwnd / 2, 2)
    - cwnd = ssthresh
    - Enter congestion avoidance (skip slow start)

    Example:
        >>> # After 3 duplicate ACKs
        >>> controller.on_duplicate_ack()
        >>> print(f"Reduced cwnd to {controller.cwnd}")
    """
    # Halve ssthresh (but not below 2)
    self._ssthresh = max(self._cwnd / 2.0, 2.0)

    # Set cwnd to ssthresh (multiplicative decrease)
    self._cwnd = self._ssthresh

    # Enter congestion avoidance (skip slow start)
```

```
        self._state = CongestionState.CONGESTION_AVOIDANCE

        # Update statistics
        self._total_decreases += 1
        self._min_cwnd_seen = min(self._min_cwnd_seen, self._cwnd)
```

פונקציה on_timeout:
בדומה לפונקציה on_duplicate_ack החלון ייחתך בגודלו בחצי, אך נעבור למצב Slow start מכיוון שמדובר במצב יותר "חמור"

```
def on_timeout(self) -> None:
        """
        Process timeout (severe congestion signal).

        TCP Reno behavior:
        - ssthresh = max(cwnd / 2, 2)
        - cwnd = initial_cwnd (restart slow start)
        - Enter slow start

        More severe than duplicate ACKs because timeout indicates
        complete loss of ACKs (not just one dropped packet).

        Example:
            >>> # Packet timed out
            >>> controller.on_timeout()
            >>> print(f"Reset cwnd to {controller.cwnd}")
        """
        # Halve ssthresh
        self._ssthresh = max(self._cwnd / 2.0, 2.0)

        # Reset cwnd to initial (severe penalty)
        self._cwnd = self._initial_cwnd

        # Restart slow start
        self._state = CongestionState.SLOW_START

        # Update statistics
        self._total_decreases += 1
        self._min_cwnd_seen = min(self._min_cwnd_seen, self._cwnd)
```

פונקציה reset: נועדה להחזיר אותנו למצב המקורי.

```
def reset(self) -> None:
        """
        Reset to initial state.

        Example:
            >>> controller.reset()
```

```
        """
        self._cwnd = self._initial_cwnd
        self._ssthresh = self._initial_ssthresh
        self._state = CongestionState.SLOW_START
        self._packets_since_increase = 0
```

עכשיו נעבור לflow_controller.py:
המהות עוברת בפונקצייה של update_rwnd:
נוודא ראשית שאנחנו עומדים בגודל מינימום
ונעדכן את הגודל של החלון בהתאם לערך שנקבל.
הפונקצייה תיקרא בעת קבלת ACK - שבחבילה יכלל גם ערך גודל החלון.

```
def update_rwnd(self, new_rwnd_bytes: int, timestamp: float = 0.0) -> bool:
        """
        Update receiver window from ACK.

        Args:
            new_rwnd_bytes: New receiver window size (bytes)
            timestamp: Timestamp of update (optional)

        Returns:
            True if window changed (False if same as before)

        Example:
            >>> # Received ACK with RWnd=524288 (512KB)
            >>> changed = controller.update_rwnd(524288)
            >>> if changed:
            ...     print(f"Window updated: {controller.rwnd_bytes} bytes")
        """
        import time

        # Enforce minimum
        new_rwnd_bytes = max(self._min_rwnd, new_rwnd_bytes)

        # Check if changed
        if new_rwnd_bytes == self._rwnd_bytes:
            return False

        # Detect zero window
        if new_rwnd_bytes == 0 and self._rwnd_bytes > 0:
            self._zero_window_events += 1

        # Update
        old_rwnd = self._rwnd_bytes
        self._rwnd_bytes = new_rwnd_bytes
        self._last_update_time = timestamp or time.monotonic()
        self._rwnd_updates += 1

        # Update statistics
        self._min_rwnd_seen = min(self._min_rwnd_seen, new_rwnd_bytes)
```

```
        self._max_rwnd_seen = max(self._max_rwnd_seen, new_rwnd_bytes)

        return True
```

קובץ protocol_utils.py:
אחראי על פענוח ויצירת פקטות להעברת מידע
פונקציה encode_data_packet: מקבלת מספר seq ואת הpayload תפקידה ליצור פקטה בהתאם. תוודה ראשית שכל המידע תקין. במידה וכן תראגן את המידע בפקטה, עם שדה seq שדה אורך ואת הpayload.

```
def encode_data_packet(seq: int, payload: bytes) -> bytes:
    """
    Encode RUDP data packet with format: <4-byte seq><2-byte length><payload>

    This format allows the receiver to know the exact payload length even if
    the UDP packet was padded, which is critical for reliable reassembly.

    Args:
        seq: Sequence number (32-bit unsigned, will wrap at 2^32)
        payload: Data chunk to encode

    Returns:
        Complete encoded packet ready for transmission

    Raises:
        ValueError: If payload is empty or exceeds size limits

    Example:
        >>> packet = encode_data_packet(42, b"Hello World")
        >>> len(packet)
        17  # 4 (seq) + 2 (len) + 11 (payload)
    """
    if len(payload) == 0:
        raise ValueError("Payload cannot be empty")

    if len(payload) > MAX_PAYLOAD_SIZE:
        raise ValueError(
            f"Payload too large: {len(payload)} bytes "
            f"(max {MAX_PAYLOAD_SIZE} bytes)"
        )

    # Ensure seq is 32-bit
    seq = seq & 0xFFFFFFFF

    # Format: seq(4 bytes) + length(2 bytes) + payload
    packet = (
        seq.to_bytes(4, "big") +
        len(payload).to_bytes(2, "big") +
        payload
    )
```

```
    # Validate total size
    if len(packet) > MAX_UDP_PAYLOAD:
        raise ValueError(
            f"Encoded packet exceeds UDP limit: {len(packet)} > {MAX_UDP_PAYLOAD}"
        )

    return packet
```

פונקצייה decode_data_packet:
עושה בדיוק הפוך. מקבלת פקטה מוכנה ומחלצת ממנה את השדות בהתאם.
בדומה תוודא שאכן מדובר באורך תקין.

```
def decode_data_packet(packet: bytes) -> Tuple[int, bytes]:
    """
    Decode RUDP data packet.

    Args:
        packet: Raw packet bytes received from network

    Returns:
        Tuple of (sequence_number, payload)

    Raises:
        ValueError: If packet is malformed or has mismatched lengths

    Example:
        >>> seq, data = decode_data_packet(packet)
        >>> print(f"Seq {seq}: {len(data)} bytes")
        Seq 42: 11 bytes
    """
    if len(packet) < PACKET_OVERHEAD:
        raise ValueError(
            f"Packet too short: {len(packet)} bytes "
            f"(minimum {PACKET_OVERHEAD} bytes required)"
        )

    # Extract sequence number (4 bytes)
    seq = int.from_bytes(packet[:4], "big")

    # Extract declared payload length (2 bytes)
    declared_len = int.from_bytes(packet[4:6], "big")

    # Extract payload
    payload = packet[6:6 + declared_len]

    # Validate payload length matches declaration
    if len(payload) != declared_len:
        raise ValueError(
            f"Payload length mismatch: declared {declared_len}, "
            f"got {len(payload)} bytes"
```

```
        )

    return seq, payload
```

פונקציה decode_control_message: תפקידה לפענח את הדגלים הרלוונטים של הפקטה וערכים נוספים.

```
def decode_control_message(data: bytes) -> Tuple[str, list]:
    """
    Decode a control message.

    Args:
        data: Raw message bytes

    Returns:
        Tuple of (msg_type, [arg1, arg2, ...])

    Raises:
        ValueError: If message cannot be decoded

    Example:
        >>> msg_type, args = decode_control_message(b'ACK|42|1024000')
        >>> msg_type
        'ACK'
        >>> args
        ['42', '1024000']
    """
    try:
        decoded = data.decode(errors="ignore").strip()
        parts = decoded.split("|")
        if not parts:
            raise ValueError("Empty message")
        return parts[0], parts[1:]
    except Exception as e:
        raise ValueError(f"Failed to decode control message: {e}")
```

פונקציה encode_control_message:
תפקידה להכניס לפקטה את הדגלים והערכים הרלוונטים

```
def encode_control_message(msg_type: str, *args) -> bytes:
    """
    Encode a control message (META, ACK, FIN, etc).

    Format: MSG_TYPE|arg1|arg2|...

    Args:
        msg_type: Message type string (e.g., "ACK", "META", "FIN")
        *args: Additional arguments to encode

    Returns:
        Encoded message bytes

    Example:
        >>> encode_control_message("ACK", 42, 1024000)
        b'ACK|42|1024000'
    """
    parts = [msg_type] + [str(arg) for arg in args]
    return "|".join(parts).encode()
```

קובץ seession_manager.py: אחראי לעקוב אחר הsession הקיימים ולסגור כאלה שלא משומשים (בשונה מTCP - כמו שאמרנו בUDP תהליך מסודר לניתוק, על כן נעבוד עם מנגנון keep-alive ששואל את הצד השני אם הוא עדיין "מחובר", זה נועד לפתור בעיה הקיימת במנגנון UDP).
פונקצייה create_session ליצירת session חדש

תחילה נוודא שאין session כמו המבוקש שכבר קיים.
אם אין, נוודא שלא הגענו למגבלה.
במידה ועברנו את הבדיקות נייצר בעזרת הfactory, seesion חדש.

לבסוף נשמור את פרטי הsession שיצרנו:

```
def create_session(
        self,
        session_id: str,
        protocol: TransportProtocol,
        client_addr: tuple,
        server: Optional[StreamingServer] = None
    ) -> StreamingServer:
        """
        Create new streaming session.

        Implements: SessionManager.create_session()
```

```
        Args:
            session_id: Unique session identifier
            protocol: Transport protocol (TCP or RUDP)
            client_addr: Client's address tuple
            server: Optional pre-created server instance

        Returns:
            New session handler

        Raises:
            ValueError: If session_id already exists or max sessions reached

        Example:
            >>> session = manager.create_session(
            ...     "session_123",
            ...     TransportProtocol.RUDP,
            ...     ("127.0.0.1", 50000)
            ... )
        """
        with self._lock:
            # Check if session already exists
            if session_id in self._sessions:
                raise ValueError(f"Session already exists: {session_id}")

            # Check max sessions limit
            if len(self._sessions) >= self._max_sessions:
                raise ValueError(
                    f"Max sessions reached ({self._max_sessions}). "
                    f"Close existing sessions first."
                )

            # Create or use provided server
            if server is None:
                if self._session_factory is None:
                    raise ValueError(
                        "No session_factory provided and no server instance given"
                    )
                server = self._session_factory(client_addr)

            # Create session info
            info = SessionInfo(
                session_id=session_id,
                server=server,
                protocol=protocol,
                client_addr=client_addr
            )

            # Store session
            self._sessions[session_id] = info
            self._total_created += 1
```

```
        return server
```

פונקציה get_active_sessions - כשמה מחזירה את הsesssions הפעילים

```
def get_active_sessions(self) -> Dict[str, StreamMetrics]:
        """
        Get metrics for all active sessions.

        Implements: SessionManager.get_active_sessions()

        Returns:
            Dictionary mapping session_id → metrics

        Example:
            >>> active = manager.get_active_sessions()
            >>> for sid, metrics in active.items():
            ...     print(f"{sid}: {metrics.bytes_transferred} bytes")
        """
        with self._lock:
            result = {}
            for session_id, info in self._sessions.items():
                try:
                    metrics = info.server.get_metrics()
                    result[session_id] = metrics
                except Exception as e:
                    print(f"[!] Error getting metrics for {session_id}: {e}")
            return result
```

פונקצייה cleanup_idle_sessions:
נבודק איזה sessions עברו את זמן הtimeout ונסגור את מה שצריך

```
def cleanup_idle_sessions(self, timeout_seconds: float) -> int:
        """
        Close sessions idle longer than timeout.

        Implements: SessionManager.cleanup_idle_sessions()

        Args:
            timeout_seconds: Idle timeout threshold

        Returns:
            Number of sessions closed

        Example:
            >>> closed = manager.cleanup_idle_sessions(timeout_seconds=30.0)
            >>> print(f"Cleaned up {closed} idle sessions")
        """
```

```
        with self._lock:
            to_close = []

            for session_id, info in self._sessions.items():
                if info.idle_time() > timeout_seconds:
                    to_close.append(session_id)

            for session_id in to_close:
                self.close_session(session_id)
                self._total_cleaned += 1

            return len(to_close)
```

קובץ http_handler.py:

הקוד מגדיר את המחלקה HTTPHandler, שתפקידה לנהל את שכבת הפרוטוקול של הזרמת הווידאו תוך הפרדה מוחלטת משכבת הזרמת הנתונים עצמה. המחלקה אחראית על פענוח בקשות טווח (Range Requests) של HTTP המאפשרות לצפות במיקום ספציפי בסרטון, יצירת כותרות תגובה מתאימות (כגון Content-Range ו-Content-Length), וקביעת קוד הסטטוס הרלוונטי (כמו 200 לקובץ מלא או 206 לתוכן חלקי). המחלקה אינה עוסקת במשיכת המידע מהשרת או בבחירת איכות הווידאו, אלא מתמקדת בתיווך הפרוטוקולי שבין הדפדפן למערכת ההזרמה, כולל אימות טווחים מול גודל הקובץ וטיפול בשגיאות HTTP.

# שאלות עיוניות

---

1) מנה לפחות ארבע הבדלים עיקריים בין פרוטוקול TCP ל-QUIC.
א) פרוטוקל QUIC מובנה עם הצפנה - בTCP זה לא ברירת מחדל
ב) תהליך הhandshake קצר יותר בQUIC - ניתן לבצע עם 2 בקשות אחת מהלקוח ותשובה מהשרת. בנוסף במידה והיה קשר יחסית לאחרונה - ניתן להפעיל מצב RTT-0 שישתמש בhandshake מתהליך קודם - ויחסוך את הליך הhandshake בקשר.
ג) quic מאפשר עבודה עם חלוקה לstreams באותו החיבור, בדומה לTCP - אך כאשר אחד נתקע למשל מסיבת packet loss - האחרים יכולים להמשיך ללא הפרעה בזמן שהראשון יעבוד על פתרון הבעיה. את הבעיה הזו ב-TCP שנקראת head of line blocking פרוטוקול QUIC פותר,
בשונה מTCP שבפרוטוקול זה כאשר יש תקלה כל הstreams עוצרים עד לטיפול.
ד) בעוד שTCP (וגם UDP) הינם פרוטוקלים אשר עובדים בkernel space
QUIC הוא פרוטוקול ברמת user space. כלומר - עדכונים לפרוטוקול פשוטים יותר מכיוון שעדכון יכול להגיע ברמת האפליקציה, בעוד בפרוטוקול UDP עדכון צריך לבוא ברמת הkernel.

---

2) מנה לפחות שני הבדלים עיקריים בין Cubic ל-.Vegas
Vegas ו-Cubic הינם שני מנגנונים אשר מוגדרים על פי פרוטוקול TCP. תפקידם של פרוטוקולים אלו הוא לשלוט בקצב שידור הנתונים.
ההבדלים העיקריים:
א) Cubic פועל בגישה מבוססת אובדן מידע, בעוד שVegas פועל בגישה המבוססת על זמן שיהוי.
כלומר כדי ש-Cubic יוריד את קצב השידור - אנחנו צריכים לחוות איבוד חבילות.
בVegas זמן השיהוי נמדד על פי RTT - ועל כן למרות שחבילות מגיעות ליעדן יכול להיות שזמן השיהוי שלהן גבוה.
במידה והם פועלים ביחד -> בזמן שVegas יקטין את קצב השליחה בעת עליית זמן השיהוי, Cubic יכול לתפוס את רוחב הפס הזה. בעזרת השוואה בין הקצב הצפוי ל-RTT לקצב בפועל ההפרש יישמש לפענוח האם צפוי אובדן חבילות
ב) אופן גידול החלון:
Cubic משתמש בפונקצייה מעוקבת (פולינום ממעלה שלישית) אשר תלוייה בזמן שעבר מאז אירוע העומס האחרון כדי להגדיל את חלון השליחה, ועל כן הוא "אגרסיבי" - קצב הגידול עד לגבול המוכר יהיה גדול מאוד, ומשם יעבוד בצורה יותר מתונה לבחון כמה אפשר להגדיל את החלון
Vegas לעומת זאת הוא יותר "מתון" ועל כן הגידול שלו בגודל החלון איטי יותר (לרוב קצב לינארי) - לכן יוכל להגיע פחות מהר לקצב השידור שcubic יגיע אליו. מצד שני, הסיכוי לאיבוד חבילות קטן יותר שכן השינוי איטי יותר.

---

3) הסבר מהו פרוטוקול BGP, במה הוא שונה מ-OSPF והאם הוא עובד על פי מסלולים קצרים.
פרוטוקול BGP הינו פרוטוקול למציאת מסלול ברשת (פרוטוקול חיצוני) בין נקודה A לנקודה B.
הפרוטוקול עובד בעזרת AS (Autonomous System) שהם בעצם
אוסף של רשתות ונתבים הנמצאים תחת ניהול של גוף אחד (למשל ספקית אינטרנט מסויימת). הנתבים שיושבים

בגבולות של ה-AS (שנקראים Border Routers) ישתפו את מפות הניתוב שלהם ביניהם.
הפרוטוקול מאפשר מציאת מסלול בעזרת hop-ים, והמסלול שייבחר אינו בהכרח המסלול הקצר ביותר. הפרוטוקול משתמש באלגוריתם Dijkstra לטובת חישוב המסלול הקצר ביותר
המשקלים ייקבעו על פי גישה של ניתוב על פי מדיניות,
בחירת המשקל תעבוד על פי ההכירכיה הבאה:
א) עלות - המסלולים יקבלו משקלים כדי שנבחר מסלול זול יותר
ב) כמות הרשתות בהן אנחנו עוברים במסלול
ג) נקודת כניסה מועדפת לרשת.
OSPF לעומת זאת נועד לניתוב בתוך AS מסויים (בתוך רשת ששייכת לארגון ספציפי, או בעצם פרוטוקול פנימי).
לעומת BGP שיכול לבחור מסלול לפי מספר פרמטרים, OSPF ייבחר לנו את המסלול המהיר ביותר. המסלול המהיר ביותר הוא לא בהכרח הקצר ביותר, יכול להיות שנבחר מסלול ארוך יותר אך שהמידע עובר בו מהר יותר.

---

4) בהינתן הקוד שפיתחתם בפרויקט זה, אנא הוסיפו את הנתונים לטבלה הזו על בסיס תהליך ההודעות של הפרויקט שלכם.
הסבירו איך ההודעות ישתנו אם יהיה NAT בין המשתמש לשרתים, והאם תשתמשו בפרוטוקול QUIC.

| Application | Port Src | Port Des | IP Src | IP Des | Mac Src | Mac Des |
|---|---|---|---|---|---|---|
| DNS | 53 | ephemeral | 127.0.0.2 | 127.0.0.1 | 58-1C-F8-6E-A7-0D | 58-1C-F8-6E-A7-0D |
| API | 8000 | ephemeral | 127.0.0.1 | 127.0.0.1 | 58-1C-F8-6E-A7-0D | 58-1C-F8-6E-A7-0D |
| DASH Proxy | 5000 | ephemeral | From DHCP | 127.0.0.1 | 58-1C-F8-6E-A7-0D | 58-1C-F8-6E-A7-0D |
| Video Server | 9000 | 5000 | 127.0.0.13 | From DHCP | 58-1C-F8-6E-A7-0D | 58-1C-F8-6E-A7-0D |

במידה ויהיה NAT בין המשתמש לשרתים יתבצע "תרגום" של כתובת ה-IP של המשתמש לכתובת הציבורית של ה-NAT
תשובות שרת ה-DNS יוחזרו אל הכתובת הציבורית של ה-NAT, ולאחר מכן ינותבו פנימה אל הלקוח
שרת ה-DHCP יצטרך להתנהל בתוך רשת ה-NAT הפנימית (יחלק כתובות פנימיות)
שימוש בפרוקטול QUIC יוכל לאפשר חיבור יציב יותר שכן פרוטוקול QUIC מזהה את המכשירים בעזרת זיהוי חיבור (connection ID) ולא בעזרת tuple של ip ו port. על כן שינוי בכתובת ה-IP לא יפריע לרציפות החיבור ב-QUIC.

הערה:
מכיוון שכל התקשורת מתבצעת על מכשיר יחיד דרך כתובות Loopback כל החבילות חולקות את אותו Mac Address פיזי. בתנאי רשת אמיתיים כתובת הMAC תשתנה בהתאם למכשיר.

---

5) הסבירו את ההבדלים בין פרוטוקול ARP ל-DNS
פרוטוקול DNS (Domain Name System) הוא פרוטוקול שנועד לאפשר לנו בני האדם דרך יותר פשוטה לגישה למחשבים. בעוד שהמחשבים מכירים כתובות IP שמורכבות מספרות, לבני אדם יותר קל לעבוד עם שמות. כאשר אנחנו ניגשים לכתובת כלשהי בעזרת הדפדפן שלנו אנחנו רושמים את הכתובת כטקסט למשל www.google.com.

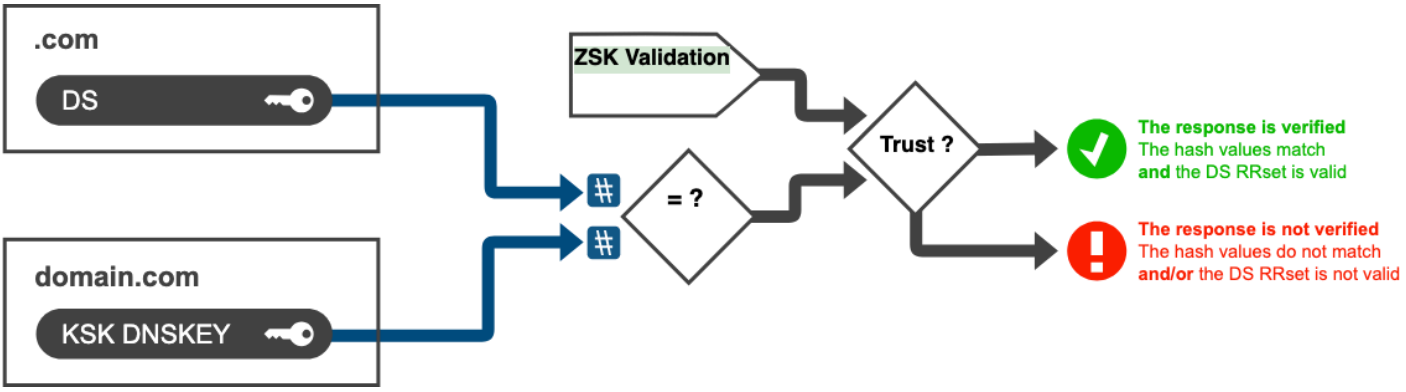
שרת הDNS יקבל בקשה עם הdomain ויבצע לנו
"תרגום" שבסופו נקבל כתובת IP.
ARP ברעיון הוא פרוטוקול דומה - שמאפשר לנו תרגום משפה אחת לשפה אחרת. אך לעומת DNS ARP מאפשר לנו תרגום בין כתובת IP לכתובת MAC. פרוטוקול ARP פועל ברמת הרשת הפנימית בלבד ומאפשר לנו את הגישה הפיזית למכשיר.
מדוע הפרוטוקול פועל ברשת הפנימית? לרשת פנימית יש את מרחב הכתובות שלה - כלומר אם ברשת שלי המחשב בעל כתובת IP 192.168.1.2 הוא אינו אותו מחשב שקיבל כתובת זהה ברשת אחרת. לשם ARP נכנס לתמונה - כתובת MAC מאפשרת לנו זיהוי ייחודי שמכוון אותנו לזיהוי הפיזי של המחשב. כתובת MAC היא ברמת כרטיס הרשת ולא אמורה להשתנות, לעומת כתובת IP שעלולה להתשנות הן ברמת הרשת הפנימית (למשל עם פרוטוקול DHCP) והן ברמת הרשת הכללית. כתובת MAC תישאר זהה למשל גם בעת החלפת מערכת הפעלה

# בונוס

## DNSSEC

---

כדי למנוע התקפות מסוג DNS Spoofing או Poisoning, מימשנו את פרוטוקול ה-**DNSSEC**. מטרת הפרוטוקול היא להוסיף חתימה דיגיטלית לכל רשומה, כך שה-Resolver יוכל לוודא שהמידע הגיע מהמקור המוסמך ולא שונה בדרך.




התהליך עובד על ידי מנגנון חתימה המבוסס על שני מפתחות, מתבצעת הצפנה אסימטרית של המפתחות, ומפתחות הפומביים מועברים בנפרד ל Resolver, ובעת קבלת תשובות, הResolver יכול לאמת שהתשובה שהוא קיבל היא אכן Authoritative Response. בפרויקט שלנו בחרנו שהמפתחות הפומביים יהיו מקודדים ב-Base64 אנחנו מעבירים אותם בתור TXT בכל Message בתהליך הDNS.

כדי להפעיל את התהליך מתבקש יש להריץ את הDNS עם הדגל dnssec דלוק באופן הבא:

```
python .\DNS\main.py --dnssec
```

לאחר מכן נראה הודעה הבאה מודפסת, שימו לב לכותרת כאן אני אדגיש אותה באדום:

```
=====================================================
[*] Initiating Custom DNS Infrastructure Boot Sequence...
[*] Mode: DNSSEC SECURE MODE
=====================================================

[*] Launching Root Server...
[*] Booting Local Root Server...
[*] Loaded 17 records from root.signed.zone
[*] Successfully loaded 17 total records across 1 files.
[*] Server Active on 127.0.0.3:53 (Max Workers: 100)
```

```
[*] Launching TLD Server...
[*] Booting Local TLD Server (.homelab)...
[*] Loaded 17 records from homelab.signed.zone
[*] Successfully loaded 17 total records across 1 files.
[*] Server Active on 127.0.0.4:53 (Max Workers: 100)

[*] Launching Auth Server...
[*] Booting Local Auth Server...
[*] Loaded 11 records from originserver.homelab.signed.zone
[*] Loaded 17 records from test.homelab.signed.zone
[*] Successfully loaded 28 total records across 2 files.
[*] Server Active on 127.0.0.5:53 (Max Workers: 100)

[*] Launching Resolver...
[*] Booting Resolver Manager...
```

| |
|---|
| עבור הוראות לבדיקה ידנית לחץ כאן |

הDNS יבצע טעינה של zones אשר עבורם נוצר RRSIG בתהליך החתמה, במידה ואתם משתמשים בGUI של Electron, או Swagger Endpoints שיצרנו עבור ה-DNS, אז עבור כל שינוי השרת יבצע חתימות ומפתחות לבד.

במידה וערכתם את zones בצורה ידנית, הzones לא יעברו החתמה, והתשובות יפלו עם הודעת שגיאה בקונסולה.

| | |
|---|---|
| ZSK | (ראשי תיבות של Zone Signing Key) זה בעצם המפתח הראשון שבעזרתו נחתום דיגיטלית על כל הרשומות הקיימות לנו בzone הרלוונטי. את המפתח הזה נסמן בעזרת הדגל 256. |
| KSK | (ראשי תיבות של Key Signing Key) המפתח הזה נועד לחתום דיגיטלית על מפתח הZSK הפומבי. את המפתח הזה נסמן בעזרת הדגל 257 |

דוגמא לקובץ חתום root.sizgned.zone:

```
$ORIGIN .
$TTL 86400
```

```
@ IN TXT
"DNSKEY|257|ViUHd6aMtQ45Uky+MQil+eqe2dEOMs6IaQcBI2cOfDp/Ogl0TzpsyPqBtVHwqBFtBnHUSqcf
H1S7/ReNdp63Fg=="
@ IN TXT
"DNSKEY|256|p5byJQWbf9IozIbf+goSpwfHPUD08OqyMfsvSKCTL5VexigYdpC7BxmHARaazQ4JgaMO6juM
0zpZ8QIlM30CfQ=="

.                       86400   IN      SOA     a.root-servers.net. admin.root.
2026022201 1800 900 604800 86400
homelab.                86400   IN      NS      tld.homelab.
homelab. IN TXT
"RRSIG|NS|MEUCIGjOEQkZAeeGEvCaC7NBvQMSYkKHnR9Uo4xBGKJjJM6JAiEA8B59X9WxptU/Vq8CBqvbu9
hz5ihiyh4GqtgZ8RyVkns="

tld.homelab.            86400   IN      A       127.0.0.4
tld.homelab. IN TXT
"RRSIG|A|MEUCIEEBFqP2pNx/6TXx90OR9luWKLoFbm/S9/FA9WL58IaDAiEA8EX5cX1C3Y8TZbRscVIsuJ5
ajngpSIuSLNZkqT3GS1Q="
```

שימו לב, זה קובץ ערוך וחתוך אשר בא להמחיש רק את הרעיון, הקובץ האמיתי ארוך מידי

נעבור רגע על הקובץ, לכל רשומה מתקיים שהשורה הראשונה היא בפורמט BIND9 כמו בקובץ לא חתום רגיל:

```
tld.homelab.            86400   IN      A       127.0.0.4
```

לאחר מכן מופיע הRRSIG שלו, הוא מכיל בתוכו TYPE של הרשומה ולאחר מכן את החתימה:

```
tld.homelab. IN TXT
"RRSIG|A|MEUCIEEBFqP2pNx/6TXx90OR9luWKLoFbm/S9/FA9WL58IaDAiEA8EX5cX1C3Y8TZbRscVIsuJ5
ajngpSIuSLNZkqT3GS1Q="
```

בזמן שהמפתחות הפומביים עצמן מופיעים בחלק העליון של הZone:

```
@ IN TXT
"DNSKEY|257|ViUHd6aMtQ45Uky+MQil+eqe2dEOMs6IaQcBI2cOfDp/Ogl0TzpsyPqBtVHwqBFtBnHUSqcf
H1S7/ReNdp63Fg=="
@ IN TXT
"DNSKEY|256|p5byJQWbf9IozIbf+goSpwfHPUD08OqyMfsvSKCTL5VexigYdpC7BxmHARaazQ4JgaMO6juM
0zpZ8QIlM30CfQ=="
```

יש לציין כי המפתחות הציבוריים (**KSK** ו-**ZSK**) נגישים באופן פומבי בשרת ה-DNS כרשומות TXT, אך אינם נשלחים אוטומטית עם כל תשובה. ה-Resolver מבצע שאילתה נפרדת לקבלת המפתחות רק כאשר הם אינם קיימים בזיכרון המטמון שלו (**Cache**).

## כעת נעבור לתהליך המימוש בפרויקט:

בפרויקט זה נעשה שימוש באלגוריתם ECDSA (עקום אליפטי SECP256R1) ליצירת מפתחות. כל אזור (Zone) מנוהל על ידי שני סוגי מפתחות (המיוצרים בקובץ zone_generator.py):

```
def sign_data(private_key, data_string):
    """Signs a string using ECDSA and SHA-256."""
    signature = private_key.sign(
        data_string.encode('utf-8'),
        ec.ECDSA(hashes.SHA256())
    )
    return base64.b64encode(signature).decode('utf-8')
```

כדי שהתהליך יוכל להתבצע כמו שצריך, נצטרך לבנות שרשרת אמון, כדי שהחבילה החתומה תוכל לעבור בין השרתים השונים בעזרת דלגציה.

תהליך החתימה מתבצע בצורה דינמית על ידי סריקת קבצי ה-Zone בקובץ **zone_signer.py** :

- **רשומות DNSKEY:** לכל Zone מוזרקות רשומות המכילות את המפתחות הציבוריים (KSK ו-ZSK).
- **רשומות DS (Delegation Signer):** כדי ליצור שרשרת אמון בין אב לבן, האב מחזיק רשומת DS שהיא **SHA-256 Hash** של מפתח ה-KSK הציבורי של הבן.
- **רשומות RRSIG:** כל רשומה ב-Zone (כולל ה-DNSKEY וה-DS) נחתמת דיגיטלית. ה-KSK חותם על ה-DNSKEYs, וה-ZSK חותם על שאר הרשומות.

```
if target_ip_ans and rrsig_b64:
            is_secure = DNSSECValidator.verify_dnssec_chain(
                domain=actual_record_name, qtype_str="A", rdata_str=target_ip_ans,
                signature_b64=rrsig_b64, auth_ip=auth_ip, bind_ip=self.config.bind_ip,
                dnskey_cache=self.dnskey_cache, ds_cache=self.ds_cache
            )

            if not is_secure:
                print(f"[*] SECURE RESOLUTION ABORTED: {qname} failed validation.")
                error_reply = request.reply()
                error_reply.header.rcode = getattr(RCODE, 'SERVFAIL')
                socket_ref.sendto(error_reply.pack(), addr)
                return None
```

תהליך האימות ב- `dnssec_validator.py` (Resolver):

כאשר ה-Resolver מקבל תשובה, הוא אינו מסתפק במידע הגולמי אלא מבצע אימות היררכי בפונקציה verify_dnssec_chain:

1. **אימות עצמי (KSK Integrity):** בדיקה שמפתח ה-KSK של ה-Zone חתם בצורה תקינה על עצמו.
2. **אימות מפתח ה-Zone (ZSK Validation):** בדיקה שמפתח ה-KSK חתם על מפתח ה-ZSK.
3. **בדיקת שרשרת האמון (DS Check):** ה-Resolver מוודא שה-Hash של מפתח ה-KSK של הבן תואם לרשומת ה-DS שנמצאת אצל האב (אותה הוא כבר אימת קודם לכן).
4. **אימות הרשומה הסופית:** ביצוע חישוב מתמטי (ECDSA) בעזרת ה-ZSK על נתוני הרשומה (למשל IP) והשוואתו לחתימה (RRSIG) שהתקבלה.

```python
    def verify_dnssec_chain(domain, qtype_str, rdata_str, signature_b64, auth_ip,
bind_ip, dnskey_cache, ds_cache, auth_port=53):
        """
        Verifies the signature, utilizing a local cache for DNSKEYs to avoid network
spam.
        """
        print(f"\n[*] DNSSEC: Initiating Chain of Trust Verification for {domain}")

        # 1. Figure out the apex domain (e.g., test.homelab.)
        apex_domain = ".".join(domain.strip(".").split(".")[-2:]) + "."

        zsk_pub = None
        ksk_pub = None

        # 2. THE CACHE CHECK (Using the passed-in dictionary)
        if apex_domain in dnskey_cache:
            print(f"    [+] KEY CACHE HIT: Using stored ZSK for {apex_domain}")
            zsk_pub = dnskey_cache[apex_domain].get('ZSK')
            ksk_pub = dnskey_cache[apex_domain].get('KSK')

        else:
            # 3. THE CACHE MISS (Network Fetch)
            print(f"    -> KEY CACHE MISS: Fetching DNSKEYs for {apex_domain} from
{auth_ip}...")
            key_request = DNSRecord.question(apex_domain, "TXT")

            try:
                sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
                sock.settimeout(2.0)

                # Bind to the Resolver's IP passed from the main class
                sock.bind((bind_ip, 0))

                sock.sendto(key_request.pack(), (auth_ip, auth_port))
                response_data, _ = sock.recvfrom(4096)
                key_reply = DNSRecord.parse(response_data)

                ksk_pub = None
                zsk_pub_temp = None
                zsk_signature = None
                ksk_signature = None

                for rr in key_reply.rr:
                    if rr.rtype == getattr(QTYPE, 'TXT'):
                        txt_val = b"".join(rr.rdata.data).decode('utf-8')

                        if txt_val.startswith("DNSKEY|257|"):
                            ksk_pub = txt_val.split("|")[2]
                        elif txt_val.startswith("RRSIG|DNSKEY|257|"):
```

```
                            ksk_signature = txt_val.split("|")[3]
                        elif txt_val.startswith("DNSKEY|256|"):
                            zsk_pub_temp = txt_val.split("|")[2]
                        elif txt_val.startswith("RRSIG|DNSKEY|256|"):
                            zsk_signature = txt_val.split("|")[3]

                if ksk_pub and zsk_pub_temp and ksk_signature and zsk_signature:
                    print(f"    -> Verifying KSK self-signature (Integrity
Check)...")
                    ksk_data_string = f"{apex_domain}|DNSKEY|257|{ksk_pub}"
                    is_ksk_valid = DNSSECValidator.verify_signature(ksk_pub,
ksk_data_string, ksk_signature)

                    if not is_ksk_valid:
                        print("    [!] DNSSEC BOGUS: KSK self-signature failed! Key
is corrupted or forged.")
                        return False
                    print("    [+] DNSSEC SECURE: KSK self-signature is intact.")

                    print(f"    -> Verifying ZSK signature using the KSK...")
                    zsk_data_string = f"{apex_domain}|DNSKEY|256|{zsk_pub_temp}"
                    is_zsk_valid = DNSSECValidator.verify_signature(ksk_pub,
zsk_data_string, zsk_signature)

                    if not is_zsk_valid:
                        print("    [!] DNSSEC BOGUS: KSK did NOT sign this ZSK!
Forgery detected.")
                        return False
                    print("    [+] DNSSEC SECURE: KSK successfully validated the
ZSK.")

                    zsk_pub = zsk_pub_temp
                    # SAVE IT TO THE VAULT (Updates the Resolver's dictionary
automatically!)
                    dnskey_cache[apex_domain] = {'KSK': ksk_pub, 'ZSK': zsk_pub}
                    print(f"    [+] Successfully fetched and cached DNSKEYs for
{apex_domain}")
                else:
                    print("    [!] DNSSEC BOGUS: Missing necessary keys/signatures
in Auth response!")
                    return False

            except socket.timeout:
                print("    [!] DNSSEC ERROR: Timeout fetching keys.")
                return False
            except Exception as e:
                print(f"    [!] DNSSEC ERROR: {e}")
                return False
            finally:
```

```
                sock.close()

        # 4. Verify the KSK against the Parent's DS Hash
        if apex_domain != ".":
            if apex_domain in ds_cache:
                parent_ds_hash = ds_cache[apex_domain]
                print(f"    -> Verifying KSK against Parent DS hash...")

                is_ds_valid = DNSSECValidator.verify_ds_record(ksk_pub,
parent_ds_hash)
                if not is_ds_valid:
                    print("    [!] DNSSEC BOGUS: KSK does not match Parent DS!")
                    return False
                print("    [+] DNSSEC SECURE: Parent DS vouches for this KSK!")
            else:
                print(f"    [!] DNSSEC BOGUS: No DS record found for {apex_domain}.
Chain broken!")
                return False

        # 5. Final math on the actual record
        data_to_verify = f"{domain}|{qtype_str}|{rdata_str}"
        print(f"    -> Running ECDSA Math on: {data_to_verify}")

        is_valid = DNSSECValidator.verify_signature(zsk_pub, data_to_verify,
signature_b64)

        if is_valid:
            print("    [+] DNSSEC SECURE: Cryptographic math checks out!")
            return True
        else:
            print("    [!] DNSSEC BOGUS: Cryptographic math FAILED!")
            return False
```

לביצוע בדיקה

כעת ארשום איך לבצע בדיקה לDNSSEC, היות שnslookup לא מגיע עם אופציה לDNSSEC, אז כל הרצה באמצעות nslookup לא תעבודת לשם כך נשתמש בפקודה הבא דרך powershell:

```
Resolve-DnsName -Name www.test.homelab -Server 127.0.0.2 -DnssecOk
```

תשובה תקינה מה DNSSEC אמורה להיראות כך:

```
Name                           Type   TTL   Section    NameHost
----                           ----   ---   -------    --------
www.test.homelab               CNAME  86400 Answer     server1.test.homelab
Name      : www.test.homelab
QueryType : TXT
```

```
TTL       : 86400
Section   : Answer
Strings   :
{RRSIG|CNAME|MEQCICy3l9GX3Si684KhiXjEaRrIuWecOf6Q5zt7uU9L+Dq1AiA/MLGbflF++2PyfgaPf+p
oMGqldY9TdE9GWkANKnnZAA                                    ==}
Name       : server1.test.homelab
QueryType  : A
TTL        : 86400
Section    : Answer
IP4Address : 192.168.1.100
Name      : server1.test.homelab
QueryType : TXT
TTL       : 86400
Section   : Answer
Strings   :
{RRSIG|A|MEUCIQCdsrIvF6Xgi9+OIr1LQtaJN0AN/EnXexBtN+ilazFJ1AIgB1DqCTyKGmXjKWMQXkpVxGe
vCYYqKcG7GcZF5XFXOWY=}
```

בקונסולה יהיה מודפס:

```
[*] CNAME CHASE: Appended A record for server1.test.homelab.
[*] NODATA: Name 'www.test.homelab.' exists, but no AAAA records.
    [+] DNSSEC: Attached RRSIG for Chased A record
    [*] Attached SOA record for test.homelab. to Authority section.
    [*] Final exact answer found at 127.0.0.5!
    [*] Authoritative NODATA (SOA) received. Record type does not exist.

[*] DNSSEC: Initiating Chain of Trust Verification for server1.test.homelab.
[!] Warning: DNSSEC validation required but missing RRSIG. Assuming insecure.
    -> KEY CACHE MISS: Fetching DNSKEYs for test.homelab. from 127.0.0.5...
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
    -> Verifying KSK self-signature (Integrity Check)...
    [+] DNSSEC SECURE: KSK self-signature is intact.
    -> Verifying ZSK signature using the KSK...
    [+] DNSSEC SECURE: KSK successfully validated the ZSK.
    [+] Successfully fetched and cached DNSKEYs for test.homelab.
    -> Verifying KSK against Parent DS hash...
    [+] DNSSEC SECURE: Parent DS vouches for this KSK!
    -> Running ECDSA Math on: server1.test.homelab.|A|192.168.1.100
    [+] DNSSEC SECURE: Cryptographic math checks out!
```

אכן מדובר בהרבה טקסט ולכן בוא נחלק אותו לחלקים באופן הבא:

חלק ראשון: הצמדה של DNS של הרשומות.

```
[*] CNAME CHASE: Appended A record for server1.test.homelab.
[*] NODATA: Name 'www.test.homelab.' exists, but no AAAA records.
    [+] DNSSEC: Attached RRSIG for Chased A record
    [*] Attached SOA record for test.homelab. to Authority section.
    [*] Final exact answer found at 127.0.0.5!
    [*] Authoritative NODATA (SOA) received. Record type does not exist.
```

הResolver מצא תשובה סופית אצל הAuth Name Server וזאת ההדפסה.

---

חלק שני: תחילת תהליך של שרשרת האימון:

```
[*] DNSSEC: Initiating Chain of Trust Verification for server1.test.homelab.
[!] Warning: DNSSEC validation required but missing RRSIG. Assuming insecure.
    -> KEY CACHE MISS: Fetching DNSKEYs for test.homelab. from 127.0.0.5...
```

על מנת לעבור את תהליך שרשרת האימון הResolver צריך מפתחות, בDNSSEC אמיתי המפתחות אינן עוברות דרך הpayload בפורמט TXT אלא נוספות לסוף החבילה, על מנת לשמור על פשטות אנחנו מעבירים אותה בצורה גלויה, אבל מכיוון שלהעביר בכל חבילה את המפתחות יעמיס המון על הרשת, את המפתחות עצמן אנחנו מעבירים אותם אך ורק אם אין לא אותם בבקשה נוספת, ואז מאתחלים אותם בCache שחיי רק במהלך ריצה של התוכנית.

---

חלק שלישי:

כאן זה החלק ה"מוזר", תחילה ניתן יהיה לחשוב שזה המפתחות של כל הZones שאנחנו עוברים בהם. אבל זה לא!

```
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
[*] ANSWER: Appended TXT record for test.homelab.
```

למעשה, זה פשוט פתרון נאיבי שביצענו כדי להשיג את כל המפתחות במכה אחתת מבלי להיכנס יותר מידי לregex או לחיתוך של TXT לפני שאנחנו שולחים אותם בחזרה, מכיוון של כל המפתחות הם מסוג TXT לא משנה מה, ואנחנו צריכים את הנתונים הבאים עבור תהליך האימות:

```
  if txt_val.startswith("DNSKEY|257|"):   → ksk_pub
  if txt_val.startswith("DNSKEY|256|"):   → zsk_pub
  if txt_val.startswith("RRSIG|DNSKEY|257|"): → ksk_signature
  if txt_val.startswith("RRSIG|DNSKEY|256|"): → zsk_signature
```

אנחנו פשוט שולחים בקשה שמביאה את כל הנתונים שהם TXT, ורק בסוף מסננים ושומרים אותם בCache, כתוצאה מכך יש לנו TXT אחד מיותר שזה בעצם ה RSIG של הAuth NameServer.

חלק רביעי וסופי:

שלב האימות, לשם האימות הResolver שולף את שאר המפתחות שנמצאים כבר ב Cache, אותם הוא השיג כשאר הוא טייל בעץ של הDNS, ולאחר מכן הוא מבצע אימות שאכן מדובר במפתחות הנכונים, במידה והאימות עובר בהצלחה יודפס ההודעה שאימות הקריפטוגרפיה עבר בהצלחה ונקבל תשובה.

```
    -> Verifying KSK self-signature (Integrity Check)...
    [+] DNSSEC SECURE: KSK self-signature is intact.
    -> Verifying ZSK signature using the KSK...
    [+] DNSSEC SECURE: KSK successfully validated the ZSK.
    [+] Successfully fetched and cached DNSKEYs for test.homelab.
    -> Verifying KSK against Parent DS hash...
    [+] DNSSEC SECURE: Parent DS vouches for this KSK!
    -> Running ECDSA Math on: server1.test.homelab.|A|192.168.1.100
    [+] DNSSEC SECURE: Cryptographic math checks out!
```

במידה ונכשל:

```
    -> Verifying KSK self-signature (Integrity Check)...
    [+] DNSSEC SECURE: KSK self-signature is intact.
    -> Verifying ZSK signature using the KSK...
    [+] DNSSEC SECURE: KSK successfully validated the ZSK.
    [+] Successfully fetched and cached DNSKEYs for test.homelab.
    -> Verifying KSK against Parent DS hash...
    [+] DNSSEC SECURE: Parent DS vouches for this KSK!
    -> Running ECDSA Math on: server1.test.homelab.|A|192.168.1.101
[!] DNSSEC BOGUS: Cryptographic signature mismatch!
    [!] DNSSEC BOGUS: Cryptographic math FAILED!
[*] SECURE RESOLUTION ABORTED: www.test.homelab. failed validation.
```

השרת מניח שמישהו נגע ברשומות, שהמידע עבר דרך איזשהו middleman חיצוני ושהוא אינו מאובטח יותר בזה הרגע, ולכן הוא מפיל את הבקשה ובתוך powershell יודפס:

```
Resolve-DnsName : www.test.homelab : DNS server failure
At line:1 char:1
+ Resolve-DnsName -Name www.test.homelab -Server 127.0.0.2 -DnssecOk
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
+ CategoryInfo          : ResourceUnavailable: (www.test.homelab:String)
[Resolve-DnsName], Win32Exception                                    +
FullyQualifiedErrorId :
RCODE_SERVER_FAILURE,Microsoft.DnsClient.Commands.ResolveDnsName
```

במידה והינכם רוצים לנסות לבד, אתם יכולים לערוך את אחת הכתובות ברשומות שאתם שולחים לה בקשה, רק שימו לב שיש למחוק את הCache של הResolver, אשר נמצא בתיקיית data. מכיוון שהוא פשוט ישלוף רשומות תקינות לפני השינוי ויאמת אותן

# API Server

---

לשליטה על הגדרות ה-DNS: מטרת ה-API לאפשר התעסקות נוחה יותר עם הגדרות ה-DNS ה-UI מבוסס על swagger endpoints.

**DNS Resolver API** 0.1.0 OAS 3.1

/openapi.json

**default**

| | | |
|---|---|---|
| GET | `/api/zones/list/{tier}` | List Zone Files |
| GET | `/api/zone/{server_name}/{zone_name}` | Get Zone |
| POST | `/api/zone/{server_name}/{zone_name}` | Save Zone |
| DELETE | `/api/zone/{server_name}/{zone_name}` | Delete Zone |
| GET | `/api/zone/{server_name}/{zone_name}/records/a` | Get A Records |
| POST | `/api/zone/{server_name}/{zone_name}/records/a/{record_id}` | Update A Record |
| GET | `/api/config/{server_name}` | Get Server Config |
| POST | `/api/config/{server_name}` | Save Server Config |

הסבר קצר על ה-Endpoints על פי קטגוריות:

**ניהול אזורי DNS (Zones)**

- **`GET /api/zones/list/{tier}`** מחזיר רשימה של שמות כל קבצי ה-Zone הקיימים בתיקייה של ה-tier המבוקש. ה-API מסנן החוצה קבצים חתומים (signed) ומציג רק את קבצי המקור.
- **`GET /api/zone/{server_name}/{zone_name}`** קורא ומפענח קובץ Zone ספציפי. הוא מחלץ את ה-Origin, ה-TTL ורשימת הרשומות (A, NS, CNAME וכו') ומחזיר אותם במבנה JSON מסודר.
- **`POST /api/zone/{server_name}/{zone_name}`** שומר נתוני Zone חדשים או מעדכן קיימים. לאחר השמירה, ה-API מנסה לבצע חתימה דיגיטלית (DNSSEC) על הקובץ באמצעות מפתחות KSK ו-ZSK.
- **`DELETE /api/zone/{server_name}/{zone_name}`** מוחק את קובץ ה-Zone המבוקש מהשרת. במידה וקיים קובץ חתום (signed.zone) תואם, ה-API מוחק גם אותו.

---

**ניהול רשומות ספציפיות (Records)**

- **`GET /api/zone/{server_name}/{zone_name}/records/a`** שולף ומציג אך ורק את רשומות ה-A (כתובות IPv4) מתוך אזור DNS מסוים. שימושי לסינון מהיר של כתובות IP המשויכות לשמות מתחם.

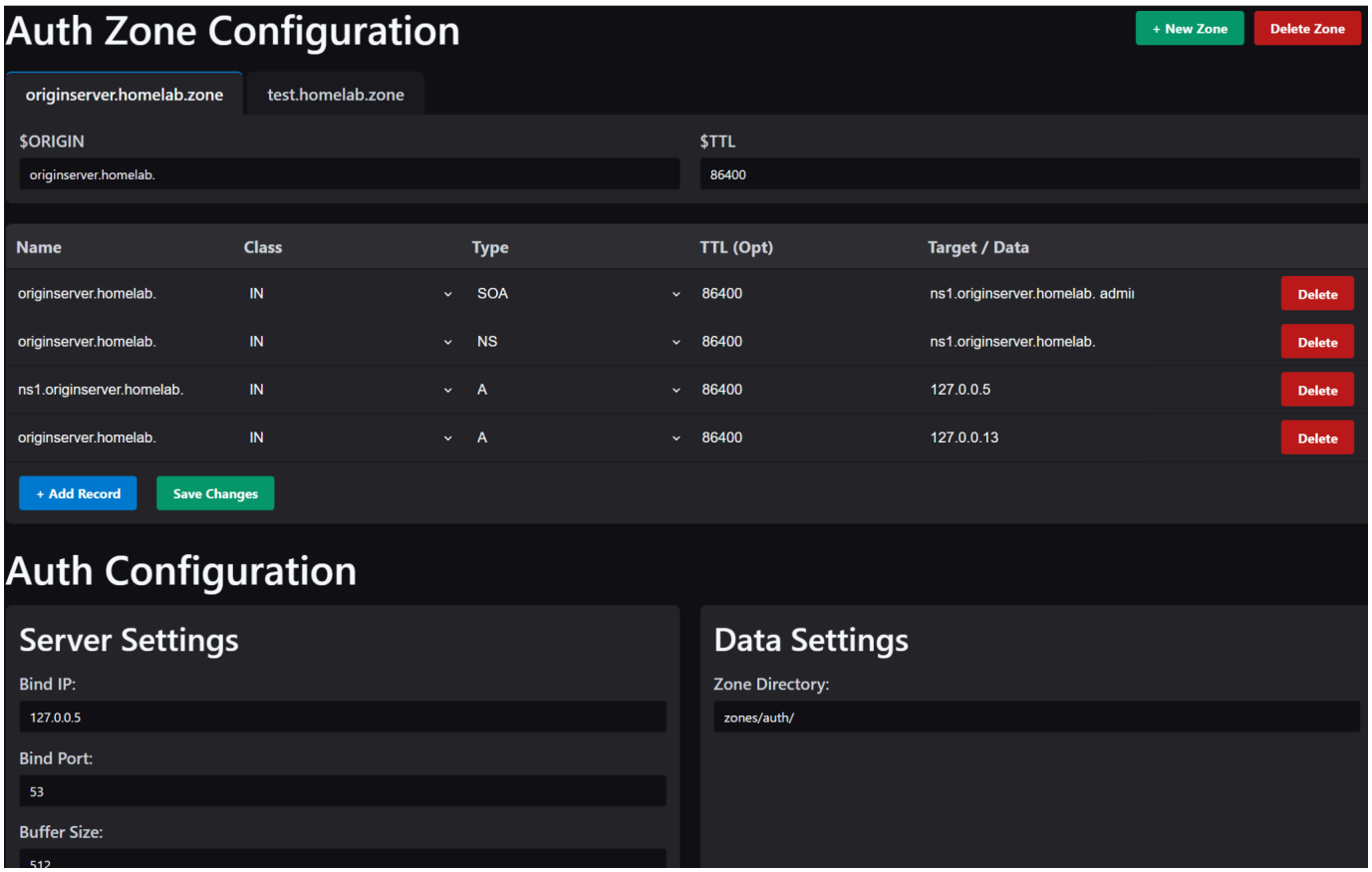
- `POST /api/zone/{server_name}/{zone_name}/records/a/{record_id}` מעדכן רשומת A קיימת לפי ה-ID שלה, או יוצר רשומה חדשה אם ה-ID לא נמצא. לאחר השינוי, ה-API שומר מחדש את כל ה-Zone ומבצע חתימה.

---

### הגדרות שרת (Config)

- `GET /api/config/{server_name}` טוען ומחזיר את קובץ ה-YAML של הגדרות השרת (Resolver או Name Server). המידע כולל כתובות IP להאזנה, פורטים, זמני Timeout והגדרות אחסון.
- `POST /api/config/{server_name}` מעדכן את קובץ הקונפיגורציה (YAML) של השרת לפי הנתונים שנשלחו. ה-API מבצע וולידציה למבנה הנתונים לפני הכתיבה לדיסק.

## Electron - GUI (DNS)




מאפשר לנו צורה נוחה יותר לעבודה בעזרת ה-API

### 1. תפריט ניווט צדדי (Sidebar)

- **DNS Servers:** רשימה של השרתים הפעילים לפי היררכיית ה-DNS (Root, TLD, AUTH). לכל שרת מוצג סטטוס (Online) וכתובת ה-IP המקומית שלו (למשל `127.0.0.3` עבור ה-Root).
- **Util Servers:** גישה לניהול ה-Resolver (שמבצע את השאילתות עבור הלקוח) וה-API (שמתווך בין הממשק לקוד).

---

### 2. ניהול אזור (Root Zone Configuration)

- **הגדרות בסיס ($TTL$ / ORIGIN):** מאפשר להגדיר את נקודת המוצא של האזור ואת זמן החיים ברירת המחדל של הרשומות בשניות.
- **טבלת רשומות (Records Table):** ממשק עריכה ויזואלי להוספה, מחיקה ושינוי של רשומות (כמו SOA, NS, A).
  - לדוגמה: ניתן לראות שה-Root מפנה את הסיומת `homelab.` לשרת ה-TLD שנמצא בכתובת `127.0.0.4`.
- **כפתורי פעולה:** כפתור `+ Add Record` להוספת שורה חדשה ו-`Save Changes` ששולח את הנתונים ל-API לצורך שמירה וחתימה מחדש (DNSSEC).

---

## 3. הגדרות שרת (Root Configuration)

- **Server Settings:** מאפשר להגדיר את פרמטרי הרשת של השרת הספציפי – לאיזו כתובת IP הוא מאזין (Bind IP) ובאיזה פורט. בנוסף באפשרותנו לערוך את גודל הבאפר.
- **Data Settings:** מגדיר למערכת איפה פיזית נמצאים קובצי ה-Zone על הדיסק (למשל `/zones/root`), כך שה-API ידע לאן לכתוב את השינויים.

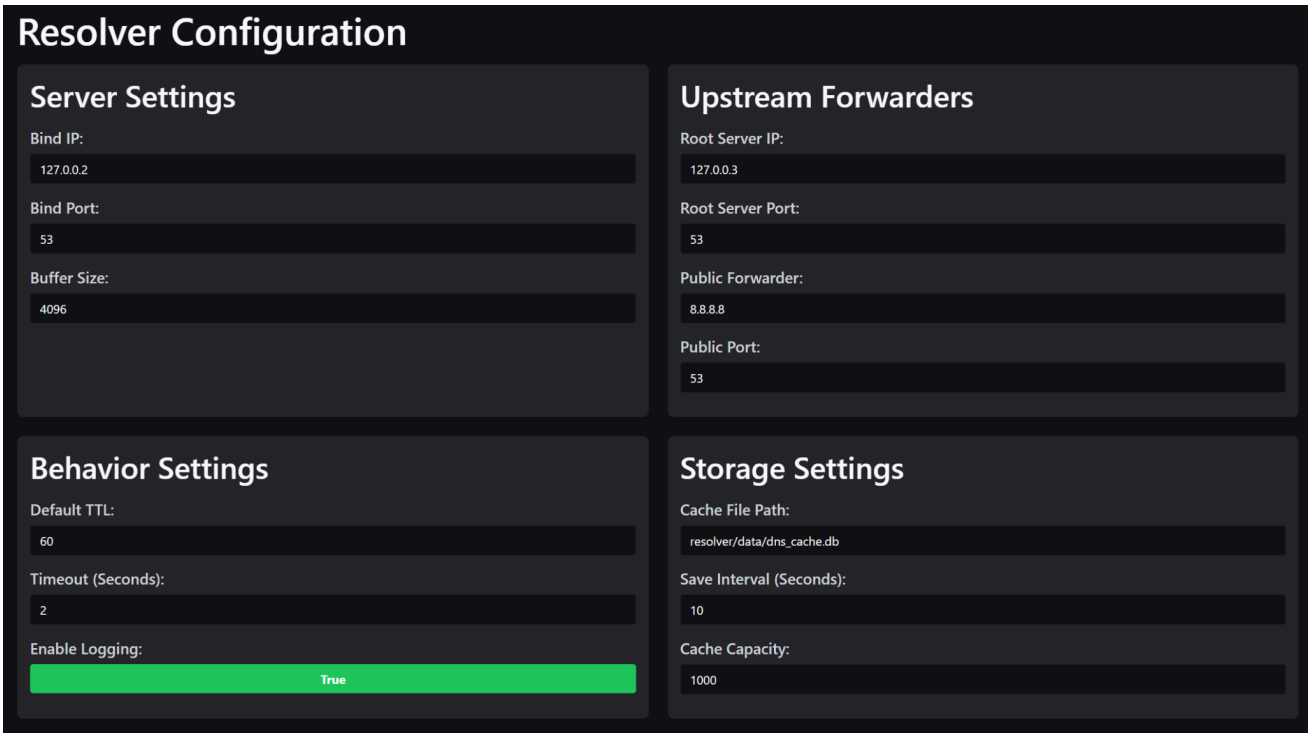
---

## 4. כלי מערכת ותצוגה

- **ניהול אזורים כללי:** כפתור `+ New Zone` ליצירת היררכיה חדשה ו-`Delete Zone` למחיקה מוחלטת של האזור הנוכחי.
- **מצב תצוגה:** כפתור `Light Mode` בפינה התחתונה למעבר בין עיצוב כהה לבהיר.

---

## 5. הגדרות ה-Resolver




- **הגדרות שרת (Server Settings)**
  - **Bind IP & Port:** הכתובת (`127.0.0.2`) והפורט (53) שבהם ה-Resolver מאזין לבקשות מהלקוחות
  - **Buffer Size:** גודל הזיכרון הזמני המוקצה לטיפול בחבילות המידע (Packet) הנכנסות מהרשת.

- **שרתי יעד (Upstream Forwarders)**
  - **Root Server IP:** נקודת ההתחלה של כל שאילתה חדשה. כאן ה-Resolver יודע לפנות קודם כל לשרת ה-Root שהגדרת בכתובת `127.0.0.3`.
  - **Public Forwarder:** "רשת ביטחון". אם ה-Resolver לא מצליח למצוא תשובה בשרתים המקומיים, הוא יכול להפנות את השאלה לשרת חיצוני (במקרה שלנו בחרנו את 8.8.8.8 של גוגל)
- **התנהגות המערכת (Behavior Settings)**
  - **Default TTL & Timeout:** מגדיר לכמה זמן ה-Resolver יזכור תשובה אם לא צוין אחרת, וכמה שניות הוא יחכה לתגובה משרת אחר לפני שיתייאש (כאן מוגדרות 2 שניות).
  - **Enable Logging:** מתג המפעיל תיעוד בזמן אמת של כל הבקשות שעוברות דרך השרת, קריטי לניפוי שגיאות (Debugging).
- **הגדרות אחסון (Storage Settings)**
  - **Cache File Path:** הנתיב לקובץ מסד הנתונים שבו נשמרות כל התשובות שה-Resolver כבר מצא, כדי שלא יצטרך לשאול שוב את אותה שאלה.
  - **Cache Capacity:** כמות הרשומות המקסימלית שהשרת ישמור בזיכרון בו-זמנית (כאן מוגבל ל-1000 רשומות) כדי למנוע צריכת זיכרון מופרזת.

# WIRESHARK:

## SERVER + PROXY:

התחלת השרת ופורקסי.

**דוגמה ב-RUDP**

מתחילים מלבקש כתובת IP מהשרת DHCP (שימוש בDORA), ולאחר מכן הפרוקסי מקבל את הכתובת של השרת מהDNS.

נשים לב לבקשה לשרת בכתובת 127.0.0.13 - מסוג REQ לבקשת הסרטון

Wireshark · Packet 93 · Adapter for loopback traffic capture

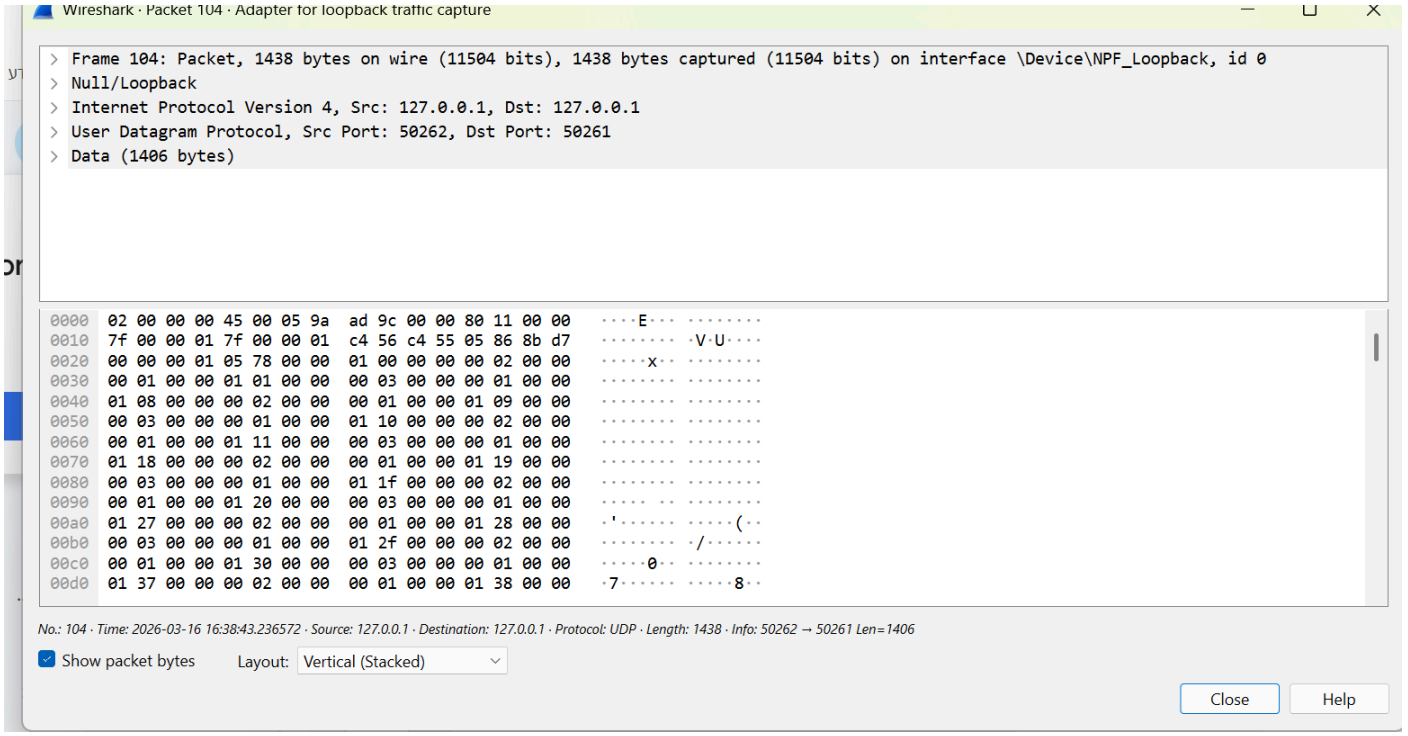
```
> Frame 93: Packet, 51 bytes on wire (408 bits), 51 bytes captured (408 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.13
> User Datagram Protocol, Src Port: 50261, Dst Port: 9000
> Data (19 bytes)

0000  02 00 00 00 45 00 00 2f  0e 44 00 00 80 11 00 00   ····E··/ ·D····· 
0010  7f 00 00 01 7f 00 00 0d  c4 55 23 28 00 1b f3 e4   ····· ·U#(····
0020  52 45 51 7c 6d 6f 76 69  65 5f 37 32 30 2e 6d 70   REQ|movi e_720.mp
0030  34 7c 30                                           4|0
```

לאחר מכן נקבל במספר חבילות את הMetadata הדרוש

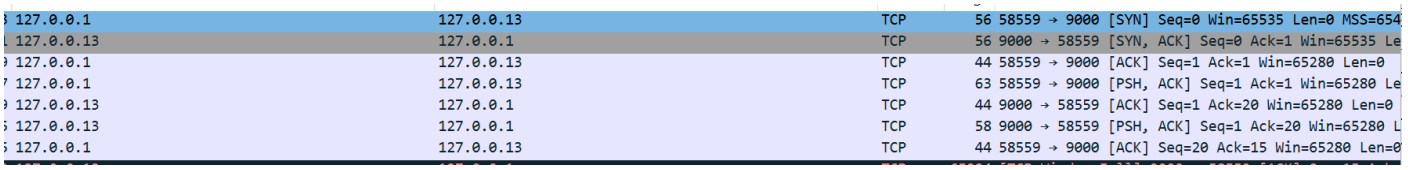
Wireshark · Packet 94 · Adapter for loopback traffic capture

```
> Frame 94: Packet, 53 bytes on wire (424 bits), 53 bytes captured (424 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 50262, Dst Port: 50261
> Data (21 bytes)

0000  02 00 00 00 45 00 00 31  ad 94 00 00 80 11 00 00   ····E··1 ·····
0010  7f 00 00 01 7f 00 00 01  c4 56 c4 55 00 1d ba 99   ····· ·V·U····
0020  4d 45 54 41 7c 31 35 30  32 35 30 32 38 7c 31 30   META|150 25028|10
0030  34 38 35 37 36                                     48576
```

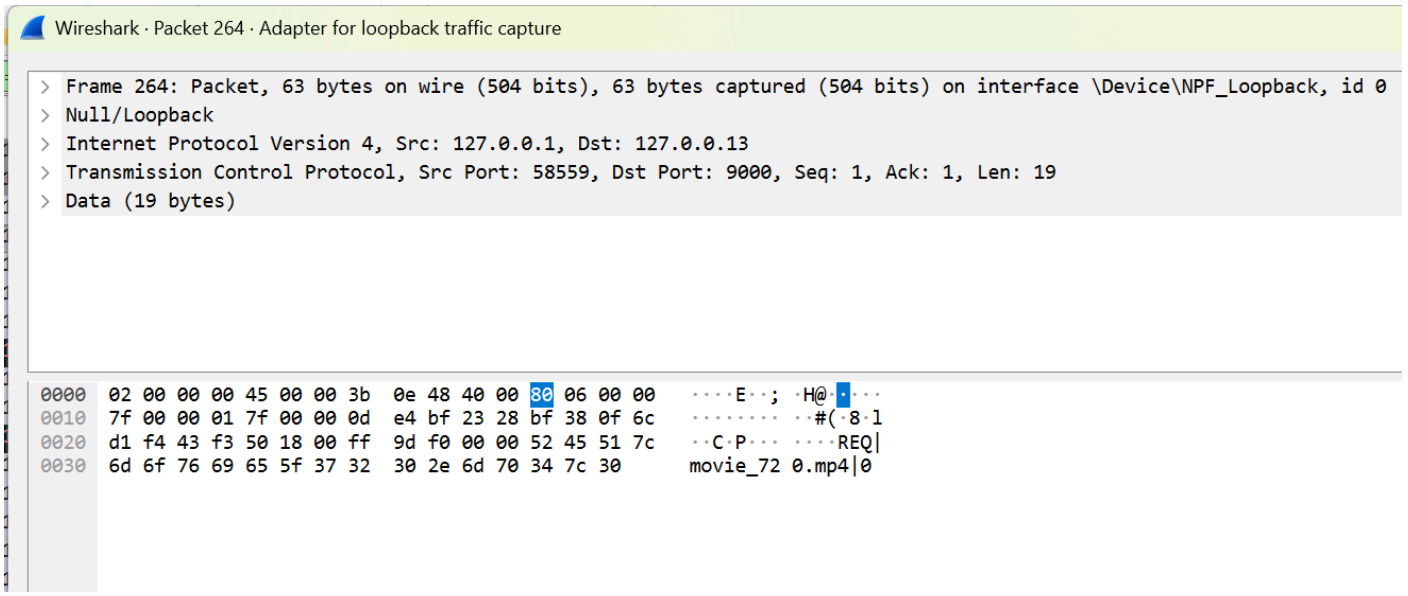
ולאחר מכן נקבל חבילות רבות של תוכן הוידאו עצמו




**דוגמה ב-TCP נשים לב לתקשורת מול השרת - יתחיל תהליך ה-handshake ולאחר מכן יתחיל לעבור מידע**




**בדומה תתבצע בקשה לקובץ הסרטון**




**וגם בקשה ל-metadata**

Wireshark · Packet 266 · Adapter for loopback traffic capture

```
> Frame 266: Packet, 58 bytes on wire (464 bits), 58 bytes captured (464 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.13, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 9000, Dst Port: 58559, Seq: 1, Ack: 20, Len: 14
> Data (14 bytes)

0000  02 00 00 00 45 00 00 36  13 d2 40 00 80 06 00 00   ····E··6 ··@·····
0010  7f 00 00 0d 7f 00 00 01  23 28 e4 bf d1 f4 43 f3   ........ #(····C·
0020  bf 38 0f 7f 50 18 00 ff  d6 cf 00 00 4d 45 54 41   ·8··P··· ····META
0030  7c 31 35 30 32 35 30 32  38 0a                     |1502502 8·
```

## ולבסוף יתחיל לעבור הסרטון

Wireshark · Packet 276 · Adapter for loopback traffic capture

```
> Frame 276: Packet, 597 bytes on wire (4776 bits), 597 bytes captured (4776 bits) on interface \Device\NPF_Loopback, id 0
> Null/Loopback
> Internet Protocol Version 4, Src: 127.0.0.13, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 9000, Dst Port: 58559, Seq: 261606, Ack: 20, Len: 553
> Data (553 bytes)

0000  02 00 00 00 45 00 02 51  13 d7 40 00 80 06 00 00   ····E··Q ··@·····
0010  7f 00 00 0d 7f 00 00 01  23 28 e4 bf d1 f8 41 d8   ........ #(····A·
0020  bf 38 0f 7f 50 18 00 ff  17 16 00 00 67 93 17 f3   ·8··P··· ····g···
0030  ef a1 1b c7 1f 7f fc 45  17 5d f4 87 3f ac 2d 8b   ·******E ·]··?·-·
0040  2b e7 99 79 78 a8 0f ff  9f cb 66 4c 6b fb c4 27   +··yx··· ··fLk··'
0050  3e f1 c9 4f 5e d3 de de  0e 2d c0 22 01 1e 9e 4e   >··O^··· ·-·"···N
0060  a7 73 81 48 84 dd 71 72  cd 4c d2 24 bd af e8 88   ·s·H··qr ·L·$····
0070  55 80 99 05 5d 22 8c 05  37 24 12 4e 77 ea 78 9f   U···]"·· 7$·Nw·x·
0080  e3 6b 55 36 82 4c 26 88  2c 1c ba 90 7d 94 f3 3f   ·kU6·L&· ,···}··?
0090  b8 56 fe 7c 3b 92 41 44  de f2 05 1f 3d 71 59 dd   ·V·|;·AD ····=qY·
00a0  ce 3d 2d 1b 36 6c ca 0e  a4 28 b8 c2 6c bc 0b f1   ·=-·6l·· ·(··l···
00b0  d3 57 d4 4b 4d d7 5a 8f  77 94 f8 b4 28 08 9a b2   ·W·KM·Z· w···(···
00c0  9f da a4 77 da 96 7d e6  19 07 8e 39 fa 55 91 1c   ···w··}· ···9·U··
00d0  54 45 d7 ae ef 62 fc 96  39 09 78 e6 d5 51 d0 5b   TE···b·· 9·x··Q·[
```

*No.: 276 · Time: 2026-03-16 16:45:19.182643 · Source: 127.0.0.13 · Destination: 127.0.0.1 · Protocol: TCP · Length: 597 · Info: 9000 → 58559 [PSH, ACK] Seq=261606 Ack=20 Win=65280 Len=553*

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 54323 → 54322 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 2 | 0.000189 | 127.0.0.1 | 127.0.0.1 | TCP | 56 | 54322 → 54323 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 3 | 0.000244 | 127.0.0.1 | 127.0.0.1 | TCP | 44 | 54323 → 54322 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |
| 4 | 0.039458 | 127.0.0.1 | 127.0.0.1 | UDP | 290 | 50491 → 6700 Len=258 |
| 5 | 0.040784 | 127.0.0.1 | 127.0.0.1 | UDP | 320 | 50492 → 50491 Len=288 |
| 6 | 0.041294 | 127.0.0.1 | 127.0.0.1 | UDP | 290 | 50491 → 6700 Len=258 |
| 7 | 0.042075 | 127.0.0.1 | 127.0.0.1 | UDP | 288 | 50492 → 50491 Len=256 |
| 8 | 0.242179 | 127.0.0.1 | 127.0.0.1 | UDP | 298 | 50493 → 6700 Len=266 |
| 9 | 0.243026 | 127.0.0.1 | 127.0.0.1 | UDP | 328 | 50492 → 50493 Len=296 |
| 10 | 0.243533 | 127.0.0.1 | 127.0.0.1 | UDP | 298 | 50493 → 6700 Len=266 |
| 11 | 0.244346 | 127.0.0.1 | 127.0.0.1 | UDP | 288 | 50492 → 50493 Len=256 |
| 12 | 0.245415 | 127.0.0.1 | 127.0.0.2 | DNS | 70 | Standard query 0xa761 A originserver.homelab |
| 13 | 0.246476 | 127.0.0.2 | 127.0.0.3 | DNS | 70 | Standard query 0xa761 A originserver.homelab |
| 14 | 0.247161 | 127.0.0.3 | 127.0.0.2 | DNS | 104 | Standard query response 0xa761 A originserver.homelab NS tld.homelab A 127.0.0. |
| 15 | 0.247612 | 127.0.0.2 | 127.0.0.4 | DNS | 70 | Standard query 0xa761 A originserver.homelab |
| 16 | 0.248968 | 127.0.0.4 | 127.0.0.2 | DNS | 302 | Standard query response 0xa761 A originserver.homelab NS ns1.originserver.homel |
| 17 | 0.250502 | 127.0.0.2 | 127.0.0.5 | DNS | 70 | Standard query 0xa761 A originserver.homelab |
| 18 | 0.251403 | 127.0.0.5 | 127.0.0.2 | DNS | 203 | Standard query response 0xa761 A originserver.homelab A 127.0.0.13 TXT |
| 19 | 0.252132 | 127.0.0.2 | 127.0.0.1 | DNS | 203 | Standard query response 0xa761 A originserver.homelab A 127.0.0.13 TXT |
| 20 | 38.579504 | 10.0.0.4 | 239.255.255.250 | IGMP… | 36 | Membership Report group 239.255.255.250 |
| 21 | 38.579986 | 10.0.0.4 | 224.0.0.251 | IGMP… | 36 | Membership Report group 224.0.0.251 |
| 22 | 53.579395 | 192.168.171.1 | 192.168.171.1 | ICMP | 128 | Destination unreachable (Host unreachable) |

# CLIENT - ip.addr == (proxy’s ip):

הלקוח שלנו זה המשתמש שמתחבר לאתר האינטרנט,
לכן התקשורת מתבצעת באמצעות פרוטוקול HTTP ופרוטוקול התעבורה הוא TCP.
בצילום ניתן לראות את בקשות המשתמש עבור האתר והתוכן שלו.

| | | | | | | |
|---|---|---|---|---|---|---|
| 3 | 0.049522 | 127.0.0.30 | 127.0.0.1 | HTTP | 5288 | HTTP/1.1 200 OK (text/html) |
| 4 | 0.105387 | 127.0.0.1 | 127.0.0.30 | HTTP | 699 | GET /static/style.css HTTP/1.1 |
| 5 | 0.106131 | 127.0.0.1 | 127.0.0.30 | HTTP | 715 | GET /stream/test.mp4?force_quality=auto HTTP/1.1 |
| 6 | 0.359661 | 127.0.0.30 | 127.0.0.1 | TCP | 288 | 5000 → 56538 [PSH, ACK] Seq=5380 Ack=1466 Win=250 Len=244 [TCP PDU reassembled |
| 7 | 0.361979 | 127.0.0.30 | 127.0.0.1 | HTTP | 3572 | HTTP/1.1 200 OK (text/css) |
| 8 | 0.526259 | 127.0.0.1 | 127.0.0.30 | HTTP | 740 | GET /favicon.ico HTTP/1.1 |
| 9 | 0.527294 | 127.0.0.30 | 127.0.0.1 | TCP | 176 | 5000 → 56538 [PSH, ACK] Seq=9152 Ack=2162 Win=247 Len=132 [TCP PDU reassembled |
| 10 | 0.527731 | 127.0.0.30 | 127.0.0.1 | HTTP… | 66 | HTTP/1.1 404 Not Found , JSON (application/json) |

דוגמה לבקשה פשוטה, במקרה שלנו להחליף את סיכויי הPACKET LOSS.
לאחר שינוי הערך יתבצע חיבור חדש לאתר ובסופו תישלח בקשת GET.
ומכניס אותי לאתר עם ערכים חדשים עבור הPACKET LOSS שבחרנו. כמובן שהכל מתבצע עם TCP אז יש ACK עבור כל חבילה.

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.1 | 127.0.0.31 | TCP | 56 | 49240 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 2 | 0.000332 | 127.0.0.31 | 127.0.0.1 | TCP | 56 | 5000 → 49240 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 3 | 0.000397 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49240 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |
| 4 | 0.002687 | 127.0.0.1 | 127.0.0.31 | HTTP | 962 | POST /set_loss HTTP/1.1 (application/x-www-form-urlencoded) |
| 5 | 0.002749 | 127.0.0.31 | 127.0.0.1 | TCP | 44 | 5000 → 49240 [ACK] Seq=1 Ack=919 Win=64512 Len=0 |
| 6 | 0.005512 | 127.0.0.1 | 127.0.0.31 | TCP | 56 | 62510 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 7 | 0.005754 | 127.0.0.31 | 127.0.0.1 | TCP | 56 | 5000 → 62510 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 8 | 0.005819 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 62510 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |
| 9 | 0.010264 | 127.0.0.31 | 127.0.0.1 | HTTP | 156 | HTTP/1.1 303 See Other |
| 10 | 0.010390 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49240 → 5000 [ACK] Seq=919 Ack=113 Win=65280 Len=0 |
| 11 | 0.016378 | 127.0.0.1 | 127.0.0.31 | HTTP | 838 | GET / HTTP/1.1 |
| 12 | 0.016450 | 127.0.0.31 | 127.0.0.1 | TCP | 44 | 5000 → 49240 [ACK] Seq=113 Ack=1713 Win=63744 Len=0 |
| 13 | 0.023629 | 127.0.0.31 | 127.0.0.1 | TCP | 179 | 5000 → 49240 [PSH, ACK] Seq=113 Ack=1713 Win=63744 Len=135 [TCP PDU reassembled |
| 14 | 0.023732 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49240 → 5000 [ACK] Seq=1713 Ack=248 Win=65280 Len=0 |
| 15 | 0.024237 | 127.0.0.31 | 127.0.0.1 | HTTP | 5501 | HTTP/1.1 200 OK (text/html) |
| 16 | 0.024374 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49240 → 5000 [ACK] Seq=1713 Ack=5705 Win=59648 Len=0 |
| 17 | 5.031662 | 127.0.0.31 | 127.0.0.1 | TCP | 44 | 5000 → 49240 [FIN, ACK] Seq=5705 Ack=1713 Win=63744 Len=0 |
| 18 | 5.031754 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49240 → 5000 [ACK] Seq=1713 Ack=5706 Win=59648 Len=0 |

הפעלת סרטון: PROXY →Client

בצילום ניתן לראות את התקשורת בין הפרוקסי למשתמש
תישלח בקשת GET מצד המשתמש לקבלת הסרטון
ולאחר מכן תתחיל העברת מידע עד לסוף הסרטון, הסרטון יחולק למספר רב של חבילות

| | | | | | | |
|---|---|---|---|---|---|---|
| 5 | 1.565765 | 127.0.0.1 | 127.0.0.31 | HTTP | 812 | GET /play/movie.mp4 HTTP/1.1 |
| 6 | 1.565835 | 127.0.0.31 | 127.0.0.1 | TCP | 44 | 5000 → 50488 [ACK] Seq=1 Ack=769 Win=252 Len=0 |
| 7 | 1.572193 | 127.0.0.31 | 127.0.0.1 | TCP | 179 | 5000 → 50488 [PSH, ACK] Seq=1 Ack=769 Win=252 Len=135 [TCP PDU reassembled |
| 8 | 1.572287 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=769 Ack=136 Win=255 Len=0 |
| 9 | 1.572789 | 127.0.0.31 | 127.0.0.1 | HTTP | 5365 | HTTP/1.1 200 OK (text/html) |
| 10 | 1.572889 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=769 Ack=5457 Win=234 Len=0 |
| 11 | 1.581381 | 127.0.0.1 | 127.0.0.31 | TCP | 56 | 49672 → 5000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 12 | 1.581784 | 127.0.0.31 | 127.0.0.1 | TCP | 56 | 5000 → 49672 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_ |
| 13 | 1.581902 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 49672 → 5000 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |
| 14 | 1.608922 | 127.0.0.1 | 127.0.0.31 | HTTP | 674 | GET /stream/movie.mp4?force_quality=auto HTTP/1.1 |
| 15 | 1.608988 | 127.0.0.31 | 127.0.0.1 | TCP | 44 | 5000 → 50488 [ACK] Seq=5457 Ack=1399 Win=250 Len=0 |
| 16 | 1.616169 | 127.0.0.31 | 127.0.0.1 | TCP | 215 | 5000 → 50488 [PSH, ACK] Seq=5457 Ack=1399 Win=250 Len=171 |
| 17 | 1.616286 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=5628 Win=234 Len=0 |
| 18 | 1.635249 | 127.0.0.31 | 127.0.0.1 | TCP | 59948 | 5000 → 50488 [ACK] Seq=5628 Ack=1399 Win=250 Len=59904 |
| 19 | 1.635476 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=65532 Win=255 Len=0 |
| 20 | 1.635531 | 127.0.0.31 | 127.0.0.1 | TCP | 5420 | 5000 → 50488 [PSH, ACK] Seq=65532 Ack=1399 Win=250 Len=5376 |
| 21 | 1.635597 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=70908 Win=234 Len=0 |
| 22 | 1.636905 | 127.0.0.31 | 127.0.0.1 | TCP | 59948 | 5000 → 50488 [ACK] Seq=70908 Ack=1399 Win=250 Len=59904 |
| 23 | 1.637056 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=130812 Win=255 Len=0 |
| 24 | 1.637111 | 127.0.0.31 | 127.0.0.1 | TCP | 65324 | 5000 → 50488 [ACK] Seq=130812 Ack=1399 Win=250 Len=65280 |
| 25 | 1.637296 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=196092 Win=511 Len=0 |
| 26 | 1.637341 | 127.0.0.31 | 127.0.0.1 | TCP | 5932 | 5000 → 50488 [PSH, ACK] Seq=196092 Ack=1399 Win=250 Len=5888 |
| 27 | 1.637404 | 127.0.0.1 | 127.0.0.31 | TCP | 44 | 50488 → 5000 [ACK] Seq=1399 Ack=201980 Win=488 Len=0 |
| 28 | 1.638423 | 127.0.0.31 | 127.0.0.1 | TCP | 259 | 5000 → 50488 [PSH, ACK] Seq=201980 Ack=1399 Win=250 Len=215 |

# SERVER → PROXY:

## RUDP - udp && !dns:

בהתחלה יש מספר הודעות עם אורך קטן, תפקידן לשלוח את ה-META DATA של הקובץ (הדבר מאפשר לנו לאתחל את הנגן, להציג את האפשרויות לבחירת האיכות ולהציג את אורך הסרטון).
לאחר מכן יישלח את הקובץ עצמו ועבור כל הודעה שנשלחה יש הודעה של ACK (ההודעות בגודל 45).
(מראה את האמינות ושימוש בSLIDING WINDOW עבור RUDP)

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.1 | 127.0.0.13 | UDP | 49 | 50476 → 9000 Len=17 |
| 2 | 0.004107 | 127.0.0.1 | 127.0.0.1 | UDP | 52 | 50477 → 50476 Len=20 |
| 3 | 0.004649 | 127.0.0.1 | 127.0.0.1 | UDP | 52 | 50477 → 50476 Len=20 |
| 4 | 0.004759 | 127.0.0.1 | 127.0.0.1 | UDP | 52 | 50477 → 50476 Len=20 |
| 5 | 0.004845 | 127.0.0.1 | 127.0.0.1 | UDP | 52 | 50477 → 50476 Len=20 |
| 6 | 0.004921 | 127.0.0.1 | 127.0.0.1 | UDP | 52 | 50477 → 50476 Len=20 |
| 7 | 0.053645 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 8 | 0.053744 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 9 | 0.053794 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 10 | 0.053835 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 11 | 0.053905 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 12 | 0.054006 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 13 | 0.054069 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 14 | 0.054128 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 15 | 0.067482 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 16 | 0.067795 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 17 | 0.067935 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 18 | 0.067941 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 19 | 0.068060 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 20 | 0.068156 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 21 | 0.068258 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 22 | 0.068350 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 23 | 0.068524 | 127.0.0.1 | 127.0.0.1 | UDP | 1438 | 50477 → 50476 Len=1406 |
| 24 | 0.068736 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 25 | 0.069360 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 26 | 0.069885 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 27 | 0.070510 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |
| 28 | 0.071135 | 127.0.0.1 | 127.0.0.1 | UDP | 45 | 50476 → 50477 Len=13 |

## TCP - tcp.port == 9000:

התמונה מציגה ניתוח תעבורת רשת ב-Wireshark המדגים את מנגנון בקרת הזרימה של פרוטוקול TCP בין שרת לקוח. ניתן לראות כי השרת שולח נתונים במהירות הממלאת את המכסה שהוקצתה לו, מצב המכונה "TCP Window Full", מה שמאלץ אותו להפסיק את השידור ולהמתין לאישור (ACK) מהלקוח. בשלב מסוים, המאגר (buffer) של הלקוח מתמלא לחלוטין והוא שולח הודעת "TCP ZeroWindow", המורה לשרת לעצור את שליחת הנתונים באופן מיידי כדי למנוע אובדן מידע. רק לאחר שהלקוח מעבד את הנתונים ומתפנה מקום במאגר, נשלחת הודעת "TCP Window Update" המאפשרת לחדש את הזרמת הווידאו.

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.1 | 127.0.0.13 | TCP | 56 | 49673 → 9000 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM |
| 2 | 0.000615 | 127.0.0.13 | 127.0.0.1 | TCP | 56 | 9000 → 49673 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_ |
| 3 | 0.000759 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=1 Ack=1 Win=65280 Len=0 |
| 4 | 0.001264 | 127.0.0.1 | 127.0.0.13 | TCP | 63 | 49673 → 9000 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=19 |
| 5 | 0.001391 | 127.0.0.13 | 127.0.0.1 | TCP | 44 | 9000 → 49673 [ACK] Seq=1 Ack=20 Win=65280 Len=0 |
| 6 | 0.003596 | 127.0.0.13 | 127.0.0.1 | TCP | 58 | 9000 → 49673 [PSH, ACK] Seq=1 Ack=20 Win=65280 Len=14 |
| 7 | 0.003727 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=15 Win=65280 Len=0 |
| 8 | 0.016096 | 127.0.0.13 | 127.0.0.1 | TCP | 65324 | [TCP Window Full] 9000 → 49673 [ACK] Seq=15 Ack=20 Win=65280 Len=65280 |
| 9 | 0.016338 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=65295 Win=65280 Len=0 |
| 10 | 0.016405 | 127.0.0.13 | 127.0.0.1 | TCP | 65324 | [TCP Window Full] 9000 → 49673 [ACK] Seq=65295 Ack=20 Win=65280 Len=65280 |
| 11 | 0.016547 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=130575 Win=130816 Len=0 |
| 12 | 0.016580 | 127.0.0.13 | 127.0.0.1 | TCP | 556 | 9000 → 49673 [PSH, ACK] Seq=130575 Ack=20 Win=65280 Len=512 |
| 13 | 0.016625 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=131087 Win=130304 Len=0 |
| 14 | 0.017220 | 127.0.0.13 | 127.0.0.1 | TCP | 65539 | 9000 → 49673 [ACK] Seq=131087 Ack=20 Win=65280 Len=65495 |
| 15 | 0.017424 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=196582 Win=65024 Len=0 |
| 16 | 0.018248 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | [TCP Window Update] 49673 → 9000 [ACK] Seq=20 Ack=196582 Win=130816 Len=0 |
| 17 | 0.020641 | 127.0.0.13 | 127.0.0.1 | TCP | 65539 | 9000 → 49673 [ACK] Seq=196582 Ack=20 Win=65280 Len=65495 |
| 18 | 0.020709 | 127.0.0.13 | 127.0.0.1 | TCP | 65365 | [TCP Window Full] 9000 → 49673 [PSH, ACK] Seq=262077 Ack=20 Win=65280 Len= |
| 19 | 0.020947 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=327398 Win=130816 Len=0 |
| 20 | 0.021035 | 127.0.0.13 | 127.0.0.1 | TCP | 65539 | 9000 → 49673 [ACK] Seq=327398 Ack=20 Win=65280 Len=65495 |
| 21 | 0.021078 | 127.0.0.13 | 127.0.0.1 | TCP | 382 | 9000 → 49673 [PSH, ACK] Seq=392893 Ack=20 Win=65280 Len=338 |
| 22 | 0.021194 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=393231 Win=65024 Len=0 |
| 23 | 0.021764 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | [TCP Window Update] 49673 → 9000 [ACK] Seq=20 Ack=393231 Win=130816 Len=0 |
| 24 | 0.021833 | 127.0.0.13 | 127.0.0.1 | TCP | 65539 | 9000 → 49673 [ACK] Seq=393231 Ack=20 Win=65280 Len=65495 |
| 25 | 0.021981 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | 49673 → 9000 [ACK] Seq=20 Ack=458726 Win=65536 Len=0 |
| 26 | 0.022032 | 127.0.0.13 | 127.0.0.1 | TCP | 65539 | 9000 → 49673 [ACK] Seq=458726 Ack=20 Win=65280 Len=65495 |
| 27 | 0.022144 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | [TCP ZeroWindow] 49673 → 9000 [ACK] Seq=20 Ack=524221 Win=0 Len=0 |
| 28 | 0.023885 | 127.0.0.1 | 127.0.0.13 | TCP | 44 | [TCP Window Update] 49673 → 9000 [ACK] Seq=20 Ack=524221 Win=130816 Len=0 |

# DROPPED PACKETS - frame contains “DROP”:

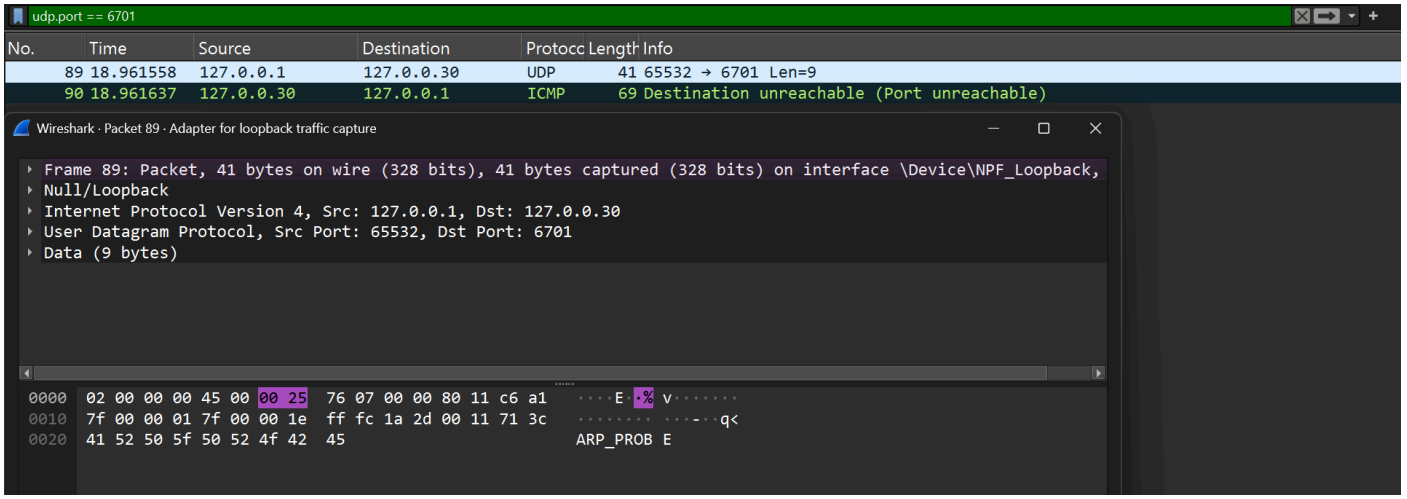
כאשר הודעה נופלת השרת שולח הודעה מסוג "DROP"
שכמו שאפשר לראות זה הודעה בערך בגודל 40 bytes
בעת קבלת ההודעה יוודע לProxy שהחבילה לא הגיעה ליעד, ויוכל לשלוח אותה מחדש
התמונה מתוך נפילת הודעה בשימוש בRUDP.

| | | | | | | |
|---|---|---|---|---|---|---|
| 19 | 0.173005 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 20 | 0.174132 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 21 | 0.178997 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 22 | 0.186686 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 23 | 0.192531 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 24 | 0.193695 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 25 | 0.198947 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 26 | 0.200578 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 27 | 0.216201 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 28 | 0.222979 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 29 | 0.223693 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 30 | 0.232268 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 31 | 0.234503 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 32 | 0.238337 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 33 | 0.239616 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 34 | 0.240662 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 35 | 0.248848 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 36 | 0.249507 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 37 | 0.249794 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 38 | 0.288587 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 39 | 0.318273 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 40 | 0.402647 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 41 | 0.500185 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 42 | 0.867942 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 43 | 0.868830 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 44 | 0.894342 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 45 | 0.919315 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |
| 46 | 1.030002 | 127.0.0.1 | 127.0.0.1 | UDP | 40 | 59883 → 59882 Len=8 |

בדומה - אם משתמשים בפורטוקול TCP
השגיאה המרכזית שתוצג היא: [TCP Previous segment not captured], מעידה על כך שהלקוח קיבל חבילות עם מספרי רצף מתקדמים מבלי שקיבל את החבילות שאמורות להופיע לפניהן, מה שיוצר "חורים" במידע. בנוסף, מופיעות חבילות המסומנות כ-Malformed Packet, המצביעות על נתונים שהגיעו פגומים או שלא פוענחו כראוי לפי הפרוטוקול. התוצאה של איבוד ההודעות המכוון הזה היא עצירה של זרימת המידע התקינה, שכן הפרוטוקול נאלץ להמתין לשידורים חוזרים כדי

להשלים את החסר לפני שיוכל להעביר את הנתונים לאפליקציה.

| | | | | | | |
|---|---|---|---|---|---|---|
| 1 | 0.000000 | 127.0.0.13 | 127.0.0.1 | TCP | 146 | 9000 → 55144 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=102 |
| 2 | 0.004922 | 127.0.0.30 | 127.0.0.1 | RSL | 65324 | [Malformed Packet] |
| 3 | 0.053554 | 127.0.0.13 | 127.0.0.1 | TCP | 1048 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=1703055 Ack=1 W |
| 4 | 0.074342 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=1710142 Ack=1 Win=25 |
| 5 | 0.103460 | 127.0.0.13 | 127.0.0.1 | TCP | 1938 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=2750761 Ack=1 W |
| 6 | 0.118576 | 127.0.0.30 | 127.0.0.1 | RSL | 65539 | [TCP Previous segment not captured] [Malformed Packet] |
| 7 | 0.160351 | 127.0.0.13 | 127.0.0.1 | TCP | 146 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=4980797 Ack=1 W |
| 8 | 0.178950 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=4986942 Ack=1 Win=25 |
| 9 | 0.309204 | 127.0.0.13 | 127.0.0.1 | TCP | 64 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=8913059 Ack=1 W |
| 10 | 0.604310 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=8918835 Ack=1 Win=25 |
| 11 | 0.626075 | 127.0.0.13 | 127.0.0.1 | TCP | 64 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=10617015 Ack=1 |
| 12 | 0.762329 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=10622934 Ack=1 Win=2 |
| 13 | 0.775690 | 127.0.0.13 | 127.0.0.1 | TCP | 146 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=11796601 Ack=1 |
| 14 | 0.782452 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=11802664 Ack=1 Win=2 |
| 15 | 0.784448 | 127.0.0.13 | 127.0.0.1 | TCP | 64 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=12320991 Ack=1 |
| 16 | 0.791400 | 127.0.0.13 | 127.0.0.1 | TCP | 146 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=12452001 Ack=1 |
| 17 | 0.796475 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=12326952 Ack=1 Win=2 |
| 18 | 0.798949 | 127.0.0.30 | 127.0.0.1 | RSL | 65539 | [TCP Previous segment not captured] UNIT DATA INDication (DTAP) (GTTP) unknown |
| 19 | 0.906559 | 127.0.0.13 | 127.0.0.1 | TCP | 64 | [TCP Previous segment not captured] 9000 → 55144 [PSH, ACK] Seq=14024967 Ack=1 |
| 20 | 0.918369 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | [TCP Previous segment not captured] 5000 → 53242 [ACK] Seq=14030806 Ack=1 Win=2 |
| 21 | 25.871699 | 127.0.0.13 | 127.0.0.1 | TCP | 64 | 9000 → 56536 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=20 |
| 22 | 25.880823 | 127.0.0.30 | 127.0.0.1 | TCP | 65539 | 5000 → 59436 [ACK] Seq=1 Ack=1 Win=250 Len=65495 |
| 23 | 29.800520 | 127.0.0.13 | 127.0.0.1 | TCP | 146 | 9000 → 56538 [PSH, ACK] Seq=1 Ack=1 Win=255 Len=102 |

# ARP:

נשלחת בקשת ARP_PROB על מנת להיות בטוחים שאין עוד מכשיר בשרת עם הכתובת IP שאנחנו רוצים לחלק מהDHCP.